Shift-Invariance and Adversarial Robustness

A Collaborator Primer

background from first principles, the questions before the answers, and full proofs

Anass Al Ammiri

June 22, 2026

A standalone booklet for a new collaborator. It assumes only undergraduate linear algebra, real analysis, basic calculus, and probability, and builds everything else from first principles. Part I teaches the background, Part II poses the open problems (before any answers), and Part III proves the results. Do the exercises; attempt the Part II problems before reading Part III.

Contents

1 Why this project exists

Convolutional networks are built to be (approximately) invariant to small spatial shifts, and for ordinary accuracy this is a virtue. Whether it is a virtue for *adversarial* robustness, the worst-case behaviour under tiny crafted perturbations, is genuinely unsettled: published work points both ways. Ge et al. (2021) prove a fully shift-invariant classifier can be far less robust than a plain one; Kamath et al. (2021) report a spatial-versus-adversarial trade-off; Galloway et al. (2019) blame a standard invariance-promoting ingredient (batch normalisation) for vulnerability; while a line of architectural work (Zhang, 2019; Saha and Gokhale, 2024) improves shift behaviour and reports robustness *gains*. These are not all wrong; they are measuring different things. The thread of this project, and of this booklet, is that one scalar, the margin of the surviving invariant signal divided by its sensitivity, reconciles them, and that a careful capacity-matched experiment under a strong attack tells the rest of the story. The full chain that produced the project's questions, starting from a practical course that reproduced and then broke the clean story, is the subject of Part II; the rest of this section just fixes expectations.

How to read this. Part I is the background you need and nothing more, in dependency order, with a worked example and an exercise per topic. Part II is the harder skill: given the situation, generating and choosing the research questions yourself, so it deliberately withholds the answers. Part III gives the answers with full proofs, each labelled honestly as classical, a rephrasing, or a genuine new contribution, and each tied back to the question it settles.

Part I: Background from first principles

This part assumes only linear algebra, real analysis, basic calculus, and probability. It builds, in order, the robustness vocabulary, the invariance and Fourier machinery, and the learning theory the project rests on. Every term is defined on first use, every central claim is proved in full, and each topic ends with a worked example and an exercise. Read it in order; do the exercises.

2 Adversarial robustness from scratch

This section answers one question: *what does it mean for a classifier to be robust, and why is the obvious thing — high test accuracy — not enough?* We follow the order that worked for the three undergraduates who got lost in the previous draft: first a surprising concrete demonstration, then just enough notation to talk about it, and only at the end the formal optimization picture. This is the order of the Kolter–Madry tutorial (Kolter and Madry, 2018), which is the spine of this whole section.

2.1 Demo first: a pig that the classifier thinks is a wombat

Imagine you have a state-of-the-art image classifier — a fixed function that takes an image and outputs a label. You feed it a clear photo of a pig. It says "pig" with 99.6% confidence. Good. Now you add to that photo a tiny amount of carefully chosen noise: so tiny that, pixel by pixel, no human eye can tell the new image apart from the old one. You feed in the new image. The same classifier now says "wombat" with 99.98% confidence (Kolter and Madry, 2018). With slightly different noise it says "airliner." The pictures look identical to you; the classifier's answer flipped completely.

This is not a bug in one model or one photo. It happens to essentially every modern image classifier, on essentially every image, and the noise needed is astonishingly small. The phenomenon was discovered by Szegedy et al. (2014), who called these crafted inputs *adversarial examples*. The rest of this section builds the language to say precisely what just happened and how we measure it.

Why should you care? Two reasons, both from Kolter and Madry (2018). First, the obvious security reason: if a system is deployed where someone has an incentive to fool it (spam filters, malware detectors, a camera that reads stop signs for a self-driving car), then "works most of the time" is not the right standard; we want "works even against someone deliberately trying to break it." Second, the deeper reason: a model that can be flipped from "pig" to "airliner" by invisible noise clearly has *not* learned the concept "pig" the way a human has. Adversarial examples expose that high test accuracy can hide a brittle, shallow understanding of the data.

2.2 The minimum notation we need

Definition 2.1 (Model / hypothesis)**.** A *model* (or *hypothesis*) is a function $h_\theta : \mathcal{X} \to \mathbb{R}^k$. It takes an input x from the input space $\mathcal{X}$ (for images, a long vector of pixel intensities) and outputs a vector of k real numbers, one per class. The subscript θ collects all the model's tunable numbers, called *parameters* (for a neural network these are all its weight matrices and biases). We *train* a model by choosing θ; once trained, θ is frozen and we study x.

The k output numbers are called *logits*.

Definition 2.2 (Logit, softmax, predicted class)**.** The components $h_\theta(x)_1, \dots, h_\theta(x)_k$ are the *logits*: real numbers, possibly negative, one per class. The predicted class is the index of the largest logit, $\arg\max_i h_\theta(x)_i$. To turn logits into probabilities one applies the *softmax* map $\sigma : \mathbb{R}^k \to \mathbb{R}^k$,

$$\sigma(z)_j \;=\; \frac{e^{z_j}}{\sum_{i=1}^{k} e^{z_i}},$$

which is positive and sums to 1. Bigger logit $\Rightarrow$ bigger probability; the arg max is unchanged by softmax, so for deciding the label we can work with the raw logits and ignore softmax.

Definition 2.3 (Loss)**.** A *loss function* $\ell(h_\theta(x), y)$ takes the model's logit vector and the true class index $y \in \{1, \dots, k\}$ and returns a single nonnegative number measuring how wrong the prediction is: small loss means the model put high probability on the true class. The standard choice is the *cross-entropy* (or *softmax*) loss

$$\ell(h_\theta(x), y) \;=\; -\log \sigma\big(h_\theta(x)\big)_y \;=\; -\,h_\theta(x)_y + \log \sum_{i=1}^{k} e^{h_\theta(x)_i},$$

i.e. the negative log-probability assigned to the true class. In the pig example the loss of the correct "pig" label was about 0.0039, which corresponds to probability $e^{-0.0039} \approx 0.996$ (Kolter and Madry, 2018).

Gradient with respect to the input. The single technical tool that makes adversarial examples possible is the *gradient of the loss with respect to the input*, written $\nabla_x \ell(h_\theta(x), y)$. Recall from calculus that for a scalar function ℓ of a vector $x \in \mathbb{R}^n$, the gradient $\nabla_x \ell$ is the vector of partial derivatives $\big(\partial \ell / \partial x_1, \dots, \partial \ell / \partial x_n\big)$; it points in the direction in input space along which the loss increases fastest, and its i-th entry tells you how much the loss changes if you nudge pixel i. Crucially, *normal* training computes the gradient with respect to the parameters θ (to improve the model); here we keep θ fixed and differentiate with respect to the *input* x (to attack the model). Both are computed by the same mechanical procedure (backpropagation); for our purposes you may treat $\nabla_x \ell$ as a vector you can evaluate at any point.

2.3 An adversarial example is an inner maximization

Training minimizes the loss over θ. An attacker does the opposite: keep the model fixed and *change the input to make the loss large*. In the words of Kolter and Madry (2018): "instead of adjusting the image to minimize the loss... we're going to adjust the image to maximize the loss."

We cannot let the attacker change the image arbitrarily — turning a pig into an actual photo of a wombat would not be impressive. So we restrict the change. Write the perturbation as δ and require it to lie in an allowed set Δ:

$$\max_{\delta \in \Delta} \; \ell\big(h_\theta(x + \delta),\, y\big). \tag{1}$$

Equation (1) is the heart of the whole field. In words: *over all allowed perturbations δ, what is the largest loss the attacker can force, and which δ achieves it?* A solution $\delta^\star$ is an adversarial perturbation, and $x + \delta^\star$ is an adversarial example. The notation $\max_{\delta \in \Delta}$ means the maximum is taken over the variable δ ranging in Δ; the result is a number (the worst-case loss), and the maximizer is the worst-case perturbation.

2.4 Threat models: how big a change is "allowed"?

The set Δ is called the *threat model*: it formalizes the attacker's power. The most common choice is a *norm ball*: all perturbations whose size, measured by some norm $\|\cdot\|$, is at most a budget ϵ. We fix conventions once: δ is the perturbation, ϵ is the radius, and

$$\Delta \;=\; \{\,\delta : \|\delta\| \le \epsilon\,\}.$$

Two norms dominate the literature; both are worth defining carefully because students confuse them.

Definition 2.4 (ℓ_∞ norm)**.** For a vector $z = (z_1, \ldots, z_n) \in \mathbb{R}^n$,

$$\|z\|_\infty \;=\; \max_{1 \le i \le n} |z_i|.$$

It is the largest single coordinate in absolute value. The constraint $\|\delta\|_\infty \le \epsilon$ says *every* pixel may move by at most ϵ, independently. Example: if $\epsilon = 0.1$ and an image has 784 pixels, an ℓ_∞ attack may shift each of the 784 pixels by up to 0.1 at once.

Definition 2.5 (ℓ_2 norm)**.** For $z \in \mathbb{R}^n$,

$$\|z\|_2 \;=\; \sqrt{z_1^2 + z_2^2 + \cdots + z_n^2}\,,$$

the ordinary Euclidean length. The constraint $\|\delta\|_2 \le \epsilon$ caps the *total* energy of the perturbation, so the attacker can spend a big change on a few pixels if it spends little elsewhere. Example: $\delta = (0.3, 0, \ldots, 0)$ and $\delta' = (0.1, 0.1, 0.1, 0, \ldots)$ have very different "shapes" but the first has $\|\delta\|_2 = 0.3$ while the second has $\|\delta'\|_2 = \sqrt{0.03} \approx 0.173$.

Why ℓ_∞ is the workhorse. Kolter and Madry (2018) explain its appeal: for small ϵ an ℓ_∞ perturbation "add[s] such a small component to each pixel. . . that they are visually indistinguishable from the original image." It is a clean, if crude, stand-in for "looks the same to a human." The two norms are also numerically different in scale: the ℓ_2 budget that matches an ℓ_∞ budget of $\epsilon = 0.1$ on a 784-pixel image is about $\frac{\sqrt{2 \cdot 784}}{\sqrt{\pi e}} \cdot 0.1 \approx 1.35$, i.e. much larger numerically, because in high dimensions a per-coordinate cap of 0.1 over 784 coordinates buys a lot of Euclidean length (Kolter and Madry, 2018). This is the first hint that *high dimension is the attacker's friend* — a theme we return to with FGSM in Section 4.

2.5 Ordinary risk versus adversarial risk

So far we attacked one input. To talk about a model's overall robustness we average over the data distribution $\mathcal{D}$ (the unknown distribution from which real inputs are drawn).

Definition 2.6 (Risk and adversarial risk)**.** The ordinary *risk* of a model is its expected loss,

$$R(h_\theta) \;=\; \mathbb{E}_{(x,y)\sim\mathcal{D}}\big[\ell(h_\theta(x), y)\big].$$

The *adversarial* (or *robust*) *risk* replaces each point's loss by the worst-case loss in its threat set:

$$R_{\text{adv}}(h_\theta) \;=\; \mathbb{E}_{(x,y)\sim\mathcal{D}}\Big[\max_{\delta \in \Delta(x)} \ell\big(h_\theta(x+\delta), y\big)\Big].$$

Here $\Delta(x)$ is allowed to depend on x (e.g. so that $x + \delta$ stays a valid image in $[0,1]^n$). In practice $\mathcal{D}$ is unknown, so we estimate both by averaging over a finite held-out test set.

Ordinary risk asks "how often is the model right on typical inputs?"; adversarial risk asks "how often is the model right *even when each input is nudged adversarially within budget* ϵ?" A model can have tiny ordinary risk and huge adversarial risk — exactly the pig situation.

2.6 Two ways to report robustness: accuracy at a radius vs. robust radius

There are two natural numbers to report, and keeping them apart is essential.

Definition 2.7 (Robust accuracy at radius ϵ)**.** Fix a radius ϵ. A test point x (true label y) is *robustly correct at radius* ϵ if the model classifies $x+\delta$ correctly for *every* δ with $\|\delta\| \le \epsilon$. The *robust accuracy at radius* ϵ is the fraction of test points that are robustly correct:

$$\mathrm{RobAcc}(\epsilon) \;=\; \frac{1}{N}\sum_{x} \mathbf{1}\big[x \text{ is classified correctly for all } \|\delta\| \le \epsilon\big],$$

where N is the number of test points and $\mathbf{1}[\cdot]$ is 1 when the bracket is true and 0 otherwise. "Robust error at radius ϵ" is one minus this. Read it as: *the fraction of points that survive every attack of size up to* ϵ*.*

Definition 2.8 (Robust radius)**.** The *robust radius* of a point x is the size of the *smallest* perturbation that changes the model's prediction,

$$r(x) \;=\; \min\{\, \|\delta\| : h_\theta(x+\delta) \text{ is misclassified} \,\}.$$

It is the distance from x to the nearest decision boundary, measured in the chosen norm. A large $r(x)$ means x sits comfortably inside its class; $r(x) = 0$ would mean x is already on the boundary.

These two are duals of each other. A point x survives radius ϵ exactly when its robust radius exceeds ϵ, so

$$\mathrm{RobAcc}(\epsilon) \;=\; \frac{1}{N}\sum_{x} \mathbf{1}\big[\, r(x) > \epsilon \,\big], \tag{2}$$

i.e. *the fraction of points whose robust radius is bigger than* ϵ. Fixing ϵ and reading off the fraction is a single vertical slice through the function $\epsilon \mapsto \mathrm{RobAcc}(\epsilon)$; measuring $r(x)$ for every point recovers the whole curve. We will see in Section 4 that the two families of attacks correspond exactly to these two views.

2.7 Certificate (lower bound) versus attack (upper bound)

We can never enumerate all δ, so for any single point we work with bounds on $r(x)$, and there are two opposite kinds.

Definition 2.9 (Attack and certificate)**.** An *attack* exhibits a concrete adversarial perturbation δ with $\|\delta\| \le \epsilon$ that flips the label. It *proves the model is not robust at radius* $\|\delta\|$, hence gives an *upper bound* on the robust radius: $r(x) \le \|\delta\|$. A *certificate* is a mathematical *guarantee* that *no* perturbation of size $\le \rho$ can flip the label; it gives a *lower bound* $r(x) \ge \rho$, i.e. the model is provably safe out to radius ρ.

The two bound $r(x)$ from opposite sides:

$$\underbrace{\rho}_{\text{certificate, lower bound}} \;\le\; r(x) \;\le\; \underbrace{\|\delta\|}_{\text{attack, upper bound}} .$$

An attack can only ever say "robustness is at most this"; a certificate can only ever say "robustness is at least this." Beating one does not give the other. Sections 4 and 3 are precisely the two sides: attacks (upper bounds) and Lipschitz certificates (lower bounds).

2.8 Worked example: the linear MNIST 0-vs-1 robustness story

We now make the accuracy–robustness gap concrete and hand-checkable, following the linear-model chapter of Kolter and Madry (2018). Consider a *binary* linear classifier on the MNIST handwritten digits, restricted to the two classes "0" and "1." Each image is a vector $x \in \mathbb{R}^{784}$ (a 28×28 grid flattened). The model is

$$h_{w,b}(x) \;=\; w^\top x + b, \qquad w \in \mathbb{R}^{784},\; b \in \mathbb{R},$$

predicting class $+1$ when $w^\top x + b > 0$ and class -1 otherwise (we use labels $y \in \{+1, -1\}$ here). The binary cross-entropy (logistic) loss is $\ell = L\big(y\,(w^\top x + b)\big)$ with $L(z) = \log(1 + e^{-z})$, which is decreasing in z.

Worked example 2.10 (The 0-vs-1 numbers)**.** Kolter and Madry (2018) train this model and report:

- **Clean.** Test error about 0.04% — the model gets all but one test digit right. Easy problem.
- **Attacked at $\epsilon = 0.2$ (ℓ_∞).** Allowing each pixel to move by up to 0.2, the test error jumps to **84.5**%. The same model that was essentially perfect is now wrong on five out of six images, under perturbations that do not fool a human.
- **Robustly trained, then attacked at $\epsilon = 0.2$.** Retraining the model to optimize the *adversarial* risk (Section 2.5) drops the worst-case error to **2.5**% robust error, at the cost of clean error rising from 0.04% to about 0.3%.

Two lessons. (1) Standard training, even of a simple linear model on a trivial task, is wildly non-robust. (2) Robustness is buyable but not free: the $0.04\% \to 0.3\%$ rise in clean error is a small instance of the *accuracy–robustness trade-off* that recurs throughout the field (Kolter and Madry, 2018).

Why is the linear case special and worth memorizing? Because for it the inner maximization (1) can be solved *exactly* in closed form, which the next subsection's exercise asks you to do. (For deep networks it cannot, which is why we need iterative attacks in Section 4.)

2.9 Exercise: the worst-case ℓ_∞ perturbation of a linear model

Exercise. Let $h_{w,b}(x) = w^\top x + b$ be a two-class linear model that currently classifies a point x correctly as class $+1$, so its *score* $s(x) = w^\top x + b$ is positive. An attacker may add any δ with $\|\delta\|_\infty \le \epsilon$ and wants to *lower* the score (toward the wrong side of the boundary). Show that the worst-case perturbation is

$$\delta^\star \;=\; -\,\epsilon\,\operatorname{sign}(w),$$

where $\operatorname{sign}(w)$ is the vector of signs of the entries of w ($+1, -1, 0$ entrywise), and that the resulting score drop is exactly

$$s(x) - s(x + \delta^\star) \;=\; \epsilon\,\|w\|_1, \qquad \|w\|_1 = \sum_{i=1}^{n} |w_i|.$$

Solution. The new score is $s(x+\delta) = w^\top(x+\delta) + b = (w^\top x + b) + w^\top\delta = s(x) + w^\top\delta$, so lowering the score means making $w^\top\delta$ as *negative* as possible, subject to $\|\delta\|_\infty \le \epsilon$. Write the dot product coordinatewise:

$$w^\top\delta \;=\; \sum_{i=1}^{n} w_i\,\delta_i.$$

Each term $w_i\delta_i$ is minimized independently because the constraint $|\delta_i| \le \epsilon$ couples nothing across coordinates. For a fixed w_i, the most negative value of $w_i\delta_i$ over $|\delta_i| \le \epsilon$ is $-\epsilon|w_i|$, attained by choosing $\delta_i = -\epsilon$ when $w_i > 0$ and $\delta_i = +\epsilon$ when $w_i < 0$, i.e. $\delta_i = -\epsilon\,\mathrm{sign}(w_i)$. Summing the per-coordinate minima,

$$\min_{\|\delta\|_\infty \le \epsilon} w^\top \delta \;=\; \sum_{i=1}^{n} \big(-\epsilon|w_i|\big) \;=\; -\epsilon \sum_{i=1}^{n} |w_i| \;=\; -\epsilon\,\|w\|_1,$$

achieved at $\delta^\star = -\epsilon\,\mathrm{sign}(w)$. Therefore the score drops by exactly $\epsilon\,\|w\|_1$, as claimed. □

Three things to take away. (i) The optimal $\delta^\star$ does not depend on x at all — the same perturbation attacks every input, which is why the MNIST attack was so cheap. (ii) The drop $\epsilon\|w\|_1$ grows with $\|w\|_1$, foreshadowing why *controlling the norm of the weights* (Sections 3, and the margin theory in later parts) controls robustness. (iii) The quantity $\|w\|_1$ is the *dual norm* of the ℓ_∞ ball: maximizing a linear function $w^\top\delta$ over $\|\delta\|_\infty \le \epsilon$ gives $\epsilon\|w\|_1$. The general fact is $\max_{\|\delta\|_p \le \epsilon} w^\top\delta = \epsilon\|w\|_q$ with $\frac{1}{p} + \frac{1}{q} = 1$ (Kolter and Madry, 2018); the ℓ_∞/ℓ_1 pair is the case $p = \infty, q = 1$, and we meet it again as FGSM's optimality and as Hein's certificate.

2.10 Robust optimization and Danskin's theorem

Putting attacker and defender together gives the *saddle-point* (min–max, or *robust optimization*) problem of Madry et al. (2018), the single organizing equation of robust training:

$$\min_\theta \; \mathbb{E}_{(x,y)\sim\mathcal{D}}\Big[\, \max_{\delta\in\Delta} \ell\big(h_\theta(x+\delta), y\big)\Big]. \tag{3}$$

The *inner* maximization is the attacker (find the worst δ); the *outer* minimization is the defender (choose θ to make even the worst-case loss small). Kolter and Madry (2018) state the unifying slogan plainly: *every adversarial attack and defense is a method for approximately solving the inner maximization and/or the outer minimization*; the right way to describe an attack is by its norm ball plus its optimization method, not by a brand name.

To train by gradient descent on (3) we must differentiate, with respect to θ, a quantity that itself contains a maximization over δ. The justification is Danskin's theorem; we state it and, as instructed, do not prove it.

Theorem 2.11 (Danskin, stated without proof)**.** *Suppose the inner maximization in* (3) *is solved* exactly*, with maximizer $\delta^\star = \delta^\star(\theta)$. Then the gradient with respect to θ of the value* $\max_{\delta\in\Delta} \ell(h_\theta(x+\delta), y)$ *equals the gradient of $\ell(h_\theta(x+\delta), y)$ evaluated at the fixed point $\delta = \delta^\star$:*

$$\nabla_\theta \Big[\max_{\delta\in\Delta} \ell\big(h_\theta(x+\delta), y\big)\Big] \;=\; \nabla_\theta\, \ell\big(h_\theta(x+\delta^\star), y\big)\Big|_{\delta^\star \text{ held fixed}}.$$

In words: to take the gradient of the worst-case loss, first find the worst $\delta^\star$, then differentiate as if $\delta^\star$ were a constant. This is what licenses "adversarial training": repeatedly attack each batch, then take a normal gradient step on the attacked inputs. Kolter and Madry (2018) stress the catch — Danskin requires the inner max to be solved *exactly*, which for deep networks it never is. In practice one solves it "well enough" (a strong attack such as PGD, Section 4) and accepts that the theorem holds only approximately; if the inner attack is weak, a stronger attack can later defeat the supposedly robust model.

3 Lipschitz constants and robustness certificates

Section 2 promised certificates: *provable lower bounds* on the robust radius, the guarantee that no attack within radius ρ can change the label. This section delivers the most important one. The single idea is: *if the model's output cannot change quickly as the input moves, and the input currently sits well inside its class, then a small input change cannot flip the decision.* "Cannot change quickly" is made precise by the Lipschitz constant; "well inside its class" by the margin. We build both from scratch — this material confused even the professor in the last draft, so we go slowly and keep the jargon minimal.

3.1 Lipschitz continuity and the Lipschitz constant

Definition 3.1 (Lipschitz continuity)**.** A function $f : \mathbb{R}^n \to \mathbb{R}^m$ is *Lipschitz continuous* if there is a constant $L \geq 0$ such that

$$\|f(x) - f(y)\| \;\leq\; L\,\|x - y\| \qquad \text{for all } x, y.$$

The smallest such L is the *Lipschitz constant* of f, written $\mathrm{Lip}(f)$ (Scaman and Virmaux, 2018). (Until Section 3.5 take both norms to be ℓ_2.)

The plain-English reading from the Wikipedia article (Wikipedia, n.d.e) is the cleanest: *the absolute value of the slope of the line joining any two points on the graph is at most L.* The output can change by at most L times the input change — L is a speed limit on the function. Three examples fix the idea.

Worked example 3.2 (Three one-dimensional functions)**.**

- $f(x) = |x|$ is 1-Lipschitz: by the reverse triangle inequality $\big||x| - |y|\big| \leq |x - y|$, so $L = 1$ works, and taking $y = 0$ shows no smaller L works. Note f is not differentiable at 0 — Lipschitz does not require differentiability.
- $f(x) = cx$ (a line of slope c) has $\mathrm{Lip}(f) = |c|$: $|cx - cy| = |c|\,|x - y|$ with equality always.
- $f(x) = x^2$ is *not* Lipschitz on all of $\mathbb{R}$: $|x^2 - y^2| = |x + y|\,|x - y|$, and the factor $|x + y|$ is unbounded, so no single finite L caps the slope everywhere (Wikipedia, n.d.e). (It *is* Lipschitz on any bounded interval — this is the difference between a global and a local constant, which matters below.)

For differentiable functions the constant is the supremum of the gradient norm. This is Rademacher's theorem applied to Lipschitz functions; we use it as a computational rule.

Theorem 3.3 (Lipschitz constant via the gradient (Scaman and Virmaux, 2018))**.** *If $f : \mathbb{R}^n \to \mathbb{R}$ is (locally) Lipschitz, then it is differentiable almost everywhere and*

$$\mathrm{Lip}(f) \;=\; \sup_x \|\nabla f(x)\|.$$

For vector-valued $f : \mathbb{R}^n \to \mathbb{R}^m$ the same holds with $\|\nabla f(x)\|$ replaced by the operator norm $\|D_x f\|$ of the Jacobian (defined next).

This already connects to robustness: a small Lipschitz constant means the output cannot move far when the input is nudged, which is exactly the kind of stability an adversarial example tries to violate (Tsuzuku et al., 2018).

3.2 The spectral norm: the Lipschitz constant of a linear map

The most important Lipschitz constant is for the simplest function, a linear map $f(x) = Wx$.

Definition 3.4 (Operator / spectral norm)**.** For a matrix $W \in \mathbb{R}^{m\times n}$, the *operator norm* (also *spectral norm*) is the largest factor by which W can stretch a unit vector:

$$\|W\|_2 \;=\; \sup_{\|z\|_2=1} \|Wz\|_2.$$

Theorem 3.5 (Spectral norm = Lipschitz constant of $x \mapsto Wx$)**.** *For $f(x) = Wx$, $\mathrm{Lip}(f) = \|W\|_2 = \sigma_{\max}(W) = \sqrt{\lambda_{\max}(W^\top W)}$, where $\sigma_{\max}$ is the largest singular value of W and $\lambda_{\max}$ the largest eigenvalue.*

Proof. We compute $\sup_{\|z\|=1} \|Wz\|$ and identify it with the largest singular value via the Rayleigh quotient. Each step on its own line.

(1) For any x, y, linearity gives $f(x) - f(y) = Wx - Wy = W(x - y)$.
(2) Hence $\|f(x) - f(y)\| = \|W(x - y)\|$, and setting $z = x - y$ the Lipschitz constant is $\mathrm{Lip}(f) = \sup_{z\neq 0} \|Wz\|/\|z\| = \sup_{\|z\|=1} \|Wz\| = \|W\|_2$. So the Lipschitz constant *is* the operator norm; it remains to evaluate it.
(3) Squaring is monotone on nonnegatives, so $\sup_{\|z\|=1} \|Wz\|$ is the square root of $\sup_{\|z\|=1} \|Wz\|^2$.
(4) Expand the squared norm using the definition of the inner product: $\|Wz\|^2 = (Wz)^\top (Wz) = z^\top W^\top W z$.
(5) The matrix $A := W^\top W$ is symmetric ($A^\top = (W^\top W)^\top = W^\top W = A$) and positive semidefinite ($z^\top A z = \|Wz\|^2 \geq 0$). By the spectral theorem it has an orthonormal eigenbasis $v_1, \ldots, v_n$ with real eigenvalues $\lambda_1 \geq \cdots \geq \lambda_n \geq 0$.
(6) Write a unit vector $z = \sum_i c_i v_i$ with $\sum_i c_i^2 = 1$. Then $z^\top A z = \sum_i \lambda_i c_i^2$, a weighted average of the λ_i with weights c_i^2 summing to 1.
(7) A weighted average is maximized by putting all weight on the largest value: $\sum_i \lambda_i c_i^2 \leq \lambda_1$, with equality when $c_1 = 1$ (i.e. $z = v_1$). Hence $\sup_{\|z\|=1} z^\top A z = \lambda_1 = \lambda_{\max}(W^\top W)$.
(8) Taking the square root from step (3): $\|W\|_2 = \sqrt{\lambda_{\max}(W^\top W)} = \sigma_{\max}(W)$, since singular values of W are square roots of eigenvalues of $W^\top W$ by definition.

□

Worked example 3.6 (Spectral norms by hand)**.**

- $W = \left(\begin{smallmatrix} 3 & 0 \\ 0 & 1 \end{smallmatrix}\right)$. Then $W^\top W = \left(\begin{smallmatrix} 9 & 0 \\ 0 & 1 \end{smallmatrix}\right)$, eigenvalues 9 and 1, so $\|W\|_2 = \sqrt{9} = 3$. (For a diagonal matrix the spectral norm is just the largest |entry|.)
- $W = \left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right)$. Then $W^\top W = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 2 \end{smallmatrix}\right)$, with eigenvalues solving $\lambda^2 - 3\lambda + 1 = 0$, i.e. $\lambda = \frac{3\pm\sqrt{5}}{2}$. The spectral norm is $\sqrt{\frac{3+\sqrt{5}}{2}} \approx 1.618$ (the golden ratio). Notice the entries are all 0 or 1 yet the stretch exceeds 1: off-diagonal coupling matters.

3.3 Activations are 1-Lipschitz, and constants multiply across layers

A neural network alternates linear maps with simple nonlinear functions called *activations*, applied entrywise. The most common is the ReLU (rectified linear unit) $\mathrm{ReLU}(z) = \max\{0, z\}$. We need two facts.

Lemma 3.7 (ReLU is 1-Lipschitz)**.** $|\mathrm{ReLU}(a) - \mathrm{ReLU}(b)| \leq |a - b|$ *for all real a, b, so* $\mathrm{Lip}(\mathrm{ReLU}) = 1$. *Applied entrywise to a vector, the map is still 1-Lipschitz in ℓ_2.*

Proof. Check the four sign cases for a, b (each line is one case).

- $a \geq 0,\ b \geq 0$: $|\mathrm{ReLU}(a) - \mathrm{ReLU}(b)| = |a - b|$.
- $a \leq 0,\ b \leq 0$: both map to 0, difference $0 \leq |a - b|$.
- $a \geq 0,\ b \leq 0$: $|\mathrm{ReLU}(a) - \mathrm{ReLU}(b)| = |a - 0| = a \leq a - b = |a - b|$ (since $b \leq 0$).
- $a \leq 0,\ b \geq 0$: symmetric to the previous case, gives $b \leq |a - b|$.

In every case the inequality holds, and equality occurs (e.g. both positive), so the constant 1 is tight. Applied entrywise, $\|\mathrm{ReLU}(u) - \mathrm{ReLU}(v)\|_2^2 = \sum_i (\mathrm{ReLU}(u_i) - \mathrm{ReLU}(v_i))^2 \leq \sum_i (u_i - v_i)^2 = \|u - v\|_2^2$. □

Other activations have known constants too: the sigmoid is $\frac{1}{4}$-Lipschitz and tanh is 1-Lipschitz (Tsuzuku et al., 2018). Now the composition rule, which lets us bound a whole network.

Theorem 3.8 (Composition / product over layers (Szegedy et al., 2014; Tsuzuku et al., 2018))**.** *If g has Lipschitz constant L_g and f has Lipschitz constant L_f, then their composition $f \circ g$ (apply g, then f) satisfies* $\mathrm{Lip}(f \circ g) \leq L_f L_g$. *Consequently, for a d-layer network $F = f_d \circ \cdots \circ f_1$,*

$$\mathrm{Lip}(F) \ \leq \ \prod_{k=1}^{d} \mathrm{Lip}(f_k).$$

Proof. Apply the two Lipschitz bounds in turn, inner first. For any x_1, x_2:

$$\begin{aligned} \|f(g(x_1)) - f(g(x_2))\| &\leq L_f \, \|g(x_1) - g(x_2)\| && (f \text{ is } L_f\text{-Lipschitz, on inputs } g(x_1), g(x_2)) \\ &\leq L_f \left(L_g \, \|x_1 - x_2\|\right) && (g \text{ is } L_g\text{-Lipschitz}) \\ &= (L_f L_g) \, \|x_1 - x_2\|. \end{aligned}$$

So $L_f L_g$ is a valid Lipschitz constant for $f \circ g$, hence $\mathrm{Lip}(f \circ g) \leq L_f L_g$. Iterating over the d layers gives the product. The bound is an *inequality*, not equality, because the worst-stretch direction of one layer need not align with that of the next; this looseness is the subject of Section 3.7. □

For a standard network, each linear layer $f_k(z) = W_k z + b_k$ has $\mathrm{Lip}(f_k) = \|W_k\|_2$ (the bias b_k shifts the output but does not change the constant, since it cancels in $f_k(x_1) - f_k(x_2)$), and each ReLU layer has constant 1. So

$$\mathrm{Lip}(F) \ \leq \ \prod_{k} \|W_k\|_2,$$

the product of the spectral norms of the weight matrices — the original bound of Szegedy et al. (2014).

3.4 The prediction margin

The Lipschitz constant says how fast the logits can move. To turn that into a guarantee we also need to know how much room the current prediction has before the top logit loses to a runner-up.

Definition 3.9 (Prediction margin (Tsuzuku et al., 2018))**.** Let $F(x) \in \mathbb{R}^k$ be the logit vector and let t be the predicted (top) class at x. The *prediction margin* is

$$M_{F,x} \ = \ F(x)_t \ - \ \max_{i \neq t} F(x)_i,$$

the gap between the winning logit and the best competitor. It is positive exactly when x is classified as t, and a larger margin means the decision is more decisive.

3.5 The Lipschitz–margin certificate

Here is the payoff: a provable lower bound on the robust radius, due to Tsuzuku et al. (2018). It is the certificate the paper builds on.

Proposition 3.10 (Lipschitz–margin certificate, Tsuzuku Prop. 1 (Tsuzuku et al., 2018))**.** *Let $F : \mathbb{R}^n \to \mathbb{R}^k$ have ℓ_2-Lipschitz constant L (so $\|F(x) - F(x+\delta)\|_2 \le L\|\delta\|_2$ for any perturbation δ) and let $M_{F,x}$ be the prediction margin at x. If*

$$M_{F,x} \;\ge\; \sqrt{2}\, L\, \|\delta\|_2,$$

then the predicted class does not change at $x+\delta$. Equivalently, the certified ℓ_2 robust radius is

$$r(x) \;\ge\; \frac{M_{F,x}}{\sqrt{2}\, L}.$$

We prove it in full, slowly. The one nonobvious step — where the $\sqrt{2}$ comes from — is a Cauchy–Schwarz inequality on two coordinates, so we first record a small lemma.

Lemma 3.11 (The "max of the rest" is 1-Lipschitz (Tsuzuku et al., 2018))**.** *For real vectors a and b, $\left|\max_{i\neq t} a_i - \max_{i\neq t} b_i\right| \le \max_{i\neq t} |a_i - b_i|$.*

Proof. Without loss of generality assume $\max_{i\neq t} a_i \ge \max_{i\neq t} b_i$ (otherwise swap a, b). Let $j = \arg\max_{i\neq t} a_i$. Then

$$\begin{aligned}
\left|\max_{i\neq t} a_i - \max_{i\neq t} b_i\right| &= \max_{i\neq t} a_i - \max_{i\neq t} b_i && \text{(the quantity is nonnegative by assumption)}\\
&= a_j - \max_{i\neq t} b_i && (j \text{ attains the max for } a)\\
&\le a_j - b_j && (\max_{i\neq t} b_i \ge b_j, \text{ so subtracting it is } \le \text{ subtracting } b_j)\\
&\le \max_{i\neq t} |a_i - b_i|. && (a_j - b_j \le |a_j - b_j| \le \text{ the max})
\end{aligned}$$

□

Proof of Proposition 3.10. It suffices to show the margin at the perturbed point stays nonnegative; in fact we show it drops by at most $\sqrt{2}\, L\|\delta\|_2$:

$$M_{F,x+\delta} \;=\; F(x+\delta)_t - \max_{i\neq t} F(x+\delta)_i \;\ge\; M_{F,x} - \sqrt{2}\, L\, \|\delta\|_2. \tag{4}$$

Granting (4), if $M_{F,x} \ge \sqrt{2}L\|\delta\|_2$ then $M_{F,x+\delta} \ge 0$, so class t still wins. Now prove (4). Add and subtract the unperturbed logits:

$$\begin{aligned}
M_{F,x+\delta} &= F(x+\delta)_t - \max_{i\neq t} F(x+\delta)_i\\
&= \underbrace{\big(F(x)_t - \max_{i\neq t} F(x)_i\big)}_{=M_{F,x}} + \underbrace{\big(F(x+\delta)_t - F(x)_t\big)}_{=:a_1} - \underbrace{\big(\max_{i\neq t} F(x+\delta)_i - \max_{i\neq t} F(x)_i\big)}_{=:a_2'}.
\end{aligned}$$

Bound the two change terms from below by their negative absolute values:

$$M_{F,x+\delta} \;\ge\; M_{F,x} - |a_1| - |a_2'|.$$

For a_1, $|a_1| = |F(x+\delta)_t - F(x)_t|$ is the change in a single coordinate of the logit vector. For a_2', Lemma 3.11 (with $a = F(x+\delta)$, $b = F(x)$) gives $|a_2'| \le \max_{i \neq t} |F(x+\delta)_i - F(x)_i|$, again the absolute change of a *single* (other) coordinate. Call the change of the top logit Δ_t and the change of the worst competitor Δ_j; both are entries of the vector $F(x+\delta) - F(x)$, so

$$\sqrt{\Delta_t^2 + \Delta_j^2} \;\le\; \big\|F(x+\delta) - F(x)\big\|_2 \;\le\; L\,\|\delta\|_2,$$

the last step being the definition of the Lipschitz constant L. We must bound $|a_1| + |a_2'| = |\Delta_t| + |\Delta_j|$, the sum of two absolute values, by something times $\sqrt{\Delta_t^2 + \Delta_j^2}$. This is where $\sqrt{2}$ enters, by Cauchy–Schwarz on the two-vector $(|\Delta_t|, |\Delta_j|)$ against $(1,1)$:

$$|\Delta_t| + |\Delta_j| \;=\; (1,1)\cdot(|\Delta_t|, |\Delta_j|) \;\le\; \big\|(1,1)\big\|_2\,\big\|(\Delta_t, \Delta_j)\big\|_2 \;=\; \sqrt{2}\,\sqrt{\Delta_t^2 + \Delta_j^2}\,.$$

Chaining the bounds:

$$|a_1| + |a_2'| \;\le\; \sqrt{2}\,\sqrt{\Delta_t^2 + \Delta_j^2} \;\le\; \sqrt{2}\,L\,\|\delta\|_2.$$

Substituting into $M_{F,x+\delta} \ge M_{F,x} - |a_1| - |a_2'|$ gives (4), completing the proof. □

Reading the certificate. The radius $r(x) \ge M_{F,x}/(\sqrt{2}\,L)$ says exactly what robustness should: *decisive predictions (big margin) and gentle models (small L) are certifiably safe over a large ball.* Both knobs appear, and the certificate is the lower-bound (defender's) counterpart to the upper-bound attacks of Section 4.

3.6 Worked examples: one-layer certificates

Worked example 3.12 (The exact two-class radius (Hein and Andriushchenko, 2017))**.** For a *linear* two-class model the $\sqrt{2}$ disappears and the certificate becomes exact, which makes a clean sanity check. Let the two class weight vectors differ by $w_c - w_j = (2, 0)$, evaluated at $x = (2, 5)$. The decision is governed by the scalar $g(x) = \langle w_c - w_j,\, x\rangle = 2 \cdot 2 + 0 \cdot 5 = 4$ (the margin), and $\|w_c - w_j\|_2 = 2$. Because here the margin is a single scalar linear function (not a max over many classes), the certified ℓ_2 radius is exactly $\dfrac{4}{2} = 2$. The general exact formula of Hein and Andriushchenko (2017) for a linear classifier is

$$r(x) \;=\; \min_{j \neq c} \frac{\langle w_c - w_j,\, x\rangle}{\|w_c - w_j\|_2}\,,$$

the smallest distance to any pairwise boundary — itself a clean instance of the dual-norm computation from Section 2.9.

Worked example 3.13 (One-hidden-layer network)**.** Take $F(x) = W_2\,\mathrm{ReLU}(W_1 x)$ with $\|W_1\|_2 = 3$ and $\|W_2\|_2 = 2$. By Theorem 3.8 and Lemma 3.7, the global Lipschitz constant is bounded by $L \le 3 \cdot 1 \cdot 2 = 6$. If the prediction margin at some x is $M_{F,x} = 12$, the Tsuzuku certificate gives

$$r(x) \;\ge\; \frac{M_{F,x}}{\sqrt{2}\,L} \;=\; \frac{12}{\sqrt{2}\cdot 6} \;=\; \frac{12}{6\sqrt{2}} \;=\; \sqrt{2} \;\approx\; 1.414.$$

So we have *proved* that no ℓ_2 perturbation of size up to 1.414 can change this prediction — no attack, however clever, can do better at this point.

3.7 Honest caveat: the bound is loose, and the exact constant is NP-hard

A certificate is only useful if you are honest about its slack. Two points, both from the sources.

First, the product bound $\mathrm{Lip}(F) \le \prod_k \|W_k\|_2$ is an *over-estimate* of the true Lipschitz constant, often badly so. The reason was already visible in the proof of Theorem 3.8: it assumes every layer is stretched maximally in the same direction, which never happens. A larger L than the truth makes the certified radius $M/(\sqrt{2}L)$ *smaller* than it could be, so the guarantee is conservative — safe, but weak. Hein and Andriushchenko (2017) report that *local* Lipschitz estimates (valid in a neighborhood of x) can be up to about $8\times$ smaller than the global product bound, yielding correspondingly stronger certificates.

Second, you might hope to just compute the exact $\mathrm{Lip}(F)$ and avoid the slack. You cannot, in general.

Theorem 3.14 (Exact Lipschitz computation is NP-hard (Scaman and Virmaux, 2018))**.** *Computing the exact Lipschitz constant of even a two-layer ReLU network is NP-hard. Hence every practical certificate uses an* upper bound *on L (such as the product of spectral norms), and the resulting robust radius is a* lower *bound on the true robust radius.*

This is the right mental model: Szegedy et al. (2014) put it as "small Lipschitz bounds forbid attacks, but large bounds do not imply attacks exist." A small certified L guarantees safety; a large L (or a loose bound) simply means we failed to prove safety, not that the model is necessarily fragile.

3.8 Exercise: a full two-layer certificate

Exercise. Let $F(x) = W_2\,\mathrm{ReLU}(W_1 x)$ with

$$W_1 = \left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right), \qquad W_2 = \left(\begin{smallmatrix} 1 & 1 \\ -1 & 1 \end{smallmatrix}\right).$$

(a) Compute $\|W_1\|_2$ and $\|W_2\|_2$ (eigenvalues of $W^\top W$). (b) Give the global Lipschitz bound L for F. (c) At $x = (1, 1)$, compute $F(x)$, the predicted class, and the prediction margin $M_{F,x}$. (d) State the Tsuzuku certified radius $M_{F,x}/(\sqrt{2}\,L)$, and explain in one sentence why it is a conservative *lower* bound on the true robust radius.

Solution. (a) W_1 is diagonal, so $\|W_1\|_2 = \max\{2, 1\} = 2$. For W_2, $W_2^\top W_2 = \left(\begin{smallmatrix} 2 & 0 \\ 0 & 2 \end{smallmatrix}\right) = 2I$, so both eigenvalues are 2 and $\|W_2\|_2 = \sqrt{2}$. (Indeed $W_2/\sqrt{2}$ is a rotation; W_2 scales every direction by $\sqrt{2}$.)

(b) By Theorem 3.8 and ReLU being 1-Lipschitz, $L \le \|W_1\|_2 \cdot 1 \cdot \|W_2\|_2 = 2 \cdot \sqrt{2} = 2\sqrt{2} \approx 2.828$.

(c) $W_1 x = (2, 1)$, both positive, so $\mathrm{ReLU}(W_1 x) = (2, 1)$. Then $F(x) = W_2(2, 1)^\top = (2 + 1,\ -2 + 1) = (3, -1)$. The top logit is the first (3), so the predicted class is 1, and $M_{F,x} = 3 - (-1) = 4$.

(d) Certified radius $= \dfrac{M_{F,x}}{\sqrt{2}\,L} = \dfrac{4}{\sqrt{2} \cdot 2\sqrt{2}} = \dfrac{4}{4} = 1$. It is a *lower* bound because $L = 2\sqrt{2}$ is itself an over-estimate of the true Lipschitz constant (the product bound assumes both layers stretch in the same direction, Theorem 3.8; and computing the exact constant is NP-hard, Theorem 3.14), so the real safe radius is at least 1 and likely larger. □

4 Attacks in detail, and how robustness is measured

We can now make Section 2's "attack = upper bound" concrete. This section gives the standard attacks, each with: definition, intuition, the algorithm, a hand-computable example, and a short exercise. We follow the unifying taxonomy of Kolter and Madry (2018); Madry et al. (2018).

4.1 The organizing idea: norm ball + optimizer

Kolter and Madry (2018) insist on one classification: *every attack is (1) a norm ball plus (2) a method for optimizing over it.* This splits attacks into two families, which match the two robustness numbers of Section 2.6.

- **Fixed-ϵ (norm-bounded).** Solve $\max_{\|\delta\|\le\epsilon} \ell(h(x+\delta), y)$: "can this point be broken within budget ϵ?" Answering it for many points gives $\mathrm{RobAcc}(\epsilon)$ — one vertical slice of the robustness curve. Members: FGSM, PGD, AutoAttack.
- **Minimum-norm.** Solve $\min \|\delta\|$ such that $x+\delta$ is misclassified: "how far is the boundary?" Answering it gives the robust radius $r(x)$ — the whole curve. Members: C&W, DDN.

They are duals via Equation (2): $\mathrm{RobAcc}(\epsilon) = \frac{1}{N}\sum_x \mathbf{1}[r(x) > \epsilon]$. A fixed-$\epsilon$ attack reads the curve at one ϵ; a minimum-norm attack measures $r(x)$ directly.

4.2 FGSM: the Fast Gradient Sign Method

Definition. The *Fast Gradient Sign Method* of Goodfellow et al. (2015) takes one step from x in the direction that, to first order, increases the loss the most under an ℓ_∞ budget:

$$x' \;=\; x + \epsilon\,\mathrm{sign}\big(\nabla_x \ell(h_\theta(x), y)\big). \tag{5}$$

One gradient evaluation, one step. The sign is applied entrywise.

Intuition: the linearity hypothesis. Why sign of the gradient, and why ℓ_∞? Goodfellow et al. (2015) argue that adversarial examples come from models being *too linear in high dimension*, not too nonlinear. Linearize the model near x: a linear unit responds to $x' = x+\delta$ as $w^\top x' = w^\top x + w^\top \delta$. By the exercise of Section 2.9, maximizing the perturbation term $w^\top\delta$ over $\|\delta\|_\infty \le \epsilon$ gives $\delta = \epsilon\,\mathrm{sign}(w)$ and value $\epsilon\|w\|_1$. Now the punchline: if w has n entries of average magnitude m, then $\|w\|_1 \approx mn$, so the change to the output grows *linearly with the dimension* n, while the per-pixel change $\|\delta\|_\infty$ stays fixed at ϵ (Goodfellow et al., 2015). In high dimension many imperceptible per-pixel nudges, all aligned with the sign of the weights, add up to a large swing in the output. The famous demonstration: panda + $0.007 \cdot \mathrm{sign}(\nabla)$ classified as "gibbon" with 99.3% confidence (Goodfellow et al., 2015).

Exactness for logistic regression. For a two-class logistic model FGSM is not an approximation — it is the *exact* worst case. The reason is the identity $w^\top \mathrm{sign}(w) = \|w\|_1$ used in Section 2.9: the sign step realizes the dual-norm optimum exactly when the model is linear (Goodfellow et al., 2015; Kolter and Madry, 2018).

Algorithm.

1. Compute the gradient $g = \nabla_x \ell(h_\theta(x), y)$ by backpropagation.
2. Form the signed step $\delta = \epsilon\,\mathrm{sign}(g)$.
3. Return $x' = x + \delta$ (clip to the valid input range if needed).

Worked example 4.1 (FGSM on a 2D logistic model)**.** Let $w = (2, -1)$, $b = 0$, true label $y = +1$, and a point x with positive score $s(x) = w^\top x$. The gradient of the logistic loss with respect to x points along $-yw = -w$ (loss decreases as the score $y\,w^\top x$ grows), so the loss-increasing sign direction is $-\mathrm{sign}(w) = (-1, +1)$, giving the attack $\delta^\star = -\epsilon\,\mathrm{sign}(w) = (-\epsilon, +\epsilon)$. The score change is

$$w^\top\delta^\star = (2)(-\epsilon) + (-1)(+\epsilon) = -3\epsilon = -\epsilon\,\|w\|_1, \qquad \|w\|_1 = |2| + |-1| = 3.$$

The margin drops by $\epsilon\|w\|_1 = 3\epsilon$, matching Section 2.9 exactly. Generalize to $w \in \mathbb{R}^n$ with average $|w_i| = m$: the drop is $\epsilon\|w\|_1 \approx \epsilon mn$, which grows with n — the high-dimension punchline, derived by hand.

Exercise. For $w = (1, 1, 1, 1)$, $b = 0$, $\epsilon = 0.1$, true label $+1$: write the FGSM perturbation $\delta^\star$, compute the margin drop $\epsilon\|w\|_1$, and compare to the same computation for $w = (1, 1, \ldots, 1) \in \mathbb{R}^{100}$. *(Answer: $\delta^\star = (-0.1, -0.1, -0.1, -0.1)$, drop 0.4; in $\mathbb{R}^{100}$ the drop is $0.1 \cdot 100 = 10$, with the same per-pixel size 0.1 — illustrating linear growth in dimension.)*

4.3 PGD: Projected Gradient Descent

Definition. FGSM takes one big step and trusts the linear approximation over the whole ball. *Projected Gradient Descent* (Madry et al., 2018) takes many small steps, projecting back into the ϵ-ball after each, and starting from a random point inside the ball. It is the empirical "ultimate first-order adversary."

The saddle point. PGD is the inner solver for the robust-optimization problem (3),

$$\min_\theta \ \mathbb{E}\Big[\max_{\delta\in\Delta} \ \ell(\theta, x+\delta, y)\Big], \qquad \Delta = \{\delta : \|\delta\|_\infty \le \epsilon\} :$$

the inner max is the attack, the outer min the defense (Madry et al., 2018).

Iteration. Starting from a *uniformly random* x^0 in the ϵ-ball around x, repeat

$$x^{t+1} \ = \ \mathrm{Proj}_{x+\Delta}\Big(x^t + \alpha \ \mathrm{sign}\big(\nabla_x \ell(h_\theta(x^t), y)\big)\Big), \tag{6}$$

where $\alpha > 0$ is the step size and $\mathrm{Proj}_{x+\Delta}$ projects back into the threat set. For the ℓ_∞ ball the projection is just *clipping*: any coordinate of $\delta = x^{t+1} - x$ that exceeds ϵ in absolute value is set to $\pm\epsilon$ (Kolter and Madry, 2018).

Step-then-project, and why a random start. Each iteration first takes a signed-gradient step (like a mini-FGSM), which may leave the box, then clips back to the nearest face or corner of the box — the "step then project" picture. FGSM is exactly one such step with no random start and $\alpha = \epsilon$. The random start matters because PGD can get stuck at local maxima of the (non-concave) inner loss; restarting from different random points inside the ball and keeping the worst found is a cheap way to escape some of them (Kolter and Madry, 2018). Madry et al. (2018) observe empirically that the loss values at many random-restart local maxima are *well concentrated*, which is the evidence behind calling PGD the "ultimate first-order adversary."

Normalized steepest descent and the step-size rule. Why $\mathrm{sign}(\nabla)$ rather than ∇ itself? At $\delta = 0$ the raw gradient is tiny (about 10^{-6} per pixel in the tutorial's CNN), so plain gradient ascent would need an absurd step size ($\sim 10^4$) and then overshoot (Kolter and Madry, 2018). The fix is *normalized steepest descent* under the chosen norm: choose the update direction v maximizing $v^\top \nabla$ subject to a norm cap on v. For ℓ_∞ this gives $v = \mathrm{sign}(\nabla)$ (the sign step); for ℓ_2 it gives $v = \nabla/\|\nabla\|_2$ (the normalized gradient) (Kolter and Madry, 2018). Practical rule of thumb (Kolter and Madry, 2018): take α a small fraction of ϵ, and the number of iterations a small multiple of ϵ/α.

Algorithm.

1. Initialize δ uniformly at random in $\{\delta : \|\delta\|_\infty \le \epsilon\}$.
2. For $t = 1, \dots, T$: compute $g = \nabla_x \ell(h_\theta(x+\delta), y)$; update $\delta \leftarrow \delta + \alpha \operatorname{sign}(g)$; clip each coordinate of δ to $[-\epsilon, \epsilon]$.
3. (Optional) repeat from random restarts; keep the δ of highest loss.

Worked example 4.2 (Step-then-project by hand)**.** Let the model be the 2D linear $s(x) = w^\top x$, $w = (2, -1)$, $y = +1$, $\epsilon = 1$, $\alpha = 2$, starting from $\delta^0 = (0, 0)$. The loss-increasing direction is $-\operatorname{sign}(w) = (-1, +1)$. **Step:** $\delta = (0, 0) + 2 \cdot (-1, +1) = (-2, +2)$. **Project** onto $\|\delta\|_\infty \le 1$: clip each coordinate to $[-1, 1]$, giving $\delta = (-1, +1)$ — a corner of the box, which is exactly the FGSM solution $-\epsilon \operatorname{sign}(w)$. So for a linear model PGD reaches the FGSM corner in one step-and-clip; the value of many small steps shows up only for genuinely nonlinear models, where the gradient direction changes as you move.

Exercise. Explain, in two sentences, where FGSM and PGD each sit on the accuracy-vs-radius curve of Section 2.6, and why PGD at a fixed ϵ never reports *higher* robust accuracy than the true value. *(Answer: both are fixed-ϵ attacks, so each estimates a single point* $\mathrm{RobAcc}(\epsilon)$*; since any successful δ they find with $\|\delta\| \le \epsilon$ proves $r(x) \le \epsilon$, an attack can only* lower *the reported robust accuracy toward the truth, i.e. it is an upper bound on robustness — Section 2.7.)*

4.4 C&W: a minimum-norm attack

Definition and intuition. The Carlini–Wagner attack (Carlini and Wagner, 2017) switches families: instead of fixing ϵ, it asks for the *smallest* perturbation that misclassifies, decoupling "be small" from "be misclassified." It minimizes

$$\min_{\delta} \ \|\delta\|_p \ + \ c \cdot f(x + \delta) \qquad \text{subject to } x + \delta \in [0, 1]^n, \tag{7}$$

where f is a surrogate that is ≤ 0 exactly when the example is misclassified, and the trade-off constant $c > 0$ is found by *binary search*: small c favors a tiny but possibly non-adversarial δ; large c forces misclassification at the cost of a larger δ. Carlini and Wagner (2017) pick the smallest c for which the attack still succeeds.

The surrogate f (the "f_6" margin objective). Among seven candidates they measured, the best is the logit-difference margin

$$f(x') \ = \ \max\Big(\max_{i \neq t} Z(x')_i - Z(x')_t, \ -\kappa \Big),$$

where $Z(\cdot)$ are the *logits* (not softmax probabilities), t the target class, and $\kappa \ge 0$ a confidence parameter. This f is ≤ 0 exactly when the target logit beats all others, so "$f \le 0 \Leftrightarrow$ misclassified." Using logits rather than softmax avoids vanishing gradients — the bug that broke an earlier defense (defensive distillation) — and Carlini and Wagner (2017) found cross-entropy the *worst* surrogate and this logit-difference the best, because it keeps the "be small" and "be misclassified" terms balanced across values of c.

The change-of-variables (box) trick. The constraint $x + \delta \in [0, 1]^n$ is awkward. C&W eliminate it by writing each coordinate as

$$\delta_i \ = \ \tfrac{1}{2}\big(\tanh(w_i) + 1\big) - x_i,$$

and optimizing freely over the new variable w. Since $\tanh \in (-1, 1)$, the value $x_i + \delta_i = \frac{1}{2}(\tanh(w_i) + 1)$ is automatically in $(0, 1)$, so any w gives a valid image and a smooth, unconstrained problem solvable by a standard optimizer (Adam).

Algorithm.

1. Choose a value of c (binary search over a log-spaced range).
2. Optimize Eq. (7) over w (with the tanh substitution) to convergence, recording the best adversarial δ.
3. Update c: increase if no adversarial example was found, decrease if one was; repeat.
4. Return the smallest-norm adversarial δ over all c.

Worked example 4.3 (Reading $\|\delta\|^2 + c\, f$ for small vs. large c)**.** Take a 1D two-class model where the runner-up-minus-target logit gap, as a function of the scalar perturbation δ, is $g(\delta) = 1 - \delta$ (so $f_6 = \max(1 - \delta, -\kappa)$ with $\kappa = 0$ flips to "misclassified" once $\delta \geq 1$). The objective is $J(\delta) = \delta^2 + c \max(1 - \delta, 0)$. For very small c (say $c = 0.1$): on $\delta < 1$, $J = \delta^2 + 0.1(1 - \delta)$ is minimized near $\delta \approx 0.05$, which is *not* adversarial ($\delta < 1$) — too small a c buys a tiny but useless δ. For large c (say $c = 10$): the penalty dominates until $\delta \geq 1$, pushing the minimizer to $\delta = 1$, the smallest adversarial value. Sweeping c traces from "too small to flip" to "flips with minimal norm," which is exactly what the binary search automates; Carlini and Wagner (2017) show this as their Figure 2, where the attack rarely succeeds for $c < 0.1$ and always succeeds for $c > 1$.

Exercise. With $J(\delta) = \delta^2 + c \max(1 - \delta, 0)$ as above, find the threshold value of c at which the unconstrained minimizer first becomes adversarial ($\delta^\star \geq 1$). *(Hint: on $\delta < 1$ the derivative is $2\delta - c$; the interior minimizer is $\delta = c/2$, which reaches 1 when $c = 2$. For $c \geq 2$ the minimizer sits at the kink $\delta = 1$.)*

4.5 DDN: Decoupled Direction and Norm

Definition and intuition. C&W is slow: a binary search over c, each with thousands of iterations. The Decoupled Direction and Norm attack (Rony et al., 2019) reaches the same minimum-norm goal in about 100 iterations — cheap enough to run *inside* adversarial training. The idea in the name: decouple the *direction* of the perturbation (the unit-normalized gradient) from its *norm* (a simple up/down rule plus a projection onto an ϵ-sphere). There is no penalty constant c at all.

The norm update. At each step DDN checks whether the current point is already adversarial. If it is, the example is "too easy," so *shrink* the radius: $\epsilon \leftarrow (1 - \gamma)\epsilon$. If it is not adversarial, *grow* the radius: $\epsilon \leftarrow (1 + \gamma)\epsilon$. The candidate is then placed on the sphere of the current radius, $\tilde{x} = x + \epsilon\, \delta / \|\delta\|_2$. The radius oscillates around the true boundary distance and converges to the minimum norm $r(x)$ (Rony et al., 2019).

Algorithm (Rony et al., 2019).

1. Initialize $\delta = 0$, $\tilde{x} = x$, $\epsilon = \epsilon_0$.
2. For $k = 1, \ldots, K$: compute the gradient g of the loss; take a normalized step $g \leftarrow \alpha\, g / \|g\|_2$ and update $\delta \leftarrow \delta + g$.
3. If $\tilde{x}$ is adversarial, set $\epsilon \leftarrow (1 - \gamma)\epsilon$; else $\epsilon \leftarrow (1 + \gamma)\epsilon$.
4. Project: $\tilde{x} \leftarrow x + \epsilon\, \delta / \|\delta\|_2$, then clip to the valid range.
5. Return the lowest-norm $\tilde{x}$ found that is adversarial.

Worked example 4.4 (Oscillating radius by hand)**.** Take a classifier whose boundary is the vertical line at distance 1 from x along the attack direction, and set $\epsilon_0 = 1$, $\gamma = 0.2$, with the step always pointing straight at the boundary. Iterate the radius rule, checking "adversarial?" = "radius ≥ 1?":

- Start $\epsilon = 1.0$: on the boundary, treat as adversarial $\rightarrow$ shrink to $1.0 \cdot 0.8 = 0.8$.
- $\epsilon = 0.8$: not adversarial $(0.8 < 1) \rightarrow$ grow to $0.8 \cdot 1.2 = 0.96$.
- $\epsilon = 0.96$: not adversarial $\rightarrow$ grow to $0.96 \cdot 1.2 = 1.152$.
- $\epsilon = 1.152$: adversarial $\rightarrow$ shrink to $1.152 \cdot 0.8 = 0.9216$.

The radius bounces $1.0, 0.8, 0.96, 1.152, 0.9216, \ldots$ around the true distance 1, the swings narrowing as γ effectively shrinks the relative gap; the converged value is the robust radius $r(x) = 1$. Contrast with C&W, which would binary-search a penalty constant c to find the same number much more slowly (Rony et al., 2019).

Exercise. With the same setup but $\gamma = 0.1$ and $\epsilon_0 = 1.5$, write the first four radii and state the limit. *(Answer:* $1.5 \rightarrow 1.35 \rightarrow 1.215 \rightarrow 1.0935 \rightarrow \ldots$*, all* > 1 *so all shrink, converging down to* $r(x) = 1$*.)*

4.6 AutoAttack: a standardized ensemble

What it is. A recurring failure in the field is reporting robustness using a weak attack: the model looks robust only because the attack was bad (a problem called *gradient masking*). AutoAttack (Croce and Hein, 2020) is the community's response: a *parameter-free* ensemble of four diverse attacks, run together as a single standardized robustness test, so that no hand-tuning can inflate the result.

The four components. We describe the ensemble at the level of its four members (Croce and Hein, 2020):

1. **APGD-CE** — Auto-PGD with the cross-entropy loss. Auto-PGD is PGD with an *adaptive* step size (no step-size tuning) and a momentum term; it spends a budget of iterations exploring then exploiting, halving the step when progress stalls.
2. **APGD-DLR** (also a targeted variant, APGD-T) — Auto-PGD with the *Difference-of-Logits-Ratio* loss. The DLR loss is used because cross-entropy is sensitive to a mere rescaling of the logits (which can cause gradient masking), whereas DLR is scale-invariant.
3. **FAB** — a minimum-norm attack (the Fast Adaptive Boundary attack).
4. **Square** — a *black-box*, gradient-free attack (random search over square-shaped perturbations), included so that the ensemble still works even when gradients are uninformative.

A point counts as robust under AutoAttack only if it survives *all four*. The finer internals of FAB and Square are beyond what a primer needs; the four-member structure is the part that matters for understanding why the ensemble is hard to fool.

Why this is the right way to measure. AutoAttack ties the section together: it is a fixed-ϵ *evaluation* that combines white-box first-order attacks (the APGD pair), a minimum-norm attack (FAB), and a gradient-free attack (Square), precisely so that the reported RobAcc(ϵ) is a trustworthy *upper bound* on the model's true robustness (Section 2.7) rather than an artifact of a weak optimizer.

Exercise. A paper reports 90% robust accuracy using only FGSM. Using the taxonomy of Section 4.1 and the certificate/attack duality of Section 2.7, explain why an AutoAttack re-evaluation can only *lower* this number, never raise it, and why that makes AutoAttack a more honest measurement.

(Answer: every attack gives an upper bound on robustness; a stronger or more diverse attack can find perturbations FGSM missed, decreasing the surviving fraction toward the truth, but it can never make a broken point survive, so the number can only fall.)

5 Orthogonal projectors: the one linear-algebra tool everything rests on

Almost every later idea in this primer turns out to be the same gesture: take an object, throw away the part of it that some symmetry can move around, and keep only the part the symmetry leaves alone. The mathematical name for "keep only the part a subspace can see" is *orthogonal projection*. We start here so that the rest of the document can lean on one clean theorem instead of re-deriving it three times.

We work in a real or complex inner-product space V (think of $V = \mathbb{R}^n$ or $V = \mathbb{C}^n$ with the usual dot product). The *inner product* of two vectors is written $\langle u, v\rangle$; in $\mathbb{R}^n$ it is $\langle u, v\rangle = \sum_k u_k v_k$, and in $\mathbb{C}^n$ it is $\langle u, v\rangle = \sum_k u_k \overline{v_k}$, where $\overline{c}$ is the *complex conjugate* of c (if $c = a + bi$ then $\overline{c} = a - bi$). The *norm* is $\|v\| = \sqrt{\langle v, v\rangle}$, and two vectors are *orthogonal*, written $u \perp v$, when $\langle u, v\rangle = 0$. Throughout, an *operator* means a linear map $P : V \to V$, which we may identify with a square matrix once a basis is fixed.

5.1 Two adjectives: idempotent and self-adjoint

Definition 5.1 (Idempotent)**.** An operator $P : V \to V$ is *idempotent* if $P^2 = P$, i.e. $P(Pv) = Pv$ for every $v \in V$. In words: applying P twice does exactly what applying it once does.

The intuition is that P *lands* every vector somewhere, and once a vector has already landed, P leaves it alone. The set of places vectors can land is the *range* (or image) $\mathrm{range}(P) = \{Pv : v \in V\}$; the set of vectors P sends to 0 is the *kernel* (or null space) $\ker(P) = \{v : Pv = 0\}$. Idempotence says precisely that P acts as the identity on its own range: if $w = Pv$ is in the range, then $Pw = P(Pv) = Pv = w$.

Worked example 5.2 (A 2×2 idempotent that is *not* a clean projection)**.** Take, for a fixed number $\alpha \neq 0$,

$$P \;=\; \begin{bmatrix} 0 & 0 \\ \alpha & 1 \end{bmatrix}.$$

A direct multiplication gives

$$P^2 \;=\; \begin{bmatrix} 0 & 0 \\ \alpha & 1 \end{bmatrix}\begin{bmatrix} 0 & 0 \\ \alpha & 1 \end{bmatrix} \;=\; \begin{bmatrix} 0\cdot 0 + 0\cdot \alpha & 0\cdot 0 + 0\cdot 1 \\ \alpha\cdot 0 + 1\cdot \alpha & \alpha\cdot 0 + 1\cdot 1 \end{bmatrix} \;=\; \begin{bmatrix} 0 & 0 \\ \alpha & 1 \end{bmatrix} \;=\; P,$$

so P is idempotent: it *is* a projection. But it is a *skewed* one. Its range is the line spanned by $(0, 1)$ and its kernel is the line spanned by $(1, -\alpha)$ (check: $P\,(1, -\alpha)^\top = (0, \alpha - \alpha)^\top = 0$). These two lines are *not* perpendicular when $\alpha \neq 0$. Such a P is called an *oblique* projection. This example, due to Wikipedia's projection article (Wikipedia, n.d.f), is the cautionary tale of the section: idempotence alone does not make a projection "drop straight down." One more property is needed.

Definition 5.3 (Self-adjoint)**.** An operator $P : V \to V$ is *self-adjoint* if

$$\langle Pu, v\rangle \;=\; \langle u, Pv\rangle \qquad \text{for all } u, v \in V.$$

In matrix terms this is $P = P^\top$ for real V (the matrix equals its transpose) and $P = P^*$ for complex V, where $P^* = \overline{P}^\top$ is the *conjugate transpose* (transpose, then conjugate every entry). Self-adjoint real matrices are also called *symmetric*.

Worked example 5.4 (The clean version of the same idea)**.** Let $a = (1, 1)^\top$ and form

$$P \;=\; \frac{a\,a^\top}{a^\top a} \;=\; \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

Then $P = P^\top$ (it is visibly symmetric), and $P^2 = \frac{1}{4}\left[\begin{smallmatrix} 2 & 2 \\ 2 & 2 \end{smallmatrix}\right] = P$. So P is *both* idempotent and self-adjoint. Geometrically it projects every vector straight down onto the line through a: for $x = (3, 1)^\top$ we get $Px = (2, 2)^\top$, and the leftover piece $x - Px = (1, -1)^\top$ is orthogonal to a (their dot product is $1 - 1 = 0$). That is the behaviour we want, and the next theorem says it always happens (Texas A&M University, n.d.; Wikipedia, n.d.f).

5.2 The central theorem

Theorem 5.5 (Idempotent + self-adjoint ⇒ orthogonal projection)**.** *Let $P : V \to V$ be idempotent ($P^2 = P$) and self-adjoint ($P = P^*$). Then P is the* orthogonal *projection onto its range $W =$ range(P): for every v, the vector Pv lies in W, the leftover $v - Pv$ is orthogonal to all of W, and Pv is the unique closest point of W to v.*

Proof. The whole proof rests on showing one orthogonality statement; everything else follows from it. We follow Garrett's argument (Garrett, n.d.).

Step 1: the complementary operator. Write I for the identity and set $Q = I - P$. Then Q measures the leftover: $Qv = v - Pv$. Note Q is also self-adjoint, because $Q^* = (I - P)^* = I^* - P^* = I - P = Q$, using $P^* = P$.

Step 2: the key product vanishes. Compute, for the operator $QP = (I - P)P$,

$$\begin{aligned} QP &= (I - P)P \\ &= IP - P^2 \\ &= P - P \qquad \text{(here we used idempotence } P^2 = P\text{)} \\ &= 0. \end{aligned}$$

So $QP = 0$ as an operator: Q annihilates everything P produces.

Step 3: range of P is orthogonal to range of Q. Take any $v, w \in V$. We test the inner product of an arbitrary range vector Pv against an arbitrary leftover vector Qw:

$$\begin{aligned} \langle Pv,\, Qw\rangle &= \langle Pv,\, (I - P)w\rangle \\ &= \langle (I - P)^* Pv,\, w\rangle \qquad \text{(move the operator to the other slot via the adjoint)} \\ &= \langle (I - P)Pv,\, w\rangle \qquad \text{(since } (I - P)^* = I - P \text{ by Step 1)} \\ &= \langle QP\,v,\, w\rangle \\ &= \langle 0,\, w\rangle \qquad \text{(since } QP = 0 \text{ by Step 2)} \\ &= 0. \end{aligned}$$

Because v and w were arbitrary, *every* vector Pv in the range of P is orthogonal to *every* leftover $Qw = w - Pw$. In particular, taking w to range over all of V, the leftover $v - Pv$ is orthogonal to the entire range W.

Step 4: closest point (Pythagoras). Fix v and let w be any point of W. Since $Pv \in W$ and $w \in W$, also $Pv - w \in W$, hence by Step 3 it is orthogonal to the leftover $v - Pv$. The Pythagorean law for orthogonal vectors (Goodman, n.d.) then gives

$$\|v - w\|^2 \;=\; \|\,(v - Pv) + (Pv - w)\,\|^2 \;=\; \|v - Pv\|^2 + \|Pv - w\|^2 \;\geq\; \|v - Pv\|^2,$$

with equality if and only if $\|Pv - w\|^2 = 0$, i.e. $w = Pv$. Therefore Pv is the unique point of W nearest to v. This is exactly what "P projects v orthogonally onto W" means. □

Remark 5.6. The converse is also true and equally short: an orthogonal projection is automatically self-adjoint (Garrett, n.d.; Wikipedia, n.d.f). We will only need the direction proved above. The single fact you must carry forward is the slogan

idempotent + self-adjoint $\implies$ orthogonal projection (the closest-point map).

Corollary 5.7 (Eigenvalues of a projector are 0 or 1)**.** *If $P^2 = P$ and $Pv = \lambda v$ with $v \neq 0$, then $\lambda v = Pv = P^2 v = \lambda(Pv) = \lambda^2 v$, so $\lambda = \lambda^2$, forcing $\lambda \in \{0, 1\}$ (Wikipedia, n.d.f). The eigenvectors with $\lambda = 1$ span the range (what P keeps); those with $\lambda = 0$ span the kernel (what P discards).*

5.3 Spectral norm: how much an operator can stretch

We also need one number that measures the largest stretching a linear map can do, because later it controls how a shift or a perturbation can grow.

Definition 5.8 (Spectral / operator norm)**.** For a matrix W the *spectral norm* (also operator norm) is the largest factor by which W can lengthen a unit vector:

$$\|W\|_2 \;=\; \max_{\|z\|=1} \|Wz\|.$$

Equivalently $\|W\|_2 = \sigma_{\max}(W)$, the largest *singular value* of W, which equals $\sqrt{\lambda_{\max}(W^*W)}$, the square root of the largest eigenvalue of W^*W.

The equality $\|W\|_2 = \sqrt{\lambda_{\max}(W^*W)}$ comes from maximizing the *Rayleigh quotient*: $\|Wz\|^2 = \langle Wz, Wz\rangle = \langle W^*Wz, z\rangle$, and over unit vectors this is largest exactly at the top eigenvector of the self-adjoint matrix W^*W, where it equals $\lambda_{\max}(W^*W)$.

Worked example 5.9. For $W = \left[\begin{smallmatrix} 3 & 0 \\ 0 & 1 \end{smallmatrix}\right]$ we have $W^*W = \left[\begin{smallmatrix} 9 & 0 \\ 0 & 1 \end{smallmatrix}\right]$, whose eigenvalues are 9 and 1, so $\|W\|_2 = \sqrt{9} = 3$. A useful sanity check on the whole section: an *orthogonal* projection never stretches, $\|P\|_2 \leq 1$, because $\|Pv\| \leq \|v\|$ is just the inequality $\|v - Pv\|^2 \geq 0$ rearranged through the Pythagoras identity of Step 4.

Problem 5.1. Build $P = aa^\top/(a^\top a)$ for $a = (2, 1)^\top$, so $P = \frac{1}{5}\left[\begin{smallmatrix} 4 & 2 \\ 2 & 1 \end{smallmatrix}\right]$. Verify $P^2 = P$ and $P = P^\top$, and check that P annihilates $(-1, 2)^\top$ (which is orthogonal to a). Then take the oblique $P' = \left[\begin{smallmatrix} 0 & 0 \\ 1 & 1 \end{smallmatrix}\right]$: confirm $P'^2 = P'$ but $P' \neq P'^\top$, find its range and kernel, and observe they are not perpendicular. The moral: Theorem 5.5's two hypotheses are both load-bearing.

6 The discrete Fourier transform and the structure of shifts

A previous version of this primer asserted, with no explanation, that "shift operators are diagonalized by the DFT." That sentence is true and it is the keystone of the whole subject, but it is useless until every word in it is defined and the claim is proved. This section does that, building up from scratch, following the "discover the transform, don't postulate it" approach of Bamieh (Bamieh, 2018), with the concrete computations of the MIT 18.06 notes (Johnson, n.d.) and the rigorous theorem–proof spine of the Rutgers notes (Goodman, n.d.).

Notation, fixed once. We work with vectors of length N, indexed $0, 1, \ldots, N-1$, and read every index *modulo* N (so index -1 means $N-1$, index N means 0, and so on; we think of the entries as sitting on a circle). The basic constant is the *primitive N-th root of unity*

$$\omega \;=\; e^{-2\pi i/N},$$

chosen with a minus sign to match the engineering / forward DFT convention used throughout this primer. It satisfies $\omega^N = 1$ but $\omega^m \neq 1$ for $0 < m < N$; its powers $1, \omega, \omega^2, \ldots, \omega^{N-1}$ are the N solutions of $z^N = 1$, equally spaced points on the unit circle. The *discrete Fourier transform* (DFT) matrix F has entries

$$F_{jk} \;=\; \omega^{jk}, \qquad 0 \le j, k \le N-1, \qquad \text{and we write } \widehat{x} = \mathrm{DFT}(x) = Fx.$$

Two facts about ω we will use constantly: $\overline{\omega} = e^{+2\pi i/N} = \omega^{-1}$ (conjugating flips the sign of the exponent), and the *finite geometric series*

$$\sum_{m=0}^{N-1} \omega^{rm} \;=\; \begin{cases} N, & \text{if } r \equiv 0 \pmod N, \\ \dfrac{1-\omega^{rN}}{1-\omega^r} = \dfrac{1-1}{1-\omega^r} = 0, & \text{otherwise,} \end{cases} \tag{8}$$

where the first case is "add N copies of 1" and the second uses $\omega^{rN} = (\omega^N)^r = 1$ while $\omega^r \neq 1$. Equation (8) is the engine behind both Parseval and the diagonalization.

6.1 What "diagonalize" means, and the shift operator

Definition 6.1 (Eigenvector, eigenvalue, diagonalize)**.** A nonzero vector v is an *eigenvector* of a matrix A with *eigenvalue* λ if $Av = \lambda v$: the matrix only stretches v, never tilts it. To *diagonalize* A is to find an invertible matrix V (its columns a full set of eigenvectors $v_1, \ldots, v_N$) so that $V^{-1}AV = D$ is diagonal, with the eigenvalues $\lambda_1, \ldots, \lambda_N$ on the diagonal. In that eigenvector basis the matrix acts by simple coordinatewise scaling. A family of matrices is *simultaneously diagonalized* by one V if the same eigenvector basis works for all of them at once.

Definition 6.2 (Cyclic shift)**.** The *cyclic (right) shift* S moves every entry one step forward around the circle:

$$(Sx)_k \;=\; x_{k-1} \quad (\text{indices mod } N), \qquad \text{e.g.} \quad S\,(x_0, x_1, x_2, x_3) = (x_3, x_0, x_1, x_2).$$

As a matrix S is a *permutation matrix* (a single 1 in each row and column). It is the most fundamental object of this whole section; we now find its eigenvectors.

6.2 The shift is diagonalized by the DFT

Theorem 6.3 (Eigenvectors of the shift)**.** *Let h_j denote the j-th column of the DFT matrix F, that is*

$$h_j \;=\; \big(\omega^{0\cdot j},\ \omega^{1\cdot j},\ \omega^{2\cdot j},\ \ldots,\ \omega^{(N-1)j}\big)^{\top} \;=\; \big(1,\ \omega^{j},\ \omega^{2j},\ \ldots,\ \omega^{(N-1)j}\big)^{\top}.$$

Then each h_j is an eigenvector of the cyclic shift S:

$$S\,h_j \;=\; \omega^{-j}\,h_j \qquad (j = 0, 1, \ldots, N-1),$$

so the eigenvalues of S are the N roots of unity $\{\omega^{-j}\} = \{1, \omega^{-1}, \ldots, \omega^{-(N-1)}\}$, the same set as $\{1, \omega, \ldots, \omega^{N-1}\}$ in a different order.

Proof. We chase one entry. The k-th entry of h_j is $(h_j)_k = \omega^{kj}$. Apply the definition of the shift, $(S\,h_j)_k = (h_j)_{k-1}$, and compute, for each k,

$$\begin{aligned}(S\,h_j)_k &= (h_j)_{k-1} \\ &= \omega^{(k-1)j} && \text{(definition of } h_j \text{ at index } k-1) \\ &= \omega^{kj}\,\omega^{-j} && \text{(law of exponents } \omega^{(k-1)j} = \omega^{kj}\cdot\omega^{-j}) \\ &= \omega^{-j}\,(h_j)_k.\end{aligned}$$

This holds for every entry k (the wrap-around at $k = 0$ is automatic because the exponent is read mod N and $\omega^N = 1$). Therefore $S\,h_j = \omega^{-j} h_j$ as vectors: h_j is an eigenvector with eigenvalue ω^{-j}. The N eigenvalues $\omega^0, \omega^{-1}, \ldots, \omega^{-(N-1)}$ are distinct, so the N eigenvectors $h_0, \ldots, h_{N-1}$ are linearly independent and form a basis: the columns of F diagonalize S. □

Remark 6.4 (A note on conventions, so you are never confused later). With the *forward* convention $\omega = e^{-2\pi i/N}$ adopted here, the shift eigenvalue attached to column h_j is ω^{-j}. Some textbooks (e.g. (Johnson, n.d.; Goodman, n.d.)) use the opposite sign $\omega = e^{+2\pi i/N}$ and obtain $Sh_j = \omega^j h_j$. The eigenvectors are the same lines, the eigenvalues are the same set of roots of unity; only the bookkeeping label changes. We will always state which convention is in force.

6.3 Circulant matrices: every shift-invariant linear map

Definition 6.5 (Circulant matrix and circular convolution)**.** A matrix C is *circulant* if each column is the cyclic shift of the previous one; equivalently its entries depend only on $k - l \bmod N$, written $(C_a)_{kl} = a_{k-l}$, where $a = (a_0, \ldots, a_{N-1})$ is its first column. The action of a circulant is exactly a *circular convolution*: writing $a \operatorname{conv} x$ for the convolution,

$$(C_a x)_k \;=\; \sum_{l=0}^{N-1} a_{k-l}\, x_l \;=\; (a \operatorname{conv} x)_k \qquad \text{(indices mod } N\text{)}.$$

A convolution "slides one signal across another and sums the overlaps." The shift itself is the circulant whose first column is $(0, 1, 0, \ldots, 0)$.

The reason circulants matter is that they are *precisely* the matrices that commute with the shift, and "commutes with the shift" is the algebraic spelling of "treats every position the same way" — the defining feature of a translation-blind operation (Bamieh, 2018; Bronstein et al., 2021).

Lemma 6.6 (Circulants are polynomials in the shift)**.** *A matrix C is circulant if and only if it is a polynomial in S,*

$$C \;=\; c_0 I + c_1 S + c_2 S^2 + \cdots + c_{N-1} S^{N-1},$$

where $(c_0, \ldots, c_{N-1})$ is the first column of C. In particular every circulant commutes with S, and any two circulants commute with each other.

Proof. Note first that S^p is the circulant that shifts by p places, $(S^p x)_k = x_{k-p}$. Hence a polynomial $\sum_p c_p S^p$ sends x to the vector with k-th entry $\sum_p c_p x_{k-p}$, which is the circular convolution $(c \operatorname{conv} x)_k$ — a circulant with first column c. Conversely, given a circulant C with first column c, the operator $\sum_p c_p S^p$ has the same first column ($S^p e_0 = e_p$, so $\sum_p c_p S^p$ applied to e_0 returns c) and, being shift-invariant, the same every column; so the two matrices agree. Finally S commutes with any polynomial in S, and two polynomials in S commute with each other because powers of S do (Goodman, n.d.). □

Theorem 6.7 (The DFT simultaneously diagonalizes all circulants; the convolution theorem)**.** *Let C_a be the circulant with first column a, equivalently $C_a x = a \operatorname{conv} x$. Then every column h_j of F is an eigenvector of C_a, and the eigenvalue is the j-th DFT coefficient of a:*

$$C_a\, h_j \;=\; \widehat{a}_j\, h_j, \qquad \text{where } \widehat{a}_j = (Fa)_j = \sum_{l=0}^{N-1} a_l\, \omega^{jl}.$$

Consequently the DFT turns convolution into entrywise multiplication — the convolution theorem*:*

$$\boxed{\operatorname{DFT}(a \operatorname{conv} x) \;=\; \operatorname{DFT}(a) \odot \operatorname{DFT}(x)} \qquad (\odot = \textit{entrywise product}).$$

Proof. Eigenvalues. Write $C_a = \sum_p a_p S^p$ as in Lemma 6.6. Apply it to the shift eigenvector h_j and use $Sh_j = \omega^{-j} h_j$ (Theorem 6.3), hence $S^p h_j = \omega^{-pj} h_j$:

$$C_a\, h_j \;=\; \sum_{p=0}^{N-1} a_p\, S^p h_j \;=\; \sum_{p=0}^{N-1} a_p\, \omega^{-pj}\, h_j \;=\; \underbrace{\Big(\sum_{p=0}^{N-1} a_p\, \omega^{-pj}\Big)}_{=\,(Fa)_{-j}=\widehat{a}_{-j}}\, h_j.$$

The bracket is a single number, so h_j is an eigenvector. (With our forward convention the eigenvalue on column h_j is $\widehat{a}_{-j}$; the whole *set* of eigenvalues is $\{\widehat{a}_0, \ldots, \widehat{a}_{N-1}\}$, just relabelled. The clean "eigenvalue = DFT of the first column" statement holds verbatim with the opposite-sign convention (Bamieh, 2018); we keep ours and note the relabelling.) Because the h_j form a basis, C_a is diagonal in that basis: this is what "simultaneously diagonalized" means, since the same columns of F work for *every* choice of a.

Convolution theorem (direct index chase, following (Goodman, n.d.; Wikipedia, n.d.b)). We compute the k-th DFT coefficient of $a \operatorname{conv} x$ from the definitions:

$$\begin{aligned}
\operatorname{DFT}(a \operatorname{conv} x)_k \;&=\; \sum_{j=0}^{N-1} (a \operatorname{conv} x)_j\, \omega^{kj} \\
&=\; \sum_{j=0}^{N-1} \Big(\sum_{l=0}^{N-1} a_{j-l}\, x_l\Big)\, \omega^{kj} && \text{(definition of convolution)} \\
&=\; \sum_{l=0}^{N-1} x_l \sum_{j=0}^{N-1} a_{j-l}\, \omega^{kj} && \text{(swap the two finite sums)} \\
&=\; \sum_{l=0}^{N-1} x_l \sum_{m=0}^{N-1} a_m\, \omega^{k(m+l)} && \text{(set } m = j-l\text{; still a full period)} \\
&=\; \sum_{l=0}^{N-1} x_l\, \omega^{kl} \sum_{m=0}^{N-1} a_m\, \omega^{km} && \text{(factor } \omega^{k(m+l)} = \omega^{km}\, \omega^{kl}\text{)} \\
&=\; \Big(\sum_{l} x_l \omega^{kl}\Big)\Big(\sum_{m} a_m \omega^{km}\Big) \;=\; \widehat{x}_k\, \widehat{a}_k.
\end{aligned}$$

That is exactly the entrywise product $\widehat{a} \odot \widehat{x}$ at index k. The substitution $m = j - l$ is legitimate because both a and the exponential are N-periodic in their index, so summing j over one full period is the same as summing m over one full period. □

6.4 Parseval: the DFT preserves energy (up to a factor of N)

Theorem 6.8 (Parseval / Plancherel)**.** *With the forward (unnormalized) DFT, for every vector $x \in \mathbb{C}^N$,*

$$\|\mathrm{DFT}(x)\|^2 \;=\; N\,\|x\|^2, \qquad \textit{equivalently} \qquad \|x\|^2 = \tfrac{1}{N}\,\|\mathrm{DFT}(x)\|^2.$$

(With the unitary *normalization $\frac{1}{\sqrt{N}}F$ it reads $\|x\| = \|\mathrm{DFT}(x)\|$ exactly.)*

Proof. The proof is the orthogonality of the columns of F, which is exactly the geometric series (8). Take two columns h_j, h_k of F and form their inner product (recall $\langle u, v\rangle = \sum_m u_m \overline{v_m}$, and $\overline{\omega^{km}} = \omega^{-km}$):

$$\langle h_j,\, h_k\rangle \;=\; \sum_{m=0}^{N-1} \omega^{jm}\,\overline{\omega^{km}} \;=\; \sum_{m=0}^{N-1} \omega^{jm}\,\omega^{-km} \;=\; \sum_{m=0}^{N-1} \omega^{(j-k)m} \;=\; \begin{cases} N, & j = k, \\ 0, & j \neq k, \end{cases}$$

where the last step is (8) with $r = j - k$. In matrix form this says $F^*F = NI$, i.e. the columns are orthogonal each of squared length N. Now expand the DFT in this basis. Writing $\widehat{x} = Fx$,

$$\|\widehat{x}\|^2 \;=\; \langle Fx,\, Fx\rangle \;=\; \langle F^*Fx,\, x\rangle \;=\; \langle Nx,\, x\rangle \;=\; N\,\|x\|^2,$$

where we moved F to the other slot of the inner product through its adjoint F^* and then used $F^*F = NI$. Dividing by N gives the second form. □

6.5 Worked examples at $N = 4$ (do these by hand once)

With $N = 4$ the root of unity is $\omega = e^{-2\pi i/4} = -i$, and the forward DFT matrix is

$$F_4 \;=\; \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{bmatrix}.$$

(The entry $F_{jk} = (-i)^{jk}$; the MIT 18.06 notes (Johnson, n.d.) print the complex-conjugate version because they use the + sign convention.)

Worked example 6.9 (Convolution two ways)**.** Let $a = (1, 1, 0, 0)$ and $x = (1, 2, 3, 4)$. Direct circular convolution: $(a \operatorname{conv} x)_k = \sum_l a_{k-l} x_l$ with $a_0 = a_1 = 1$, $a_2 = a_3 = 0$, so $(a \operatorname{conv} x)_k = x_k + x_{k-1}$:

$$a \operatorname{conv} x = (x_0 + x_3,\; x_1 + x_0,\; x_2 + x_1,\; x_3 + x_2) = (1 + 4,\; 2 + 1,\; 3 + 2,\; 4 + 3) = (5, 3, 5, 7).$$

Via the transform: $\widehat{a} = F_4 a = (2,\, 1 - i,\, 0,\, 1 + i)$ and $\widehat{x} = F_4 x = (10,\, -2 + 2i,\, -2,\, -2 - 2i)$. Their entrywise product is $\widehat{a} \odot \widehat{x} = (20,\, (1 - i)(-2 + 2i),\, 0,\, (1 + i)(-2 - 2i)) = (20,\, 4i,\, 0,\, -4i)$, hmm let us simplify: $(1 - i)(-2 + 2i) = -2 + 2i + 2i - 2i^2 = -2 + 4i + 2 = 4i$ and similarly the last entry is $-4i$. Applying the inverse DFT, $\frac{1}{4}F_4^*(20, 4i, 0, -4i)$, returns $(5, 3, 5, 7)$, matching the direct computation (Goodman, n.d.). The two routes agree exactly, which is the convolution theorem at work.

Worked example 6.10 (Parseval check)**.** For $x = (1, 2, -1, 0)$ we have $\|x\|^2 = 1 + 4 + 1 + 0 = 6$. Its DFT is $\widehat{x} = F_4 x = (2,\, 2 - 2i,\, -2,\, 2 + 2i)$, so $\|\widehat{x}\|^2 = |2|^2 + |2 - 2i|^2 + |-2|^2 + |2 + 2i|^2 = 4 + 8 + 4 + 8 = 24 = 4 \cdot 6 = N\|x\|^2$, exactly as Theorem 6.8 predicts.

Problem 6.1. Show by hand that the 4×4 cyclic shift S has eigenvalues equal to the 4th roots of unity $\{1, i, -1, -i\}$, with eigenvectors the columns of F_4. Then verify Parseval for $x = (1, 2, -1, 0)$ (the example above) and for one vector of your own choosing. Finally, confirm the convolution theorem on $a = (1, 1, 0, 0)$, $x = (1, 2, 3, 4)$ by reproducing both routes in the previous example.

7 Symmetry made precise: group actions, orbits, invariance, and group averaging

The DFT story was about *one* symmetry, the cyclic shift. To talk about shift-invariance and adversarial robustness in general we need the vocabulary that makes "a symmetry acting on data" precise. The right language is that of group actions, and the right tool is the operation that "averages a function over a symmetry" — the classical Reynolds operator. We follow the geometric deep learning treatment of Bronstein, Bruna, Cohen and Veličković (Bronstein et al., 2021) and the Wikipedia articles on group actions and equivariant maps (Wikipedia, n.d.d,n), downgrading their integrals over abstract groups to finite sums an undergraduate can compute.

7.1 Groups, actions, orbits

Definition 7.1 (Group)**.** A *group* $(G, \cdot)$ is a set G with a multiplication $\cdot$ that is associative, has an *identity* element e (with $eg = ge = g$ for all g), and gives every g an *inverse* g^{-1} (with $gg^{-1} = g^{-1}g = e$). The group is *abelian* (commutative) if $gh = hg$ always. The running example is the *cyclic group* $\mathbb{Z}_N = \{0, 1, \ldots, N-1\}$ under addition mod N, which is abelian and is generated by the single element 1 (every element is a repeated sum of 1's).

Definition 7.2 (Group action)**.** A (left) *action* of a group G on a set X is a rule $G \times X \to X$, written $(g, x) \mapsto g \cdot x$, satisfying two axioms (Wikipedia, n.d.d):

$$\text{(identity)} \quad e \cdot x = x, \qquad\qquad \text{(compatibility)} \quad g \cdot (h \cdot x) = (gh) \cdot x.$$

The identity axiom says "doing nothing does nothing"; the compatibility axiom says "doing h then g is the same as doing the product gh." When X is a vector space and each g acts by a linear (in fact invertible) map, the action is called a *representation*; we write that map as a matrix $\rho(g)$, and compatibility reads $\rho(g)\rho(h) = \rho(gh)$.

Worked example 7.3 (The cyclic shift action, the one we care about)**.** Let $G = \mathbb{Z}_N$ act on signals $x \in \mathbb{C}^N$ by shifting: the group element u acts by $\rho(u) = S^u$, so $(\, u \cdot x \,)_k = x_{k-u}$ (shift the signal forward by u). The identity $u = 0$ does nothing; compatibility $S^u S^v = S^{u+v}$ holds because shifting by u then by v shifts by $u + v$. This is the action that turns "$\mathbb{Z}_N$-symmetry" into "the shift operators of Section 6."

Definition 7.4 (Orbit)**.** The *orbit* of a point x is everything reachable from x by the group,

$$\mathcal{O}(x) \;=\; G \cdot x \;=\; \{\, g \cdot x : g \in G \,\}.$$

Orbits *partition* X: every point is in exactly one orbit, because the relation "$y = g \cdot x$ for some g" is reflexive (e), symmetric (g^{-1}) and transitive (compatibility). An orbit is the set of all configurations that the symmetry regards as "the same up to the symmetry."

Worked example 7.5 (Orbits of the shift on $\mathbb{R}^4$)**.** Under $\mathbb{Z}_4$ acting by shifts on $\mathbb{R}^4$: the orbit of $(1,0,0,0)$ is the four cyclic shifts $\{(1,0,0,0), (0,1,0,0), (0,0,1,0), (0,0,0,1)\}$ — the four standard basis vectors. The orbit of the constant vector $(1,1,1,1)$ is just itself: shifting it changes nothing, so it is a *fixed point*. Already you can feel the shift-invariance theme: a function that should not care about position must give the same value everywhere along an orbit.

7.2 Invariant versus equivariant

This distinction is the single most important conceptual point of the section, and the old primer never made it.

Definition 7.6 (Invariant and equivariant functions)**.** Let G act on a domain X and (for equivariance) also on a codomain Y. A function $f : X \to Y$ is

$$\textbf{invariant} \quad \text{if} \quad f(g \cdot x) = f(x) \ \ \forall g, \qquad\qquad \textbf{equivariant} \quad \text{if} \quad f(g \cdot x) = g \cdot f(x) \ \ \forall g.$$

An invariant function gives the same output no matter how the input is transformed: it *collapses each orbit to a single value.* An equivariant function lets the transformation pass through predictably: transform the input and the output transforms the same way. Invariance is the special case of equivariance where the action on the codomain is trivial (Wikipedia, n.d.c).

Worked example 7.7 (The textbook pictures)**.** For triangles under the rigid motions of the plane (translations, rotations, reflections): the *area* is invariant — moving a triangle does not change its area. The *centroid* is equivariant — move the triangle and its centroid moves to exactly the image of the old centroid (Wikipedia, n.d.c). In statistics, the sample *mean* is equivariant under rescaling of the data (change units, the mean changes units the same way), while the *median* is equivariant under *any* monotone relabelling. In machine learning the canonical pair is: image *classification* is shift-invariant (a cat is a cat wherever it sits), whereas *segmentation* is shift-equivariant (shift the image and the predicted mask shifts identically) (Bronstein et al., 2021).

Remark 7.8 (A warning the old primer should have given)*.* Convolutional layers are shift-*equivariant*, not invariant; this is exactly the statement $S\,C_a x = C_a\,Sx$ for a circulant C_a, which you already proved when you showed circulants commute with S (Lemma 6.6). Invariance is only obtained at the *end*, by a pooling/averaging step that collapses the orbit. Many signal-processing texts loosely call equivariance "shift-invariance"; do not be fooled (Bronstein et al., 2021).

7.3 The Reynolds (group-averaging) operator

How does one *manufacture* an invariant function from an arbitrary one? Average over the group. This idea is classical (Reynolds; Weyl and Hilbert in invariant theory) and is the symmetry layer's workhorse (Bronstein et al., 2021).

Definition 7.9 (Group average / Reynolds operator)**.** For a finite group G acting on a vector space, define the *group-averaging operator* R on functions (or, when G acts linearly, on vectors) by

$$(Rf)(x) \;=\; \frac{1}{|G|}\sum_{g\in G} f(g\cdot x), \qquad \text{or for the linear action on vectors:} \quad R \;=\; \frac{1}{|G|}\sum_{g\in G}\rho(g).$$

For a finite group this finite sum is the exact counterpart of Bronstein et al.'s continuous average $\frac{1}{\mu(G)}\int_G f(g\cdot x)\,d\mu(g)$ over the group's Haar measure (Bronstein et al., 2021): for a finite group the Haar measure just counts elements.

Theorem 7.10 (The group average is invariant, idempotent, and (for an orthogonal action) an orthogonal projection onto the invariants)**.** *Let G be finite and act linearly. Then:*

(i) *R is* invariant*: $R\,\rho(h) = R$ for every $h \in G$, i.e. $(Rf)(h\cdot x) = (Rf)(x)$.*

(ii) *R is* idempotent*: $R^2 = R$. Its range is exactly the invariant subspace $V_{\text{inv}} = \{v : \rho(g)v = v \ \forall g\}$.*

(iii) *If every $\rho(g)$ is* orthogonal *(or unitary), $\rho(g)^* = \rho(g)^{-1}$, then R is also self-adjoint, hence by Theorem 5.5 the* orthogonal *projection onto V_{inv} — the closest invariant vector to any given v.*

Proof. (i) Invariance, by re-indexing the group. Fix $h \in G$ and compute, using compatibility $\rho(g)\rho(h) = \rho(gh)$:

$$R\,\rho(h) \;=\; \frac{1}{|G|}\sum_{g\in G}\rho(g)\rho(h) \;=\; \frac{1}{|G|}\sum_{g\in G}\rho(gh) \;=\; \frac{1}{|G|}\sum_{g'\in G}\rho(g') \;=\; R.$$

The middle step is the only subtlety: as g runs once through all of G, the product $g' = gh$ also runs *once* through all of G (right multiplication by the fixed h is a bijection of G, with inverse right multiplication by h^{-1}). So we are summing the same $|G|$ terms in a different order. Hence R is unchanged by any group element on its input.

(ii) Idempotence and range. First, R fixes every invariant vector: if $\rho(g)v = v$ for all g, then $Rv = \frac{1}{|G|}\sum_g \rho(g)v = \frac{1}{|G|}\sum_g v = v$. Second, every output Rv *is* invariant: by part (i), $\rho(h)\,Rv = Rv$ would follow from $\rho(h)R = R$; let us verify that form too,

$$\rho(h)R = \frac{1}{|G|}\sum_{g}\rho(h)\rho(g) = \frac{1}{|G|}\sum_{g}\rho(hg) = \frac{1}{|G|}\sum_{g'}\rho(g') = R,$$

the same re-indexing as in (i) but with *left* multiplication. So $Rv \in V_{\text{inv}}$ for every v, and R fixes V_{inv} pointwise; therefore $R(Rv) = Rv$, i.e. $R^2 = R$, and the range of R is exactly V_{inv}.

(iii) Self-adjointness for an orthogonal action. Compute the adjoint of the sum, using $(\rho(g))^* = \rho(g)^{-1} = \rho(g^{-1})$:

$$R^* = \frac{1}{|G|}\sum_{g}\rho(g)^* = \frac{1}{|G|}\sum_{g}\rho(g^{-1}) = \frac{1}{|G|}\sum_{g'}\rho(g') = R,$$

where again $g' = g^{-1}$ ranges over all of G as g does (inversion is a bijection of G). So $R = R^*$. Now R is idempotent (part ii) and self-adjoint, so Theorem 5.5 applies: R is the orthogonal projection onto its range V_{inv}, the unique nearest invariant vector. This is the rigorous content of the slogan "averaging over the symmetry returns the closest symmetric thing." □

7.4 The cyclic case, and why linear invariants are too weak

Worked example 7.11 (Shift-averaging returns the mean)**.** Apply Theorem 7.10 to $G = \mathbb{Z}_N$ acting on $\mathbb{C}^N$ by shifts, $\rho(u) = S^u$. The group average is

$$(Rx)_k = \frac{1}{N}\sum_{u=0}^{N-1}(S^u x)_k = \frac{1}{N}\sum_{u=0}^{N-1}x_{k-u} = \frac{1}{N}\sum_{j=0}^{N-1}x_j = \text{mean}(x),$$

the constant vector all of whose entries equal the average of x. Each S^u is a permutation matrix, hence orthogonal, so by part (iii) this R is the orthogonal projection onto the one-dimensional invariant subspace spanned by $(1,1,\dots,1)$ — which is exactly the orbit of fixed points from Example 7.5. Connecting to Section 6: $R = \frac{1}{N}\sum_u S^u$ is a circulant, and by Theorem 6.7 its eigenvalues are the DFT of its first column $\frac{1}{N}(1,1,\dots,1)$, which is $(1,0,0,\dots,0)$ — it keeps only the "zero frequency" (the mean) and kills everything else.

The example carries a moral that motivates the entire next section. The *only* linear shift-invariant feature of a signal is its mean (any other shift-invariant linear functional must be a multiple of the sum, by the same averaging argument). Bronstein et al. make the same point vividly: for images under translation, a linear invariant can see nothing but the *average colour* (Bronstein et al., 2021). To get richer invariants — ones that can actually tell two signals apart while ignoring their position — we must go *nonlinear*. That is precisely what the power spectrum and bispectrum do.

Problem 7.1. Show that under $\mathbb{Z}_N$ the group average of any vector equals the constant vector whose entries are the mean; verify directly that $R^2 = R$ on a length-3 example; and argue that the only shift-invariant *linear* functionals $x \mapsto \langle w, x\rangle$ are the multiples of the sum $\sum_k x_k$ (hint: invariance forces $S^* w = w$, so w is itself shift-invariant). Conclude that nontrivial shift-invariants must be nonlinear.

8 Shift-invariant descriptors: the power spectrum and the bispectrum

We now build the two nonlinear shift-invariants that the project actually uses. Both are computed from the DFT, both throw away absolute position, and the difference between them — one discards phase, the other keeps relative phase — is the punchline. The sources are the MIT 6.011 notes on power spectral density (MIT OpenCourseWare, n.d.), de Buyl's Wiener–Khinchin derivation (de Buyl, n.d.), Kakarala's introduction to bispectral invariants (Kakarala, 2012), and Chen, Zehni and Zhao's discrete treatment (Chen et al., 2018).

8.1 The DFT shift theorem: shifting only rotates phase

Lemma 8.1 (Shift theorem)**.** *Let R_τ shift a signal by τ, $(R_\tau x)_n = x_{n-\tau}$. Then*

$$\mathrm{DFT}(R_\tau x)_k \;=\; \widehat{x}_k\, \omega^{k\tau} \;=\; \widehat{x}_k\, e^{-2\pi i k\tau/N}.$$

A shift leaves the magnitude $|\widehat{x}_k|$ *of every Fourier coefficient untouched and only multiplies it by a* unit-modulus phase $e^{-2\pi i k\tau/N}$ *(a complex number of absolute value* 1*).*

Proof. Compute from the definition, substituting $m = n - \tau$ and using periodicity:

$$\begin{aligned}\mathrm{DFT}(R_\tau x)_k \;&=\; \sum_{n=0}^{N-1} (R_\tau x)_n\, \omega^{kn} \;=\; \sum_{n=0}^{N-1} x_{n-\tau}\, \omega^{kn} \;=\; \sum_{m=0}^{N-1} x_m\, \omega^{k(m+\tau)} \\ &=\; \omega^{k\tau} \sum_{m=0}^{N-1} x_m\, \omega^{km} \;=\; \omega^{k\tau}\, \widehat{x}_k.\end{aligned}$$

□

Everything below is a consequence of one observation: *position lives entirely in the phase.* Any quantity built from the $\widehat{x}_k$ that cannot detect a phase-rotation $e^{-2\pi i k\tau/N}$ will be shift-invariant.

8.2 Power spectrum and Wiener–Khinchin

Definition 8.2 (Autocorrelation and power spectrum)**.** The *circular autocorrelation* of x is $r_\tau = \sum_{t=0}^{N-1} x_t\, x_{t+\tau}$ (for real signals; conjugate the second factor for complex ones), measuring how much x resembles its own shift by τ. The *power spectrum* is the squared magnitude of the DFT,

$$P_k \;=\; |\widehat{x}_k|^2 \;=\; \widehat{x}_k\, \overline{\widehat{x}_k}.$$

It is real, nonnegative, and (for real x) symmetric (MIT OpenCourseWare, n.d.).

Theorem 8.3 (Power spectrum is shift-invariant)**.** *For every shift* τ, $|\mathrm{DFT}(R_\tau x)_k|^2 = |\widehat{x}_k|^2$.

Proof. Using the shift theorem (Lemma 8.1) and $|e^{-2\pi i k\tau/N}| = 1$:

$$|\mathrm{DFT}(R_\tau x)_k|^2 = \left|\, \widehat{x}_k\, e^{-2\pi i k\tau/N} \,\right|^2 = |\widehat{x}_k|^2 \cdot \left|e^{-2\pi i k\tau/N}\right|^2 = |\widehat{x}_k|^2 \cdot 1 = |\widehat{x}_k|^2.$$

The phase, which is where the position information sat, is annihilated by the absolute-value square. Hence the power spectrum is the same for x and every shift of x: it is a shift-invariant descriptor. □

Theorem 8.4 (Wiener–Khinchin: power spectrum = transform of autocorrelation)**.** *The power spectrum is the DFT of the autocorrelation:* $\widehat{r} = P$*, equivalently the autocorrelation is the inverse DFT of the power spectrum.*

Proof skeleton (continuous version, following de Buyl (de Buyl, n.d.)) Start from the autocorrelation

$$C(\tau) = \lim_{T\to\infty} \tfrac{1}{T} \int_0^T x(t)\, x(t+\tau)\, dt$$

and replace each x by its inverse Fourier transform. After exchanging the order of integration one is left with a time-average of $e^{i(\omega+\omega')t}$: that average is 0 unless $\omega + \omega' = 0$ (the oscillations cancel to order $1/T$) and equals 1 when $\omega + \omega' = 0$. Keeping only the surviving $\omega' = -\omega$ terms and using realness $\widetilde{x}(-\omega) = \overline{\widetilde{x}(\omega)}$ gives

$$C(\tau) = \frac{1}{2\pi} \int e^{i\omega\tau}\, \widetilde{x}(\omega)\, \overline{\widetilde{x}(\omega)}\, d\omega = \frac{1}{2\pi} \int e^{i\omega\tau}\, |\widetilde{x}(\omega)|^2\, d\omega,$$

i.e. the autocorrelation is the inverse transform of $|\widetilde{x}|^2$, the power spectrum. The discrete version replaces the integrals by the length-N DFT; the random (noisy) version is handled by MIT 6.011's periodogram argument (MIT OpenCourseWare, n.d.). □

Why does this matter? Because it shows the power spectrum is a *complete summary of the second-order structure* of a signal — but *only* the second order. By discarding all phase it cannot tell apart two signals whose Fourier magnitudes agree; Kakarala's Figure 1 shows two genuinely different shapes with identical power spectra, the power spectrum "fooled" (Kakarala, 2012). To recover the lost discriminative power without giving up shift-invariance, we keep *relative* phase.

8.3 The bispectrum: a shift-invariant that remembers relative phase

Definition 8.5 (Triple correlation and bispectrum)**.** The *triple correlation* is the autocorrelation with one extra shift, $a_3(s_1, s_2) = \sum_t \overline{x_t}\, x_{t+s_1}\, x_{t+s_2}$. Its DFT is the *bispectrum*, which in frequency variables is the triple product

$$B_{k_1,k_2} \;=\; \widehat{x}_{k_1}\, \widehat{x}_{k_2}\, \overline{\widehat{x}_{k_1+k_2}}$$

(Kakarala, 2012; Wikipedia, n.d.a). Chen–Zehni–Zhao write the equivalent form

$$B_{k_1,k_2} \;=\; \widehat{x}_{k_1}\, \overline{\widehat{x}_{k_2}}\, \widehat{x}_{k_2-k_1}$$

(Chen et al., 2018); the two differ only by a relabelling of the frequency axes. Setting $k_2 = 0$ recovers the power spectrum: $B_{k,0} = \widehat{x}_k\, \widehat{x}_0\, \overline{\widehat{x}_k} = \widehat{x}_0\, |\widehat{x}_k|^2$, so the bispectrum *contains* the power spectrum (Kakarala, 2012).

Theorem 8.6 (Bispectrum is shift-invariant, by explicit phase cancellation)**.** *For every shift* τ, $B^{(R_\tau x)}_{k_1,k_2} = B^{(x)}_{k_1,k_2}$.

Proof. Apply the shift theorem (Lemma 8.1) to each of the three Fourier coefficients in the triple product. Write $\zeta = e^{-2\pi i\tau/N}$, so a shift multiplies $\widehat{x}_k$ by ζ^k, and *conjugating* $\widehat{x}_{k_1+k_2}$ multiplies by $\overline{\zeta^{k_1+k_2}} = \zeta^{-(k_1+k_2)}$:

$$\begin{aligned}
B^{(R_\tau x)}_{k_1,k_2} &= \mathrm{DFT}(R_\tau x)_{k_1}\,\mathrm{DFT}(R_\tau x)_{k_2}\,\overline{\mathrm{DFT}(R_\tau x)_{k_1+k_2}} \\
&= \left(\zeta^{k_1}\widehat{x}_{k_1}\right)\left(\zeta^{k_2}\widehat{x}_{k_2}\right)\overline{\left(\zeta^{k_1+k_2}\widehat{x}_{k_1+k_2}\right)} && \text{(shift theorem on each factor)} \\
&= \zeta^{k_1}\,\zeta^{k_2}\,\zeta^{-(k_1+k_2)}\,\widehat{x}_{k_1}\,\widehat{x}_{k_2}\,\overline{\widehat{x}_{k_1+k_2}} && \text{(conjugation flips the phase)} \\
&= \zeta^{\,k_1+k_2-(k_1+k_2)}\,\widehat{x}_{k_1}\,\widehat{x}_{k_2}\,\overline{\widehat{x}_{k_1+k_2}} \\
&= \zeta^0\,B^{(x)}_{k_1,k_2} \;=\; B^{(x)}_{k_1,k_2}.
\end{aligned}$$

The three shift-phases cancel exactly: $k_1 + k_2 - (k_1 + k_2) = 0$, so $\zeta^0 = 1$. That cancellation is the whole point — it is why the bispectrum is shift-invariant (Kakarala, 2012; Chen et al., 2018). Unlike the power spectrum, though, the bispectrum did *not* have to take an absolute value to achieve invariance: it kept the *relative* phase among the three coefficients (the part of the phase that a global shift cannot touch), and that retained phase is exactly the extra information that lets it separate signals the power spectrum confuses. □

8.4 Worked examples at $N = 4$

Worked example 8.7 (A shift leaves $|\mathrm{DFT}|^2$ unchanged)**.** Take $x = (1, 0, 0, 0)$. Then $\widehat{x} = F_4 x = (1, 1, 1, 1)$, so $P = (1, 1, 1, 1)$. Now shift to $x' = (0, 1, 0, 0)$: $\widehat{x'} = (1, -i, -1, i)$, and $|\widehat{x'}|^2 = (1, 1, 1, 1)$ again — identical power spectrum, even though every phase changed.

Worked example 8.8 (Discrete Wiener–Khinchin by hand)**.** Take $x = (2, 1, 0, 1)$. Its circular autocorrelation $r_\tau = \sum_t x_t x_{t+\tau}$ is $r = (6, 4, 2, 4)$ (e.g. $r_0 = 4 + 1 + 0 + 1 = 6$). Its DFT is $\widehat{r} = F_4 r = (16, 4, 0, 4)$. Independently, $\widehat{x} = F_4 x = (4, 2, 0, 2)$, so the power spectrum is $|\widehat{x}|^2 = (16, 4, 0, 4)$. The two agree: $\widehat{r} = |\widehat{x}|^2$, exactly Theorem 8.4.

Worked example 8.9 (Same power spectrum, different bispectrum — the disambiguation)**.** Build two real length-4 signals

$$x_A = (1.25,\ 0.25,\ 0.25,\ 0.25), \qquad x_B = (0.75,\ -0.25,\ 0.75,\ 0.75).$$

Their DFTs are $\widehat{x}_A = (2, 1, 1, 1)$ and $\widehat{x}_B = (2,\ i,\ 1,\ -i)$, so both have the *identical* magnitude spectrum $|\widehat{x}| = (2, 1, 1, 1)$ and hence the identical power spectrum $(4, 1, 1, 1)$. The power spectrum cannot tell them apart. The bispectrum can: with $B_{k_1,k_2} = \widehat{x}_{k_1}\widehat{x}_{k_2}\overline{\widehat{x}_{k_1+k_2}}$,

$$B^{(A)}_{1,1} = \widehat{x}_{A,1}\,\widehat{x}_{A,1}\,\overline{\widehat{x}_{A,2}} = 1 \cdot 1 \cdot 1 = +1, \qquad B^{(B)}_{1,1} = \widehat{x}_{B,1}\,\widehat{x}_{B,1}\,\overline{\widehat{x}_{B,2}} = i \cdot i \cdot 1 = -1.$$

The two differ ($+1$ versus -1), entirely because of the relative phase on the first coefficient. And since both are shift-invariant, this distinction survives every shift: shifting x_B to $(0.75, 0.75, -0.25, 0.75)$ leaves $|\widehat{x}|^2 = (4, 1, 1, 1)$ and $B_{1,1} = -1$ unchanged. This is the discrete analogue of Kakarala's "two shapes the power spectrum confuses but the bispectrum separates" (Kakarala, 2012).

Problem 8.1. For $x = (2, 1, 0, 1)$: (a) verify the discrete Wiener–Khinchin identity $\widehat{r} = |\widehat{x}|^2$ as in Example 8.8; (b) shift x by 2 and confirm $|\widehat{x}|^2$ is unchanged while the individual phases rotate; (c) construct a second signal with the same $|\widehat{x}|$ but a flipped phase on one coefficient, and show that some bispectrum entry B_{k_1,k_2} differs — so the bispectrum distinguishes what the power spectrum cannot, even up to shift.

Where this leaves us. The four sections fit together as one thread. A shift-invariant feature is what you get by averaging away the symmetry (Section 7); doing it *linearly* keeps only the mean, so useful invariants are nonlinear (Section 8); the natural nonlinear ones live in the Fourier domain, where shifting is just a phase rotation (Section 6), and the whole averaging-is-projection story rests on the one projector theorem of Section 5. The power spectrum is the phase-blind invariant; the bispectrum is the phase-retaining one; both are exactly what the project measures when it asks whether a network has learned to ignore position.

9 Margins and the max-margin classifier

Everything in this primer eventually comes back to a single geometric quantity: *how far is a data point from the surface where the classifier changes its mind?* That distance is called the *margin*, and a classifier that makes it as large as possible is a *max-margin classifier*, the simplest of which is the (hard-margin) *support vector machine*, or SVM (Boser et al., 1992; Cortes and Vapnik, 1995; Vapnik, 1998). We build the idea up slowly, prove the one formula that every textbook quietly assumes, and then look at the same object from a second angle (closest points of two convex hulls) that will make the later robustness story intuitive.

9.1 Hyperplanes and two notions of margin

We work in $\mathbb{R}^d$. Our data are pairs $(\mathbf{x}_i, y_i)$ with inputs $\mathbf{x}_i \in \mathbb{R}^d$ and binary labels $y_i \in \{+1, -1\}$. A *linear classifier* predicts the label of a new point $\mathbf{x}$ by looking at the sign of an affine function $\mathbf{w} \cdot \mathbf{x} + b$, where $\mathbf{w} \in \mathbb{R}^d$ is a *weight vector* and $b \in \mathbb{R}$ is a *bias* (also called an *intercept* or *threshold*). Here $\mathbf{w} \cdot \mathbf{x} = \sum_{j=1}^{d} w_j x_j$ is the ordinary dot product. The prediction is $+1$ when $\mathbf{w} \cdot \mathbf{x} + b > 0$ and -1 when it is negative.

Definition 9.1 (Hyperplane)**.** A *hyperplane* in $\mathbb{R}^d$ is the set of points $\{\mathbf{x} : \mathbf{w} \cdot \mathbf{x} + b = 0\}$ for some nonzero $\mathbf{w}$ and scalar b. It is a flat set of dimension $d-1$: a line in $\mathbb{R}^2$, an ordinary plane in $\mathbb{R}^3$. The vector $\mathbf{w}$ is *normal* (perpendicular) to the hyperplane, and the hyperplane is the *decision boundary* of the classifier: the place where the score $\mathbf{w} \cdot \mathbf{x} + b$ is exactly zero, so the predicted label flips.

Why is $\mathbf{w}$ perpendicular to the hyperplane? Take any two points $\mathbf{x}$ and $\mathbf{x}'$ that lie on it. Both satisfy the equation, so $\mathbf{w} \cdot \mathbf{x} + b = 0$ and $\mathbf{w} \cdot \mathbf{x}' + b = 0$. Subtracting, $\mathbf{w} \cdot (\mathbf{x} - \mathbf{x}') = 0$. The vector $\mathbf{x} - \mathbf{x}'$ points along the hyperplane (it is the difference of two of its points), and it is orthogonal to $\mathbf{w}$; since $\mathbf{x}, \mathbf{x}'$ were arbitrary, $\mathbf{w}$ is orthogonal to every direction inside the hyperplane.

There are two different things one might call "the margin," and keeping them apart is the single most common source of confusion, so we name them both (Burges, 1998).

Definition 9.2 (Functional and geometric margin)**.** For a labelled point $(\mathbf{x}_i, y_i)$ and a classifier $(\mathbf{w}, b)$, the *functional margin* is

$$\hat{\gamma}_i \;=\; y_i \,(\mathbf{w} \cdot \mathbf{x}_i + b).$$

The *geometric margin* is the functional margin divided by the length of the weight vector,

$$\gamma_i \;=\; \frac{y_i \,(\mathbf{w} \cdot \mathbf{x}_i + b)}{\|\mathbf{w}\|},$$

where $\|\mathbf{w}\| = \sqrt{\mathbf{w} \cdot \mathbf{w}}$ is the Euclidean norm.

The functional margin is positive exactly when the point is classified correctly (the label y_i and the score share a sign). But it is *not* a geometric distance: if we multiply both $\mathbf{w}$ and b by 10, the decision boundary does not move at all (the equation $10\mathbf{w}\cdot\mathbf{x}+10b=0$ has the same solution set), yet the functional margin grows tenfold. The geometric margin fixes this: it is *scale-invariant* (replacing $(\mathbf{w},b)$ by $(c\mathbf{w},cb)$ for $c>0$ leaves it unchanged because the c in the numerator cancels the c in $\|c\mathbf{w}\|=c\|\mathbf{w}\|$), and as we now prove it equals the honest perpendicular distance from $\mathbf{x}_i$ to the boundary.

9.2 The point-to-hyperplane distance, with proof

Burges' classic tutorial states the distance from a point to a hyperplane as a fact and moves on (Burges, 1998). Because this formula is the backbone of margins, certificates and attacks alike, we supply the proof in full.

Theorem 9.3 (Point-to-hyperplane distance)**.** *Let $H=\{\mathbf{x}:\mathbf{w}\cdot\mathbf{x}+b=0\}$ with $\mathbf{w}\neq\mathbf{0}$, and let $\mathbf{x}_0\in\mathbb{R}^d$ be any point. The Euclidean distance from $\mathbf{x}_0$ to H is*

$$\operatorname{dist}(\mathbf{x}_0,H) \;=\; \frac{|\mathbf{w}\cdot\mathbf{x}_0+b|}{\|\mathbf{w}\|}.$$

Proof. The distance from $\mathbf{x}_0$ to the set H is by definition the smallest value of $\|\mathbf{x}_0-\mathbf{x}\|$ over all $\mathbf{x}\in H$. We will (i) guess the closest point by walking from $\mathbf{x}_0$ straight toward H along the normal direction, and (ii) prove no point of H is nearer.

Step 1 (a unit normal). Set $\mathbf{n}=\mathbf{w}/\|\mathbf{w}\|$, the normal $\mathbf{w}$ rescaled to length one, so $\mathbf{n}\cdot\mathbf{n}=1$.

Step 2 (walk along the normal). Consider the point obtained by moving a signed amount $t\in\mathbb{R}$ from $\mathbf{x}_0$ along $\mathbf{n}$,

$$\mathbf{p}(t) \;=\; \mathbf{x}_0 - t\,\mathbf{n}.$$

We want the t that lands $\mathbf{p}(t)$ on H, i.e. that makes $\mathbf{w}\cdot\mathbf{p}(t)+b=0$.

Step 3 (solve for t). Substitute and expand, using linearity of the dot product in the variable $\mathbf{p}$,

$$\begin{aligned}\mathbf{w}\cdot\mathbf{p}(t)+b &= \mathbf{w}\cdot(\mathbf{x}_0-t\,\mathbf{n})+b\\ &= \mathbf{w}\cdot\mathbf{x}_0 \;-\; t\,(\mathbf{w}\cdot\mathbf{n}) \;+\; b.\end{aligned}$$

Now $\mathbf{w}\cdot\mathbf{n}=\mathbf{w}\cdot(\mathbf{w}/\|\mathbf{w}\|)=\|\mathbf{w}\|^2/\|\mathbf{w}\|=\|\mathbf{w}\|$. Setting the expression to zero and solving for t,

$$\mathbf{w}\cdot\mathbf{x}_0 - t\,\|\mathbf{w}\|+b=0 \quad\Longrightarrow\quad t^\star \;=\; \frac{\mathbf{w}\cdot\mathbf{x}_0+b}{\|\mathbf{w}\|}.$$

So the foot of the perpendicular is $\mathbf{p}(t^\star)\in H$, and the length of the step we took is

$$\|\mathbf{x}_0-\mathbf{p}(t^\star)\| = \|t^\star\,\mathbf{n}\| = |t^\star|\,\|\mathbf{n}\| = |t^\star| = \frac{|\mathbf{w}\cdot\mathbf{x}_0+b|}{\|\mathbf{w}\|}.$$

Step 4 (no point of H is nearer). Let $\mathbf{x}\in H$ be arbitrary, so $\mathbf{w}\cdot\mathbf{x}+b=0$. Write the gap from $\mathbf{x}_0$ as $\mathbf{x}_0-\mathbf{x}=\big(\mathbf{x}_0-\mathbf{p}(t^\star)\big)+\big(\mathbf{p}(t^\star)-\mathbf{x}\big)$. The first piece equals $t^\star\mathbf{n}$ (a multiple of the normal). The second piece, $\mathbf{p}(t^\star)-\mathbf{x}$, is the difference of two points of H, hence (as shown above) orthogonal to $\mathbf{w}$ and so to $\mathbf{n}$. The two pieces are therefore orthogonal, and Pythagoras gives

$$\|\mathbf{x}_0-\mathbf{x}\|^2 = \|t^\star\mathbf{n}\|^2 + \|\mathbf{p}(t^\star)-\mathbf{x}\|^2 \;\geq\; \|t^\star\mathbf{n}\|^2 = |t^\star|^2,$$

with equality iff $\mathbf{p}(t^\star)=\mathbf{x}$. Hence the minimum distance is exactly $|t^\star|=|\mathbf{w}\cdot\mathbf{x}_0+b|/\|\mathbf{w}\|$. □

Two corollaries we will reuse constantly. First, the distance from the hyperplane to the *origin* (set $\mathbf{x}_0 = \mathbf{0}$) is $|b|/\|\mathbf{w}\|$. Second, comparing with the geometric margin: $\gamma_i = y_i(\mathbf{w}\cdot\mathbf{x}_i + b)/\|\mathbf{w}\|$ is precisely the signed distance of $\mathbf{x}_i$ to the boundary, positive when the point is on its correct side. *The geometric margin is a distance.*

9.3 Canonical scaling, support vectors, and the hard-margin SVM

Because the geometric margin does not change under rescaling, we are free to fix the scale of $(\mathbf{w}, b)$ however is most convenient. The convention everyone uses is the *canonical* one (Burges, 1998): rescale so that the closest correctly classified points sit at functional margin exactly one. Concretely we require

$$y_i\,(\mathbf{w}\cdot\mathbf{x}_i + b) \;\geq\; 1 \qquad \text{for all } i, \tag{9}$$

with equality achieved by the nearest points. Under this convention the two "rails" $\mathbf{w}\cdot\mathbf{x} + b = +1$ and $\mathbf{w}\cdot\mathbf{x} + b = -1$ are parallel to the boundary and sit on either side of it. By Theorem 9.3 each rail is at distance $1/\|\mathbf{w}\|$ from the boundary, so the total width of the separating slab is

$$\text{margin} \;=\; d_+ + d_- \;=\; \frac{1}{\|\mathbf{w}\|} + \frac{1}{\|\mathbf{w}\|} \;=\; \frac{2}{\|\mathbf{w}\|},$$

where d_+ (d_-) is the distance to the nearest positive (negative) point (Burges, 1998).

Definition 9.4 (Support vectors)**.** The *support vectors* are the training points that actually touch a rail, i.e. for which equality holds in (9): $y_i(\mathbf{w}\cdot\mathbf{x}_i + b) = 1$. They are the points lying exactly on the slab boundary; the solution is determined entirely by them, and moving or deleting a non-support point (one strictly inside its own half) does not change the classifier.

We want the widest slab, i.e. the largest $2/\|\mathbf{w}\|$. Since $2/\|\mathbf{w}\|$ is a strictly decreasing function of $\|\mathbf{w}\|$, maximizing the margin is the same as minimizing $\|\mathbf{w}\|$, and equivalently $\tfrac{1}{2}\|\mathbf{w}\|^2$ (a smooth, convex stand-in). This is the *hard-margin SVM* (Boser et al., 1992; Vapnik, 1998; Burges, 1998):

$$\min_{\mathbf{w},b}\; \tfrac{1}{2}\|\mathbf{w}\|^2 \qquad \text{subject to} \qquad y_i\,(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 \;\; \forall i. \tag{10}$$

Proposition 9.5 (Max-margin $\Longleftrightarrow$ minimum norm)**.** *Among all $(\mathbf{w}, b)$ satisfying the canonical constraints (9), the one with the largest geometric margin is exactly the minimizer of $\tfrac{1}{2}\|\mathbf{w}\|^2$ in (10).*

Proof. The margin under canonical scaling is $2/\|\mathbf{w}\|$. The map $r \mapsto 2/r$ is strictly decreasing on $r > 0$, so its maximum over the feasible set is attained precisely where $\|\mathbf{w}\|$ (hence $\tfrac{1}{2}\|\mathbf{w}\|^2$) is smallest. The two optimizations have the same minimizer. □

Problem (10) is a convex quadratic program with linear constraints, so it has a unique global solution $\mathbf{w}$; the *Karush–Kuhn–Tucker* (KKT) conditions characterize it. We pause to define KKT cleanly because it returns in Sections 10 and is the language all of these results speak.

Definition 9.6 (KKT point)**.** Consider $\min_{\mathbf{z}} f(\mathbf{z})$ subject to inequality constraints $g_i(\mathbf{z}) \geq 0$. A point $\mathbf{z}^\star$ is a *KKT point* if there exist multipliers $\alpha_i \geq 0$ (one per constraint) such that

(i) *stationarity:* $\nabla f(\mathbf{z}^\star) = \sum_i \alpha_i \nabla g_i(\mathbf{z}^\star)$;
(ii) *primal feasibility:* $g_i(\mathbf{z}^\star) \geq 0$ for all i;
(iii) *dual feasibility:* $\alpha_i \geq 0$ for all i;
(iv) *complementary slackness:* $\alpha_i\, g_i(\mathbf{z}^\star) = 0$ for all i.

For a convex problem the KKT conditions are necessary *and* sufficient for a global minimum; for a non-convex problem they are only necessary (a KKT point need not be a global minimizer). Intuitively, a multiplier α_i is the "price" of constraint i, and complementary slackness says we only pay for constraints that are tight ($g_i = 0$).

Applying Definition 9.6 to the SVM (10) with $g_i(\mathbf{w}, b) = y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1$, complementary slackness reads

$$\alpha_i \left[y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 \right] = 0.$$

So $\alpha_i > 0$ forces $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) = 1$: the points with nonzero multiplier are exactly the support vectors. Stationarity in $\mathbf{w}$ gives $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$, so the optimal weight vector is a nonnegative combination of the support vectors alone. *Remember this: the SVM solution is built only from the points that sit on the margin.* Section 10 will show plain gradient descent rediscovers exactly this same combination on its own.

9.4 The convex-hull view: closest points of two hulls

There is a second picture, due to Bennett and Bredensteiner (Bennett and Bredensteiner, 2000), that many people find more intuitive than the algebra and which previews the robustness story. It needs one definition.

Definition 9.7 (Convex hull)**.** The *convex hull* $\operatorname{conv}(A)$ of a finite set $A = \{\mathbf{a}_1, \ldots, \mathbf{a}_m\}$ is the set of all *convex combinations* of its points,

$$\operatorname{conv}(A) = \left\{ \sum_i u_i \mathbf{a}_i \;:\; u_i \geq 0,\ \sum_i u_i = 1 \right\}.$$

It is the smallest convex set containing A: in the plane, the rubber band stretched around the points.

Suppose the two classes are linearly separable. Let A be the positive points and B the negative points. Bennett's theorem says the max-margin hyperplane is determined by the two closest points of the two hulls.

Theorem 9.8 (Closest points of the hulls (Bennett and Bredensteiner, 2000))**.** *For separable classes, let*

$$(\mathbf{c}^\star, \mathbf{d}^\star) = \arg \min_{\mathbf{c} \in \operatorname{conv}(A),\, \mathbf{d} \in \operatorname{conv}(B)} \tfrac{1}{2} \|\mathbf{c} - \mathbf{d}\|^2$$

be the closest pair of points, one in each hull. Then the max-margin hyperplane is the perpendicular bisector of the segment $[\mathbf{c}^\star, \mathbf{d}^\star]$, its normal is $\mathbf{w} \propto \mathbf{c}^\star - \mathbf{d}^\star$, and the margin (slab width) equals the distance between the hulls, $\|\mathbf{c}^\star - \mathbf{d}^\star\|$.

Proof skeleton. The argument is by contradiction on optimality (Bennett and Bredensteiner, 2000). (a) *The normal must be along $\mathbf{c}^\star - \mathbf{d}^\star$.* If the optimal separating slab's normal were *not* parallel to the segment joining the closest hull points, then the two parallel supporting planes (one touching each hull) would not be as far apart as possible: tilting the slab to align with the segment would increase the gap, contradicting maximality. (b) *The segment is orthogonal to the supporting planes.* If it were not, then the foot of one endpoint could be slid along its hull to a strictly nearer point, contradicting that $\mathbf{c}^\star, \mathbf{d}^\star$ are the closest pair. Conditions (a) and (b) together force the boundary to be the perpendicular bisector of $[\mathbf{c}^\star, \mathbf{d}^\star]$ with normal $\mathbf{c}^\star - \mathbf{d}^\star$; the slab width is then the length of that segment. The full statement is that the "closest points of the hulls" problem is the Wolfe dual of the "push two parallel supporting planes apart" problem, so they have the same solution (Bennett and Bredensteiner, 2000). □

The geometric reading is worth keeping: *maximizing the margin is the same as finding the two nearest points of the two clouds and bisecting the segment between them.* (When classes overlap, Bennett shrinks each hull by capping every weight $u_i \le 1/K$ – the *reduced convex hull* – and shows this softening of the hulls is the dual of allowing margin violations, the soft-margin SVM of Cortes and Vapnik (1995). We will not need the soft-margin machinery, but it is good to know the picture extends.)

9.5 Two worked examples and an exercise

Worked example 9.9 (Two points, margin 2)**.** Take a single positive point $\mathbf{x}_+ = (2, 0)$ with $y = +1$ and a single negative point $\mathbf{x}_- = (0, 0)$ with $y = -1$ in $\mathbb{R}^2$. By symmetry the boundary is the vertical line halfway between them, $x_1 = 1$. Let us verify both ways.

Canonical algebra. The boundary is normal to the x_1-axis, so $\mathbf{w} = (w_1, 0)$. The canonical constraints with equality at both points read $w_1 \cdot 2 + b = +1$ and $-(w_1 \cdot 0 + b) = +1$, i.e. $b = -1$ and $2w_1 - 1 = 1$, giving $w_1 = 1$. Then $\|\mathbf{w}\| = 1$ and the margin is $2/\|\mathbf{w}\| = 2$. The boundary $\mathbf{w} \cdot \mathbf{x} + b = x_1 - 1 = 0$ is indeed $x_1 = 1$.

Convex-hull view. Each "hull" is a single point: $\mathbf{c}^\star = (2, 0)$, $\mathbf{d}^\star = (0, 0)$. The normal is $\mathbf{w} \propto \mathbf{c}^\star - \mathbf{d}^\star = (2, 0)$, the bisector is $x_1 = 1$, and the margin is $\|\mathbf{c}^\star - \mathbf{d}^\star\| = 2$. The two pictures agree.

Worked example 9.10 (Four facing segments, margin 3)**.** Let $A = \{(3, 1), (3, -1)\}$ $(y = +1)$ and $B = \{(0, 1), (0, -1)\}$ $(y = -1)$. Each hull is now a vertical segment: $\mathrm{conv}(A)$ is the segment from $(3, -1)$ to $(3, 1)$, and $\mathrm{conv}(B)$ the segment from $(0, -1)$ to $(0, 1)$. The two segments face each other; the closest distance between them is the horizontal gap, $\|(3, y) - (0, y)\| = 3$, and it is achieved by *every* horizontally aligned pair (e.g. $(3, 0)$ and $(0, 0)$, or $(3, 1)$ and $(0, 1)$). So the margin is 3. Canonically, the boundary is $x_1 = 1.5$ with $\mathbf{w} = (w_1, 0)$; equality at $x_1 = 3$ gives $3w_1 + b = 1$ and at $x_1 = 0$ gives $-b = 1$, so $b = -1$ and $w_1 = 2/3$, hence margin $2/\|\mathbf{w}\| = 2/(2/3) = 3$, matching the hull distance. Note all four points are support vectors: the closest pair is non-unique even though the optimal plane is unique.

Problem 9.1 (Point versus segment)**.** Let $A = \{(2, 2)\}$ $(y = +1)$ and $B = \{(0, 0), (0, 2)\}$ $(y = -1)$. Sketch $\mathrm{conv}(B)$, the vertical segment from $(0, 0)$ to $(0, 2)$. Project the single positive point $(2, 2)$ onto that segment to find the closest hull point $\mathbf{d}^\star$. Deduce the SVM normal $\mathbf{w} \propto \mathbf{c}^\star - \mathbf{d}^\star$ and the margin $\|\mathbf{c}^\star - \mathbf{d}^\star\|$, then solve for the canonical $(\mathbf{w}, b)$ and identify which points are support vectors. (Hint: the foot of the perpendicular from $(2, 2)$ to the segment $x_1 = 0$, $0 \le x_2 \le 2$ is $\mathbf{d}^\star = (0, 2)$, so $\mathbf{w} \propto (2, 0)$, margin 2; $(2, 2)$ and $(0, 2)$ are support vectors, $(0, 0)$ is not.)

10 What gradient descent actually learns: implicit bias

We have a max-margin classifier as an *optimization problem* (10). But that is not how neural networks (or even logistic regression) are trained in practice. In practice we pick a loss function, write down the average loss over the training set, and run gradient descent, with *no margin or norm term anywhere in the objective.* So here is a genuinely surprising fact, and the bridge from Section 9 to the rest of the primer: even though nothing in the objective mentions the margin, gradient descent on the logistic loss drifts toward the max-margin solution all by itself. This silent preference is called *implicit bias* (Soudry et al., 2018).

10.1 The setup and the meaning of "convergence in direction"

Fold the labels into the inputs by replacing $\mathbf{x}_n$ with $y_n\mathbf{x}_n$, so we may assume every label is $+1$ and write the linear predictor as $\mathbf{w}^\top\mathbf{x}_n$. The data are *linearly separable* if there is some $\mathbf{w}_*$ with $\mathbf{w}_*^\top\mathbf{x}_n > 0$ for all n – a weight vector that gets every point right (Soudry et al., 2018). We minimize the empirical loss

$$\mathcal{L}(\mathbf{w}) = \sum_{n=1}^{N} \ell\big(\mathbf{w}^\top\mathbf{x}_n\big),$$

where ℓ is, for instance, the logistic loss $\ell(u) = \log(1+e^{-u})$ or the exponential loss $\ell(u) = e^{-u}$. Both are positive, smooth, strictly decreasing, and flatten toward 0 as $u \to \infty$. Gradient descent takes steps

$$\mathbf{w}(t+1) = \mathbf{w}(t) - \eta\,\nabla\mathcal{L}(\mathbf{w}(t)), \qquad \nabla\mathcal{L}(\mathbf{w}) = \sum_{n=1}^{N} \ell'\big(\mathbf{w}^\top\mathbf{x}_n\big)\mathbf{x}_n,$$

with a fixed learning rate $\eta > 0$ (a small step-size multiplier).

Here is the catch that makes the question subtle. On separable data, to push the loss to its infimum 0 the predictor must send every $\mathbf{w}^\top\mathbf{x}_n \to +\infty$, which forces $\|\mathbf{w}(t)\| \to \infty$. There is no finite minimizer to converge to. But for *classification* only the *direction* of $\mathbf{w}$ matters, since the predicted label is $\operatorname{sign}(\mathbf{w}^\top\mathbf{x})$ and rescaling $\mathbf{w}$ does not change a sign. So the right question is about the direction.

Definition 10.1 (Convergence in direction)**.** We say $\mathbf{w}(t)$ *converges in direction* to a unit vector $\mathbf{u}$ if $\|\mathbf{w}(t)\| \to \infty$ while the normalized iterate settles, $\mathbf{w}(t)/\|\mathbf{w}(t)\| \to \mathbf{u}$.

10.2 The linear separable case, proved

The centerpiece of this section is the theorem of Soudry et al. (2018): the direction that gradient descent locks onto is the L_2 max-margin (hard-margin SVM) direction. We state it, then give the two-part argument it rests on – a convergence lemma and the "support-vector gradient-domination" idea that explains *why* the SVM direction emerges.

Theorem 10.2 (Implicit bias of gradient descent, linear case (Soudry et al., 2018))**.** *Let the data be linearly separable and ℓ be a smooth, decreasing loss with an exponential tail (the logistic and exponential losses qualify). For any small enough learning rate η and any starting point,*

$$\mathbf{w}(t) \;=\; \widehat{\mathbf{w}}\,\log t \;+\; \boldsymbol{\rho}(t), \qquad \|\boldsymbol{\rho}(t)\| = O(\log\log t),$$

where $\widehat{\mathbf{w}}$ is the L_2 max-margin vector, the solution of the hard-margin SVM $\widehat{\mathbf{w}} = \arg\min \|\mathbf{w}\|^2$ s.t. $\mathbf{w}^\top\mathbf{x}_n \ge 1$. Consequently $\mathbf{w}(t)/\|\mathbf{w}(t)\| \to \widehat{\mathbf{w}}/\|\widehat{\mathbf{w}}\|$.

We first record that gradient descent does reach zero loss, with the norm diverging. This is the "no finite critical point" lemma of Soudry et al. (2018).

Lemma 10.3 (The loss goes to zero, the norm diverges (Soudry et al., 2018))**.** *Under the assumptions of Theorem 10.2, $\mathcal{L}(\mathbf{w}(t)) \to 0$, $\|\mathbf{w}(t)\| \to \infty$, and $\mathbf{w}(t)^\top\mathbf{x}_n \to +\infty$ for every n.*

Proof. Take any separator $\mathbf{w}_*$, so $\mathbf{w}_*^\top\mathbf{x}_n > 0$ for all n. Dot the gradient with $\mathbf{w}_*$:

$$\mathbf{w}_*^\top\nabla\mathcal{L}(\mathbf{w}) = \sum_{n=1}^{N} \ell'\big(\mathbf{w}^\top\mathbf{x}_n\big)\,\big(\mathbf{w}_*^\top\mathbf{x}_n\big).$$

Every term is a product of $\ell'(\cdot) < 0$ (the loss is strictly decreasing) and $\mathbf{w}_*^\top \mathbf{x}_n > 0$, so every term is *strictly negative*. Hence at any finite $\mathbf{w}$ the sum is strictly negative and cannot be 0, which means *there is no finite critical point*: $\nabla\mathcal{L}(\mathbf{w}) \neq \mathbf{0}$ everywhere. Gradient descent on a smooth loss with a small enough step size is guaranteed to drive $\nabla\mathcal{L}(\mathbf{w}(t)) \to \mathbf{0}$; since that can only happen "at infinity," the iterate norm diverges, $\|\mathbf{w}(t)\| \to \infty$. Because $\ell'(u) \to 0$ only as $u \to +\infty$, the gradient can vanish only if every margin $\mathbf{w}(t)^\top \mathbf{x}_n \to +\infty$; this forces $\ell(\mathbf{w}(t)^\top \mathbf{x}_n) \to 0$ for each n, so $\mathcal{L}(\mathbf{w}(t)) \to 0$. □

Now the heart of the matter: why is the limiting *direction* the SVM direction? The argument is what Soudry et al. (2018) call gradient domination by the support vectors.

Why the direction is the max-margin direction (sketch of Theorem 10.2). Use the exponential loss, $\ell(u) = e^{-u}$, for clarity (the logistic loss behaves the same way to leading order). Suppose, as Lemma 10.3 permits, that $\mathbf{w}(t)/\|\mathbf{w}(t)\|$ approaches some unit direction, and write the iterate as $\mathbf{w}(t) = g(t)\,\mathbf{w}_\infty + \boldsymbol{\rho}(t)$ with $g(t) \to \infty$ along the limit direction $\mathbf{w}_\infty$ and $\boldsymbol{\rho}(t)/g(t) \to 0$. The negative gradient is

$$
\begin{aligned}
-\nabla\mathcal{L}(\mathbf{w}(t)) &= \sum_{n=1}^{N} \exp\big(-\mathbf{w}(t)^\top \mathbf{x}_n\big)\,\mathbf{x}_n \\
&= \sum_{n=1}^{N} \exp\big(-g(t)\,\mathbf{w}_\infty^\top \mathbf{x}_n\big)\,\exp\big(-\boldsymbol{\rho}(t)^\top \mathbf{x}_n\big)\,\mathbf{x}_n.
\end{aligned}
$$

As $g(t) \to \infty$ the factor $\exp(-g(t)\,\mathbf{w}_\infty^\top \mathbf{x}_n)$ decays geometrically, and it decays *most slowly* for the points with the *smallest* value of $\mathbf{w}_\infty^\top \mathbf{x}_n$ – precisely the points closest to the boundary, the support vectors $\arg\min_n \mathbf{w}_\infty^\top \mathbf{x}_n$. All other terms are exponentially smaller and become negligible. So in the limit the update direction is dominated by, and is a *nonnegative combination of*, the support vectors:

$$
\mathbf{w}_\infty \;\propto\; \widehat{\mathbf{w}} \;=\; \sum_n \alpha_n \mathbf{x}_n, \qquad \alpha_n \geq 0, \quad \text{with } \alpha_n > 0 \text{ only for support vectors.}
$$

These are exactly the KKT conditions (Definition 9.6) of the hard-margin SVM (10): a nonnegative support-vector combination, with $\widehat{\mathbf{w}}^\top \mathbf{x}_n = 1$ on the support vectors and > 1 off them. Hence $\mathbf{w}_\infty$ is proportional to the SVM solution $\widehat{\mathbf{w}}$. The detailed proof of Soudry et al. (2018) additionally shows $g(t) = \log t$ and bounds the residual $\boldsymbol{\rho}(t)$, giving the stated rate. □

Three things to take away, all of which appear later. (1) The convergence to the SVM direction is *very slow*: the direction error decays like $O(1/\log t)$, whereas the loss itself decays like $O(1/t)$. So the loss can be tiny while the margin is still quietly improving – a reason to keep training past zero training error, and a warning that loss-based early stopping can cut off margin (hence robustness) gains (Soudry et al., 2018). (2) The bias is *specific to gradient descent and to exponential-tail losses*: adaptive methods such as Adam, or polynomial-tail losses, need not converge to the L_2 max-margin direction (Soudry et al., 2018). (3) The same geometric object – the support vectors, the convex-hull closest points of Section 9 – is what gradient descent homes in on.

10.3 Homogeneous networks: the normalized margin and KKT points

Linear predictors are a warm-up. Do deep networks, trained the same way, also drift toward a margin-maximizing solution? Lyu and Li (2020) answer this for an important class of networks, and we must define two terms carefully before stating their result.

Definition 10.4 (L-homogeneous network)**.** A network $\Phi(\boldsymbol{\theta}; \mathbf{x})$ with parameters $\boldsymbol{\theta}$ is *(positively) L-homogeneous* if scaling all the parameters by a positive constant c scales the output by c^L:

$$\Phi(c\,\boldsymbol{\theta}; \mathbf{x}) = c^L\,\Phi(\boldsymbol{\theta}; \mathbf{x}) \qquad \text{for all } c > 0.$$

Fully-connected and convolutional networks with ReLU (or LeakyReLU) activations and *no bias terms* are homogeneous, with L equal to the depth (the number of layers) (Lyu and Li, 2020). The reason: ReLU satisfies $\mathrm{ReLU}(c\,z) = c\,\mathrm{ReLU}(z)$ for $c > 0$, and each linear layer multiplies the output by its weight, so scaling the weights of all L layers multiplies the final output by c^L.

Just as in the linear case, only the direction of $\boldsymbol{\theta}$ matters for the predicted class, and by homogeneity the raw margin scales with $\|\boldsymbol{\theta}\|^L$. So to compare directions fairly we normalize.

Definition 10.5 (Normalized margin (Lyu and Li, 2020))**.** Let $q_n(\boldsymbol{\theta}) = y_n \Phi(\boldsymbol{\theta}; \mathbf{x}_n)$ be the margin on point n, and $q_{\min}(\boldsymbol{\theta}) = \min_n q_n(\boldsymbol{\theta})$ the smallest margin over the data. The *normalized margin* divides out the scale,

$$\bar{\gamma}(\boldsymbol{\theta}) = \frac{q_{\min}(\boldsymbol{\theta})}{\|\boldsymbol{\theta}\|^L}.$$

It is invariant to rescaling $\boldsymbol{\theta}$, just as the geometric margin was for the linear model.

Theorem 10.6 (Gradient descent maximizes the normalized margin (Lyu and Li, 2020))**.** *Consider an L-homogeneous network with an exponential-tail loss (logistic, exponential, cross-entropy), trained by gradient descent / gradient flow, and suppose at some time the network reaches* 100% *training accuracy. Then:*

(a) a smoothed version of the normalized margin $\bar{\gamma}$ is monotonically non-decreasing *thereafter; and*

(b) every limit point of the normalized direction $\boldsymbol{\theta}(t)/\|\boldsymbol{\theta}(t)\|$ lies along a KKT point *of the margin-maximization program*

$$\min \ \tfrac{1}{2}\|\boldsymbol{\theta}\|^2 \quad s.t. \quad q_n(\boldsymbol{\theta}) \geq 1 \ \ \forall n.$$

We state Theorem 10.6 *without proof*: the argument uses the Clarke subdifferential (a generalized gradient for the non-smooth ReLU network), an Euler-type radial/tangential decomposition of the parameter flow, and a monotonicity argument for the smoothed margin – machinery well beyond a first course, and not needed to use the result (Lyu and Li, 2020). Ji and Telgarsky (2019) prove a closely related statement and, for general (possibly non-separable) data, show gradient descent follows a unique ray whose direction is the max-margin separator of the separable part of the data.

An honest caveat: which KKT point? The crucial difference from the linear case is the word "KKT." For the convex SVM, a KKT point *is* the global max-margin solution (Definition 9.6). For a non-convex deep network the margin-maximization program is non-convex, so a KKT point is only a *first-order stationary point*; it need not be the global maximum-margin solution, and Lyu and Li (2020) note one can construct examples where it is not. Theorem 10.6 therefore tells us gradient descent maximizes margin *locally / in a stationarity sense*, but it does *not* pin down which margin-stationary solution among possibly many. This honesty matters for the project: "gradient descent maximizes the margin" is exactly true for linear models and only true "up to KKT" for deep nets.

The margin/robustness bridge. Why care about margin at all? Because the (normalized) margin *lower-bounds the distance to the decision boundary*, and that distance is the robust radius. Concretely Lyu and Li (2020) state that for a ReLU network the normalized margin on a point,

divided by the network's Lipschitz constant, is a lower bound on the L_2 distance from that point to the boundary (the same Lipschitz–margin certificate that the robustness section of this primer proves via Tsuzuku et al. (2018)). So: plain gradient descent grows the margin, a larger margin means a larger certified robust radius, and that is the thread connecting "what training does" to "how robust the model is."

Worked example 10.7 (2D logistic gradient descent finds the SVM direction)**.** Take two positive points $\mathbf{x}_1 = (1, 1)$ and $\mathbf{x}_2 = (1, -1)$ (both label $+1$, so no label folding needed) and the exponential loss. The hard-margin SVM solution is $\widehat{\mathbf{w}} = (1, 0)$: it is the shortest $\mathbf{w}$ with $\mathbf{w}^\top \mathbf{x}_1 \geq 1$ and $\mathbf{w}^\top \mathbf{x}_2 \geq 1$, and by symmetry it points along the x_1-axis. Now run gradient flow from $\mathbf{w}(0) = (0, c)$. The negative gradient is

$$-\nabla \mathcal{L}(\mathbf{w}) = e^{-(w_1+w_2)}(1, 1) + e^{-(w_1-w_2)}(1, -1),$$

whose second component is $e^{-(w_1+w_2)} - e^{-(w_1-w_2)}$. When $w_2 > 0$ this is negative (the first exponent is larger, so its term is smaller), pushing w_2 down; when $w_2 < 0$ it is positive, pushing w_2 up. Either way $w_2 \to 0$, while the first component is always positive so $w_1 \to +\infty$. Hence $\mathbf{w}(t)/\|\mathbf{w}(t)\| \to (1, 0)$: gradient descent recovers the SVM direction, exactly as Theorem 10.2 promises.

Problem 10.1 (Non-support points do not move the limit)**.** Add a third positive point $\mathbf{x}_3 = (3, 0)$ to Example 10.7. Check that $\widehat{\mathbf{w}}^\top \mathbf{x}_3 = 3 > 1$, so $\mathbf{x}_3$ is *not* a support vector and its KKT multiplier is $\alpha_3 = 0$. Verify the SVM stationarity $\widehat{\mathbf{w}} = \alpha_1 \mathbf{x}_1 + \alpha_2 \mathbf{x}_2$, and argue that although $\mathbf{x}_3$ tugs on the early gradient, the limiting direction is unchanged and would be the same if $\mathbf{x}_3$ were deleted. (This is gradient domination by support vectors in action.)

11 The neural tangent kernel and the lazy-versus-rich dichotomy

The implicit-bias results tell us a network's training drifts toward a margin-maximizing direction. But *what features* does it use to get there? Does it learn new features tuned to the task, or does it ride a fixed set of features baked in at the start? This is the lazy-versus-rich question, and the tool for making it precise is the *neural tangent kernel* (Jacot et al., 2018). We first build the kernel from a Taylor expansion, state Jacot's two theorems, give Chizat's criterion for when training is "lazy," and close with the one definition (feature averaging) that links all of this back to robustness.

11.1 Linearization and the neural tangent kernel

Write a network's scalar output as $f(\boldsymbol{\theta}, \mathbf{x})$, a function of parameters $\boldsymbol{\theta}$ and input $\mathbf{x}$. Training changes $\boldsymbol{\theta}$; suppose it stays near its starting value $\boldsymbol{\theta}_0$. A first-order Taylor expansion in the parameters (not the input) gives the *linearization*.

Definition 11.1 (Linearization in parameters)**.** The *linearized model* around $\boldsymbol{\theta}_0$ is the first-order Taylor approximation in $\boldsymbol{\theta}$,

$$f(\boldsymbol{\theta}, \mathbf{x}) \;\approx\; f(\boldsymbol{\theta}_0, \mathbf{x}) + \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_0, \mathbf{x}) \cdot (\boldsymbol{\theta} - \boldsymbol{\theta}_0).$$

This is *linear in the parameter shift* $\boldsymbol{\theta} - \boldsymbol{\theta}_0$, with a fixed *feature map* $\boldsymbol{\phi}(\mathbf{x}) = \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_0, \mathbf{x})$ (the gradient of the output with respect to the parameters, frozen at $\boldsymbol{\theta}_0$). It is still nonlinear in $\mathbf{x}$, because $\boldsymbol{\phi}(\mathbf{x})$ can be a complicated function of $\mathbf{x}$.

Definition 11.2 (Neural tangent kernel (Jacot et al., 2018))**.** The *neural tangent kernel* (NTK) is the inner product of two such feature maps,

$$K(\mathbf{x}, \mathbf{x}') = \left\langle \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}, \mathbf{x}),\, \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}, \mathbf{x}') \right\rangle = \sum_{p} \frac{\partial f}{\partial \theta_p}(\boldsymbol{\theta}, \mathbf{x})\, \frac{\partial f}{\partial \theta_p}(\boldsymbol{\theta}, \mathbf{x}'),$$

summing over all parameters θ_p. A *kernel* is just a symmetric function $K(\mathbf{x}, \mathbf{x}')$ that measures similarity between inputs by an inner product of feature vectors; it lets us do linear algebra in a feature space without writing the features out. The NTK is the kernel whose features are the parameter-gradients of the network.

Under gradient flow the network's *outputs* on the training set move according to this kernel. If $u_i(t) = f(\boldsymbol{\theta}(t), \mathbf{x}_i)$ collects the outputs and $\mathbf{y}$ the targets, then (Jacot et al., 2018)

$$\frac{d\,\mathbf{u}(t)}{dt} = -\,H(t)\,\big(\mathbf{u}(t) - \mathbf{y}\big), \qquad H(t)_{ij} = K(\mathbf{x}_i, \mathbf{x}_j; \boldsymbol{\theta}(t)),$$

i.e. *kernel gradient descent in function space* driven by the NTK Gram matrix H. Two facts make this useful: the cost is *convex in function space* (even though it is wildly non-convex in $\boldsymbol{\theta}$), and the dynamics relax fastest along the top eigenvectors (principal components) of H – which is a clean theoretical motivation for early stopping (Jacot et al., 2018).

11.2 Width, initialization scale, and Jacot's theorems

Two quantities decide whether the kernel picture is exact: *width* and *initialization scale*.

Definition 11.3 (Width and initialization scale)**.** The *width* of a hidden layer is its number of neurons; "infinite width" means letting every hidden layer's neuron count grow without bound. The *initialization scale* is the overall size of the random parameters at the start, controlled by a factor $\alpha > 0$ multiplying the output (or equivalently the variance of the initial weights). Both are "knobs" that, turned up, make the network behave more like its fixed linearization.

Jacot's two theorems say that in the infinite-width limit the NTK becomes deterministic and frozen (Jacot et al., 2018).

Theorem 11.4 (NTK at initialization (Jacot et al., 2018), stated)**.** *As all hidden widths $\to \infty$, the NTK at initialization converges (in probability) to a deterministic limiting kernel Θ_∞ that depends only on the choice of nonlinearity, the depth, and the initialization variance – not on the particular random draw of weights.*

Theorem 11.5 (NTK stays constant during training (Jacot et al., 2018), stated)**.** *Under mild smoothness, in the same infinite-width limit the NTK stays equal to Θ_∞* throughout *training. Hence training is exactly kernel gradient descent with one fixed kernel; for a least-squares loss the network function follows a linear differential equation and the outputs relax as $\mathbf{u}(t) = \mathbf{y} + e^{-Ht}(\mathbf{u}(0) - \mathbf{y})$.*

We state Theorems 11.4–11.5 *without proof*: they rest on a central-limit-theorem argument over the many neurons and on controlling how the kernel drifts during training, both of which are genuinely involved (Jacot et al., 2018). The intuition, though, is simple and worth internalizing: *a wide network has so many neurons that each one only has to move a tiny amount to fit the data, and if the parameters move only a little, the first-order Taylor approximation (the linearization) stays accurate.* One honest subtlety from Jacot et al. (2018) (their Remark 4): each individual neuron's contribution to the kernel does shrink with width, but the *collective* contribution of the lower layers does not, which is why even "lazy" lower layers still participate.

11.3 Lazy versus rich, and Chizat's criterion

We can now name the two regimes. Both terms are now standard, traceable to Jacot et al. (2018) and Chizat et al. (2019).

Definition 11.6 (Lazy and rich training)**.** Training is *lazy* (also called the *kernel* or *fixed-features* regime) when the NTK barely changes, so the network behaves like its linearization around $\boldsymbol{\theta}_0$: it is doing kernel regression with features *fixed at initialization*, and the parameters hardly move (Chizat et al., 2019). Training is *rich* (the *feature-learning* regime) when the NTK *evolves* during training, so the features adapt to the task; this regime lives outside kernel theory (it is described by mean-field / optimal-transport analyses).

Chizat et al. (2019) pinned down *when* training is lazy with one dimensionless number. Introduce an explicit scale factor α via the rescaled objective $F_\alpha(\mathbf{w}) = \frac{1}{\alpha^2} R(\alpha\, h(\mathbf{w}))$. Their criterion (for the square loss R) is

$$\kappa \;=\; \frac{\|h(\mathbf{w}_0) - \mathbf{y}^\star\|\;\|D^2 h(\mathbf{w}_0)\|}{\|Dh(\mathbf{w}_0)\|^2} \;\ll\; 1 \quad\Longrightarrow\quad \text{training is lazy,} \tag{11}$$

where Dh and D^2h are the first and second derivatives of the model in the parameters. Small κ means the model's curvature (D^2h) is negligible relative to its slope (Dh) over the distance training has to travel, so the linearization holds. The key scaling facts (Chizat et al., 2019):

- *Initialization scale.* Under rescaling by α, $\kappa_{\alpha h} = \kappa/\alpha$. So *turning up the scale α forces laziness*: large $\alpha \Rightarrow$ small $\kappa \Rightarrow$ lazy. This is the scaling law $\kappa_\alpha = \kappa/\alpha$ in Chizat et al. (Chizat et al., 2019).
- *Width.* For a one-hidden-layer net the gradient norm grows like $\sqrt{m}$ (with width m) while the curvature stays $O(1)$, so $\kappa \sim O(1)/(\sqrt{m})^2 = O(1/m) \to 0$. Width is a second route to the same small-κ laziness.
- *Parameters barely move.* In the lazy regime $\sup_t \|\mathbf{w}_\alpha(t) - \mathbf{w}_0\| = O(1/\alpha)$, confirming "the parameters hardly change."

Remark 11.7 (Chizat's honest counterpoint). Chizat et al. (2019) stress that lazy training is *not* the secret of deep learning's success. Their experiments show lazy-trained deep CNNs *underperform* ordinary (rich) training. Laziness gives clean theory and guaranteed fast fitting, but at the cost of recovering a fixed-kernel method; the practical wins come from feature learning. We carry this caveat into the bridge: the lazy/rich distinction is a useful conceptual axis, not a claim that real networks are lazy.

11.4 Worked examples: NTK of simple models

Worked example 11.8 (NTK of a linear model is constant)**.** Let $f(\boldsymbol{\theta}, \mathbf{x}) = \boldsymbol{\theta} \cdot \mathbf{x}$ (plain linear model). Then $\nabla_{\boldsymbol{\theta}} f = \mathbf{x}$ for *every* $\boldsymbol{\theta}$, so

$$K(\mathbf{x}, \mathbf{x}') = \langle \mathbf{x}, \mathbf{x}' \rangle = \mathbf{x} \cdot \mathbf{x}',$$

which does not depend on $\boldsymbol{\theta}$ or on time at all. A linear model is "perfectly lazy": its NTK is the constant kernel $\mathbf{x} \cdot \mathbf{x}'$, and training it *is* kernel / linear regression. (In the scalar case $f = \theta x$ the kernel is just xx'.)

Worked example 11.9 (NTK of one hidden unit depends on initialization)**.** Let $f(\boldsymbol{\theta}, x) = b\,\sigma(ax)$ with parameters $\boldsymbol{\theta} = (a, b)$ and a smooth activation σ. Then $\nabla_{\boldsymbol{\theta}} f = \big(b\,\sigma'(ax)\,x,\;\; \sigma(ax)\big)$, so

$$K(x, x') = b^2\,\sigma'(ax)\,\sigma'(ax')\,xx' \;+\; \sigma(ax)\,\sigma(ax').$$

Unlike the linear case, this *depends on* (a, b) – so at random initialization the kernel is random, and at finite width it drifts as (a, b) move during training. What infinite width buys is exactly the freezing of this dependence into a deterministic Θ_∞ (Theorem 11.4).

Problem 11.1 (Rescaled-model laziness)**.** Consider a rescaled model $f_\alpha = \alpha\, g$ with the model arranged so that $g(\mathbf{w}_0, \mathbf{x}) = 0$ at initialization. Using the criterion (11), show $\kappa_{\alpha g} = \frac{1}{\alpha}\,\kappa_g$, and conclude that the linearization error vanishes as $\alpha \to \infty$, so the model trains lazily. (Extension: verify $\kappa \sim 1/m$ for a one-hidden-layer net using the $\sqrt{m}$-versus-$O(1)$ scaling of the gradient and curvature norms.)

11.5 Feature averaging and orbit-structured data

The last definition is the mechanism that ties lazy/rich to the project's real question: *why a max-margin solution can fail to be robust.*

Definition 11.10 (Feature averaging, informally; orbit-structured data)**.** Many natural inputs are built from *shifted (or otherwise transformed) copies* of a few underlying patterns. Calling the set of all shifts of a pattern its *orbit* (the group-action term made precise in the symmetry section of this primer), we say data are *orbit-structured* when each class is a union of such orbits: a cat is a cat wherever it sits in the image. *Feature averaging* is the phenomenon where a network, instead of detecting the discriminative pattern wherever it appears, learns a weight that effectively sums or averages the signal across all positions of the orbit. Such an averaged feature can separate the training data with a large margin, yet a small, carefully placed perturbation that disturbs the averaging can flip the prediction – so the large-margin solution is *not* robust (Li et al., 2024).

We keep Definition 11.10 at the level of motivation, because none of the five learning-theory sources read here (Soudry et al., 2018; Lyu and Li, 2020; Ji and Telgarsky, 2019; Jacot et al., 2018; Chizat et al., 2019) proves a direct theorem of the form "lazy/rich $\Leftrightarrow$ shift-invariant/robust." What they *do* give us is the scaffolding: a lazy network cannot learn a new invariance that is not already in its initialization kernel (its features are frozen), whereas a rich network can adapt its features and *learn* shift-invariance rather than inherit it. Whether the network's margin-maximizing solution is a robust, invariance-respecting one or a brittle, feature-averaging one therefore depends on which regime it trained in – the conceptual hinge the project investigates.

Closing bridge: how the Part-1 thread fits together

The one-paragraph story this whole part has been building. *Shift-invariance* is, at bottom, a *group-averaging* operation: averaging a signal over all the shifts in its orbit is the Reynolds projection onto the invariant subspace (the symmetry section of this primer makes this an honest orthogonal projection when the shifts act by orthogonal matrices). The classical *descriptors* of that invariant content are the *power spectrum* and *bispectrum* (the shift-invariants section): a shift only rotates the phase of each Fourier coefficient, leaving $|X|^2$ untouched, and the bispectrum keeps enough relative phase to tell apart signals the power spectrum confuses. *Robustness* of a classifier is governed by two numbers we met here and in the Lipschitz section: the *margin* (Section 9) and the *Lipschitz constant*, with the certified robust radius behaving like margin over Lipschitz constant. The reason margin is the right currency is the implicit-bias result (Section 10): *plain gradient descent, with no robustness term in the objective, already drifts toward the max-margin solution* – exactly the SVM solution / convex-hull closest points for linear models, and a KKT point of the margin program for homogeneous networks. Finally,

whether the network *learns* a shift-invariant, robust representation or merely *inherits* a fixed set of features (and possibly a brittle, feature-averaging margin) depends on the *lazy-versus-rich* regime (Section 11): a lazy, wide-or-large-scale network is stuck with its initialization kernel and cannot learn a new invariance, whereas a rich network can. That is the chain – *group-averaging* → *spectral descriptors* → *margin and Lipschitz* → *implicit bias of gradient descent* → *lazy vs. rich* – on which the rest of the project is built.

Part II: Finding the questions

Part I gave you the tools. This part gives you the situation that the project started from and asks you to generate the research questions yourself. Each problem is presented with candidate directions and their trade-offs, and no answer: commit to a direction before you read Part III.

Part I handed you a toolbox: a way to measure how robust a classifier is, the Lipschitz–margin inequality (a margin divided by a Lipschitz constant lower-bounds the certified radius) that turns a smoothness budget and a margin into a certified radius, the Fourier/projector machinery that says what a symmetry preserves, and the implicit-bias and NTK lenses that say where gradient descent actually lands. This part does something different. It does *not* give you results. It puts you in the chair of the person who had to decide *which questions to ask in the first place*, hands you the same situation they faced, and asks you to form your own hypotheses before Part III tells you what happened.

That is a deliberate choice. The fastest way to misread a research paper is to learn its answers and then reverse-engineer questions that make those answers look inevitable. The answers almost never were inevitable. They were one defensible bet among several, taken at a fork where reasonable people would have gone different ways. If you can stand at that fork yourself and feel the pull of each branch, you will read Part III as a sequence of decisions rather than a sequence of revelations, and you will be able to do the same thing on a problem nobody has solved yet.

So: read this part with a pencil. Every research question below is presented as a genuine open problem with two to four candidate directions and their trade-offs. At each one you will be asked to commit, on paper, to the direction *you* would take and why, before turning to Part III. The goal is not to guess the "right" answer. It is to develop the instinct that lets you generate good questions and triage them.

12 The situation: a clean story that broke

Research questions do not come from the sky. They come from a concrete irritant: something a previous effort could measure but not explain. Here is the irritant, told as it actually unfolded, because the shape of the chain is itself a lesson in where questions live.

12.1 Act I — a tidy hypothesis, reproduced

The starting point was a clean and quotable claim from the literature: *shift invariance can reduce adversarial robustness* (Ge et al., 2021). The mechanism, for the linear case, is exactly the kind of thing Part I prepared you to read. A classifier that is invariant to all circular shifts of its input must give the same answer to a signal and to every shift of it; for a *linear* classifier this forces the weight vector into the fixed subspace of the shift group, which (Part I, the Fourier section) is spanned by the constant vector $\mathbf{1}$. The model can then see the data only through its mean, the DC (zero-frequency)

component, and discards all the rest. On a synthetic "one black pixel versus one white pixel" problem this collapses the achievable max-margin from a constant down to $2/\sqrt{d}$ in the image dimension d (Ge et al., 2021). More invariance, smaller margin, less robustness. A satisfying monotone story.

A team reproduced it. On small image families (MNIST, Fashion-MNIST) trained from a shared template, they measured a *shift-consistency* score — the fraction of inputs whose predicted label is unchanged when the input is shifted, an empirical stand-in for "how invariant did this model turn out to be" — and they measured adversarial robustness with a projected-gradient (PGD) attack. Across the families the headline ordering held: *higher consistency went with lower robustness*, just as predicted.

12.2 Act II — the deviations it could not name

Then the tidy story sprang leaks, and the leaks are the interesting part.

First, the ordering was only *mostly* right. Specific architectures sat in the wrong place: a couple of convolutional variants were more robust than their consistency score said they should be, or less. The reproduction could record the deviation but had no second axis to explain it. The hypothesis "consistency orders robustness" was being falsified at the margins by its own data, and nobody could say what the better axis was.

Second, to turn the picture into something actionable — "give me a model that is *both* shift-invariant and robust" — the team drew a two-dimensional scatter (consistency on one axis, robust accuracy on the other), split it into four quadrants using a fully-connected baseline as the origin, and aimed for the desirable corner. It was a useful screening picture. But it was a picture, not a principle: nothing in it said *why* a given architecture lands in a given quadrant, or whether the quadrants would survive a stronger attack, or whether they would persist as models grow. The diagnostic worked and was unjustified at the same time, which is the most uncomfortable place a result can sit.

Third, and most honestly, the write-up's own appendix for theory read, in effect, "to do." Everything was empirical, on tiny images, with a single attack family. There was no certificate, no worst-case attack, no measurement of the very margin whose collapse the base theory was about.

The lesson of Act II. Notice where the questions are being born. They are not born in the parts that worked. They are born at the *deviations* (the models in the wrong quadrant), at the *unjustified tool* (the quadrant diagram), and at the *explicit hole* (the empty theory appendix). A reproduction that merely confirms a result generates no questions. A reproduction whose seams show generates all of them. Train yourself to read for the seams.

12.3 Act III — the literature contradicts itself

If the base story were the whole story, the fix would just be "measure more carefully." It is not, because the supervising literature does not agree with itself. Four papers, read precisely, point in different directions while using the same word, "invariance."

- **Invariance hurts — conditionally.** Ge et al. (2021) is the base result above. But read past the headline and Ge themselves add a crucial nuance: on a dataset whose classes live in orthogonal frequencies, where shifting does *not* inflate the effective dimension, the convolutional network is *more* robust than the fully-connected one. Their own conclusion is that invariance hurts only when it shrinks the margin, and that "the linear margin predicts robustness better than the dimension of the data" (Ge et al., 2021). The base paper's own fine print already suspects the headline axis is wrong.

- **Invariance trades off — provably.** Kamath et al. (2020) show empirically that training for invariance to larger rotations shrinks the average distance to the decision boundary and hurts ℓ_∞ robustness; Kamath et al. (2021) prove a trade-off on a synthetic distribution built from a cyclic error-correcting code, and offer a curriculum that buys a better Pareto point without beating the single-objective adversarial baseline. Their "rate of invariance" $\Pr(f(TX) = f(X))$ is structurally the same quantity as the consistency score from Act I — yet neither Kamath paper cites Ge. Two independent lines, one seam between them.

- **A specific architecture choice causes vulnerability.** Galloway et al. (2019) argue that batch normalization, by fixing each neuron's first two moments, destroys input-space distances and drops PGD robustness by amounts (roughly 8–20% across datasets) comparable to the architecture effects Act I reported. This is not a contradiction so much as a warning: any cross-architecture robustness comparison is confoundable as a normalization effect unless normalization is controlled.

- **Invariance helps.** The anti-aliasing line says the opposite of Ge. Inserting a low-pass blur before downsampling restores shift-equivariance, and Zhang (2019) reports it *improves* shift-consistency and clean accuracy and average-case corruption robustness at once. Crucially, Zhang makes no worst-case ℓ_p adversarial claim; that is left as future work. The natural "invariance helps" counter-voice is about *average-case* robustness, not worst-case.

State the contradiction plainly. Same word, opposite verdicts: one camp says (conditional) invariance hurts worst-case ℓ_p robustness; another reports it helps; a third proves a trade-off on a cyclic-shift construction; and two of the three do not cite the base result. A contradiction in the literature is not a nuisance to be smoothed over in related work. It is a flashing sign that some shared assumption is wrong, and the assumption is almost always a word doing too much work. Here the overloaded word is "invariance."

12.4 Act IV — the gap the wider field leaves open

Widen the search and the contradiction sharpens into a precise hole. There is a theory camp that shows invariance or implicit bias can hurt robustness — but never about shift. Tramèr et al. (2020) prove that defending against ℓ_p sensitivity forces *excessive invariance*, opening a distinct invariance-based attack that changes the true label while the prediction stays put; but their "invariance" is over-invariance to an ℓ_p ball, not a group symmetry. Frei et al. (2023) prove that gradient descent's max-margin implicit bias on two-layer ReLU networks reaches solutions that generalize yet are *provably non-robust*, even when robust solutions fitting the same data exist; Li et al. (2024) pin the mechanism on feature averaging and recover robustness with finer supervision; Min and Vidal (2024) flip the bias from non-robust to robust by swapping the activation. All three are about the optimizer's bias on synthetic, near- orthogonal clusters *with no group symmetry*, under ℓ_2. Their lever on robustness is activation or supervision, not translation.

And there is an empirical/linear camp that says symmetry helps — but not in a way that closes the gap. Saha and Gokhale (2024) state verbatim that "better shift invariance is generally correlated with better adversarial robustness," and show the effect is architectural rather than a capacity artifact — but it is a pure correlation, with PGD/FGSM only (so possibly gradient-masked, see Part I), one backbone, no causal isolation, and no theorem linking shift to any radius. Wang et al. (2025) report that rotation/scale equivariance helps without adversarial training, but that is the *wrong group* (CNNs are already translation-equivariant), it is enforced rather than emergent, and again no ℓ_p theorem. Chen and Zhu (2023) and Lawrence et al. (2021) prove that linear equivariant

networks reach a wider margin or a characterized implicit bias — but they are *linear only* and stop at margin/generalization, never an ℓ_p radius.

The gap, in one sentence. Across every paper on both sides, the same thing is missing: *no result says what discriminative signal survives a given translation-invariance, turns that into an ℓ_p robustness certificate, and validates it under a standardized strong attack at matched capacity.* The "helps" side is empirical, or linear, or the wrong group. The "hurts" side is about ℓ_p-ball over-invariance or optimizer bias on symmetry-free clusters. The seam between them is exactly where the Act I reproduction's monotone story tore.

This is the situation. A reproduced result whose deviations have no second axis; a working diagnostic with no principle; a four-paper contradiction; and a field-wide gap with a precise shape. Everything that follows in this part is a question you could have asked while standing here. Now you get to ask them.

13 Five open problems, and the forks inside them

For each question I give you the *situation* (what you know and what is unsettled), the *candidate directions* you could take with their honest trade-offs, and a prompt asking you to commit before reading on. I will not name the quantity, the theorem, or the experiment that resolves it. Resist the urge to flip ahead. The value is entirely in deciding under uncertainty.

A note on the format. "Direction" means a research strategy, not a guaranteed win. Each has a cost. Part of taste is reading those costs accurately, because a direction that cannot be wrong is usually a direction that cannot teach you anything either.

13.1 Q1 — What single quantity actually orders robustness?

Situation. Act I gave you a metric, shift-consistency, that orders robustness *most* of the time and is falsified at the margins by its own outliers. Act III gave you the base paper quietly conceding that "the linear margin predicts robustness better than the dimension of the data." You suspect consistency is a symptom, not a cause. The deeper question is whether a *single measurable scalar* even exists that you can attach to a trained model and that ranks models by their robust radius — and if one does, which family of quantities it belongs to. You are genuinely choosing among candidate *families* here, not polishing a known winner: a *consistency* score, a bare *margin*, a *sensitivity* measure (a Lipschitz or gradient-norm proxy), or some *ratio* of a margin to a sensitivity. Each is a different bet about what governs the robust radius, and when your chosen scalar disagrees with consistency, you want it to be the one that is *right*.

Candidate directions.

- *Refine consistency.* Keep the consistency axis but measure it better — average over more shifts, weight by frequency, use a soft (probability) version instead of a hard label-flip count. **Trade-off:** cheap, continuous with the prior work, and it might clean up the outliers. But it doubles down on an axis the base paper's own nuance already suspects. If consistency is the wrong *kind* of quantity, no amount of polishing fixes the outliers; you will have built a better thermometer for the wrong temperature.

- *Bet on a single geometric quantity.* Drop consistency and pick *one* geometric scalar instead: a bare margin (distance to the boundary), a bare sensitivity (a Lipschitz or gradient-norm estimate),

or a *margin-to-sensitivity ratio* that trades the two against each other. Part I gives you reason to take the ratio family seriously — the Lipschitz–margin certificate lower-bounds the robust radius by a margin over a Lipschitz constant — but the certificate is background, not a verdict: it does not tell you that a ratio *orders* models better than a bare margin or a bare sensitivity does, nor in which feature space to compute any of them. **Trade-off:** principled, and each candidate ties to geometry you can compute. But a margin and a Lipschitz constant are both nontrivial to measure on a real network; the certificate inequality is one-directional (a lower bound, not an equality — you will revisit this in Q3); and you must decide *which* feature space (raw pixels? the space the invariance leaves behind?) to measure in. Picking the wrong family, or the wrong space, is the way this direction fails.

- *Let the data choose by regression.* Throw a basket of candidate scalars (consistency, margin, Lipschitz, gradient norms, boundary distance) at a panel of models and report which correlates best with measured robustness. **Trade-off:** maximally empirical and honest about what predicts what; it cannot be accused of theory-bias. But correlation on a model zoo confounds everything with everything (capacity, normalization, training), and a leaderboard of correlations is not an explanation — it tells you *that* something orders robustness, never *why*.

Problem 13.1 (commit before Part III)**.** First decide whether you even believe a single scalar can order robustness across architectures; if not, say what minimal set of scalars you would need instead. If you do, pick the family you would bet on — consistency, a margin, a sensitivity, or a margin-to-sensitivity ratio — and name the single most likely reason your bet fails. If you chose a ratio or any geometric scalar, write down which feature space you would measure it in and why raw pixels might be the wrong choice. If you chose regression, write down the one confound you would control first.

13.2 Q2 — What discriminative signal survives an imposed invariance?

Situation. The linear story is brutally clean: force shift-invariance on a linear model and it sees only the mean. But real networks are not linear classifiers on raw pixels; they push data through nonlinear feature maps first. If a convolution followed by a squaring nonlinearity and a pooling step is invariant to shifts, it is *not* reduced to the mean — it keeps something richer. You want to know *exactly what*. Not "it keeps more"; the precise functional. Because whatever survives the invariance is the only material left to build a margin out of, and Q1 told you margin is the thing that matters.

Candidate directions.

- *Quotient and diagonalize.* Treat the invariance as a projection: the model can only depend on the part of the input that the group leaves fixed, so write that projector down and diagonalize the group action (Fourier, for cyclic shifts) to read off the surviving coordinates in closed form. **Trade-off:** this is exactly the machinery Part I built, it gives an *exact* answer for the linear and quadratic cases, and it makes the margin computable. But it is cleanest for a single, idealized invariant layer; pushing it through a deep stack of real layers is where the rigor frays.

- *Climb the ladder of classical invariants.* There is a known hierarchy: the mean (degree 1), the power spectrum / autocorrelation (degree 2, what a square-and-pool computes), the bispectrum (degree 3), and beyond. Ask which rung a given architecture sits on and what it can therefore separate. **Trade-off:** it connects the architecture to a beautiful, well-understood body of signal-processing theory, and it predicts *failure* cases — classes that one rung cannot tell apart but

the next can. But higher rungs cost more (a larger Lipschitz constant; recall that any margin-to-sensitivity ratio has a sensitivity in its denominator), and you must show the architecture *actually* realizes the rung, not just that the rung exists.

- *Probe empirically.* Skip the analysis: train the invariant model and read out which input perturbations it is blind to, mapping the surviving signal by experiment. **Trade-off:** works for any architecture, no idealization. But it gives you a point cloud, not a formula; it will not tell you the *margin* the surviving signal supports, and it cannot certify anything.

A pointed sub-question. Recall the base paper's worst case: the "one hot pixel" example, $\pm e_j$, where the class is encoded by *where* the spike sits. Two such inputs that differ only by a shift have the *same* power spectrum — the degree-2 invariant is blind to location. So if your architecture stops at degree 2, it cannot separate them at all, and the forced trade-off bites hardest exactly there. Does a richer invariant rescue this corner, and at what cost to the Lipschitz denominator? Hold that thought; it is a live fork.

Problem 13.2 (commit before Part III)**.** For a convolution–square–pool layer, write your best guess at the surviving signal in one line. Then decide: would you try to *defeat* the hot-pixel worst case with a higher-order invariant, or would you accept it as a genuine limit of low-degree invariance? State the price of your choice in terms of the margin and the sensitivity (Lipschitz) it would move.

13.3 Q3 — Does the surviving signal certify an ℓ_p radius?

Situation. Suppose Q2 hands you an exact margin η for the surviving signal, and suppose you are pursuing the margin-to-sensitivity ratio of Q1's second direction, so the object on the table is the Lipschitz–margin certificate $r_2 \geq \eta/L$ from Part I. It is a *lower* bound: it guarantees no adversary inside radius η/L, but it says nothing about how much *further* the true safe radius might extend. For this to be the missing link the whole field lacks — a result tying translation-invariance to an actual ℓ_p radius — you have to decide what kind of statement you are even after. (Whether this ratio actually *orders* models better than its rivals is still Q1's open business; here you are only asking what the certificate, taken on its own terms, can and cannot promise.)

Candidate directions.

- *Settle for a one-sided certificate (and predictor).* Accept $r_2 \geq \eta/L$ as a guarantee and an empirical predictor, and do not claim it pins the radius exactly. **Trade-off:** honest and immediately usable; it is enough to *order* models and to *guarantee* safety up to the bound. But a lower bound alone cannot tell you whether a model with a small certificate is actually fragile or merely has a loose bound — and a referee will ask whether your quantity *characterizes* robustness or merely lower-bounds it.

- *Chase a converse.* Try to prove an upper bound too, so the ratio brackets the true radius from both sides. **Trade-off:** a two-sided bracket is far stronger and would make the quantity a genuine measure of robustness, not just a safety floor. But you should first ask whether a *universal* converse can even exist — look for a degenerate example where the ratio is tiny yet the true radius is large. If you can build one, an unconditional converse is dead, and the honest move is to find the extra *condition* under which a converse holds. (This is the search for what has no converse, a research move in its own right; see Section 14.)

- *Add a second, invariance-specific radius.* The certificate above is the *additive-noise* radius. But Tramer's excessive-invariance attack changes the true label *along the orbit* while the prediction stays fixed — a different failure the additive radius does not see. You could certify a separate "orbit-flip" radius for that mode. **Trade-off:** it captures a real and distinct threat and makes the "invariance can also *hurt*" direction precise. But it complicates the story — now there are two radii to report, and you must say when each one is the binding constraint.

Problem 13.3 (commit before Part III)**.** Decide whether you would chase a converse or settle for a one-sided certificate. If you would chase it, sketch the degenerate example you would build *first* to check whether an unconditional converse is even possible — a function whose certificate is loose by an unbounded factor. If you would not, say what claim you are giving up.

13.4 Q4 — Does any of this survive a strong attack at matched capacity?

Situation. Acts I–IV are littered with the same two methodological landmines. First, PGD/FGSM attacks can *appear* to show robustness that vanishes under a stronger evaluation because the gradients were masked, not the model genuinely robust (Part I). The "invariance helps" evidence is almost entirely PGD/FGSM. Second, the architectures being compared differ in *capacity* (and in normalization, per Galloway et al. (2019)), so any robustness gap might be a capacity or normalization gap wearing an invariance costume. You want to know whether any geometric story you settle on in Q1–Q3 is *real* or an artifact of weak attacks and unmatched models.

Candidate directions.

- *Trust a stronger attack ensemble.* Re-evaluate every model with a standardized, parameter-free, ensemble attack designed precisely to catch gradient masking, and treat that number as ground truth. **Trade-off:** this is the community standard and the only way to kill the gradient-masking objection. But it is expensive, and a single robust-accuracy number at one perturbation budget hides the *radius* you actually care about.

- *Measure the radius directly with a min-norm attack.* Instead of "what fraction survives at budget ϵ," find, per input, the *smallest* perturbation that flips it — the empirical robust radius. **Trade-off:** this is the right quantity to compare against an η/L *radius* certificate, and it gives a distribution, not one number. But min-norm attacks are themselves attacks and can be fooled; you should cross-check them against the ensemble above.

- *Control capacity by construction.* Build a family of architectures — a plain one, an anti-aliased one, an exactly-cyclic-invariant one, a shift-augmented one — with *identical* parameter counts, the normalization confound pinned down, and the *same* training. Then any robustness difference is attributable to the invariance, not to size. **Trade-off:** this is the strongest possible causal isolation and lets you ask not "does invariance help?" but "which term, margin or Lipschitz, does *this* invariance move?" But matching capacity across structurally different models is fiddly, and you must verify the matched models really are comparable in every nuisance dimension.

- *Stay on synthetic data where the truth is known.* Run everything on the constructions where the margin and radius are computable in closed form, sidestepping attack noise entirely. **Trade-off:** clean and unimpeachable, and it isolates mechanism. But a referee — and the gap from Act IV — will say the open question is precisely about *real* data under a *strong* attack; synthetic-only re-opens the very gap you set out to close.

Problem 13.4 (commit before Part III)**.** Design your headline experiment in three lines: which models you compare, what you hold fixed to make the comparison causal, and which attack(s) you trust for the final number. Name the one confound that, if a reviewer raised it, would most damage your conclusion — and say how your design already answers it.

13.5 Q5 — Why do trained models land where the diagnostic put them?

Situation. Q1–Q4 are about what a *given* model's geometry implies. But the Act II quadrant diagram was about *trained* models: gradient descent, not a fixed weight vector, decides where each architecture ends up on the consistency-versus-robustness plane. So even a perfect geometric theory leaves a gap: *why does optimization land an architecture in the quadrant it lands in?* Answer this and the heuristic quadrant diagram becomes a principle; leave it and you have explained the map but not the territory. Part I's implicit-bias and NTK lenses are the tools here.

Candidate directions.

- *Reduce to a regime where the bias is known.* In the lazy/NTK regime the trained network behaves like a fixed kernel method, where "what gradient descent selects" has a clean answer. Ask what enforcing invariance does to that kernel's solution. **Trade- off:** tractable, and it can turn an empirical trend into a theorem about the selected solution. But the lazy regime is an idealization (wide, near-linear training), the activation matters (a smooth activation behaves very differently from bare ReLU under this lens), and what holds lazily may not hold in the feature-learning regime real training lives in.

- *Split by data geometry.* The Act III/IV papers suggest the answer is conditional. On data whose classes are *closed under the symmetry* (every shift of a class member is in the class), enforcing invariance may cost nothing — gradient descent might symmetrize for free. On near-orthogonal *cluster* geometry (the Frei/Li/Min-Vidal setting) invariance may genuinely help the ratio. Prove the easy case, conjecture the hard one. **Trade-off:** this matches the contradiction's actual shape — different geometries, different verdicts — and gives you a provable positive result plus an honest open problem. But it splits the question into pieces, one of which you will have to leave open, and you must be candid that the cluster case resists a full trajectory-level proof.

- *Stay empirical and characterize the landing pattern.* Train the matched families from Q4 across seeds and data regimes and report *where* they land and how tightly, upgrading the quadrant diagram into a measured phenomenon without proving it. **Trade-off:** robust and honest, and it documents exactly the regime-dependence the theory should later explain. But "we observed where models land" is a description, not the principle the Act II diagram was missing; a reviewer asks *why*.

Problem 13.5 (commit before Part III)**.** Decide whether you would attack this in the lazy regime, split it by data geometry, or characterize it empirically — and say which part of the question you are willing to leave *open*. (Honest answer: at least one piece of Q5 is still open in the project. Predict which piece, and why it is the hard one.)

14 Research taste: how to find questions like these

The five questions above did not arrive labelled. Someone had to manufacture them from a tangle of contradictory papers and an unexplained scatter plot. This section is about that manufacturing. It

is not a checklist — taste is not a checklist — but there are recurring *moves* that generate good questions, and there are real *criteria* for deciding which questions are worth the months they cost. I illustrate each move with a situation from this project, without giving the answer, so you can practice the move rather than memorize the result.

14.1 Recurring moves

Quotient out the nuisance. When a symmetry or a degree of freedom is making the problem look complicated, do not fight it — divide by it. Ask what is left once the nuisance is collapsed. *In this project:* "the model is shift-invariant" looks like a statement about infinitely many shifted inputs. Quotient by the shift group and it becomes a statement about a single object — the projection of the data onto the part the group fixes. The infinitely-many-shifts problem becomes a one-projector problem. (This is the engine behind Q2.)

Diagonalize the symmetry. A symmetry that commutes with your operator can be diagonalized, and in the eigenbasis the operator is just multiplication — everything decouples. *In this project:* circular shifts are diagonalized by the Fourier basis, so an invariant's action on the data becomes coordinate-wise on frequencies, and "what survives invariance" becomes "which frequency coordinates the invariant can still read." The hard algebra of convolutions turns into reading off a diagonal. Whenever you see a group acting linearly, ask for its eigenbasis first.

Bracket what has no converse. A one-sided inequality is a confession that you do not yet know the other side. Before assuming both sides hold, deliberately hunt for a counterexample to the converse. If you find one, you have learned that the clean two-sided statement is *false in general* — which is itself a result — and the real question becomes "under what extra condition does a converse hold?" *In this project:* the certificate $r_2 \geq \eta/L$ is one-sided; the right reflex is to ask whether a universal converse can exist before trying to prove one. (This is the heart of Q3, and the degenerate-example reflex below.)

Find the degenerate case. The fastest way to test a claim or kill a converse is to push it to an extreme where one term blows up or collapses, and see if the claim survives. *In this project:* to probe whether η/L pins the radius, you reach for a function whose curvature can be cranked arbitrarily high so the certificate becomes arbitrarily loose while the true radius does not move. A single such example settles a general question instantly. Degenerate cases are cheap and decisive; build them *before* you invest in a hard proof.

Reduce to the per-unit atom. A network is a composition of many small pieces. Instead of reasoning about the whole, find the smallest piece that carries the phenomenon and solve *that* exactly, then ask how the pieces compose. *In this project:* rather than analyze a deep invariant network, isolate one convolution–nonlinearity–pool unit and compute exactly what it keeps and what margin it supports. The atom is solvable; the composition is the next question. If you cannot solve the atom, you certainly cannot solve the network — and if you can, you have a building block and a sanity check for everything larger.

Transport a finite lemma to function space. A clean fact about finite-dimensional vectors (projections are non-expansive; averaging cannot increase a norm) often has an exact analogue in a function space (a reproducing-kernel Hilbert space) that turns an empirical trend about trained

networks into a theorem. *In this project:* "averaging over the orbit cannot increase the norm" is a one-line fact for vectors; transported to the kernel that describes a wide network's training, it becomes a statement about the solution gradient descent selects. The move is: prove it where it is trivial, then carry it across the bridge that your regime (e.g. the lazy/NTK lens from Part I) provides. The catch is always whether the bridge holds — which is exactly why Q5's lazy direction carries an activation caveat.

14.2 Which questions are worth asking

Moves generate candidate questions faster than you can answer them. Triage is the other half of taste. Three criteria did the real work here.

1. There is a small, testable instance. A question you can probe on a two-frequency toy or a closed-form linear case *this afternoon* is worth a hundred you can only settle with a six-month training run. The small instance tells you fast whether the big claim is even plausible, and it doubles as a sanity check on the eventual general proof. Every one of Q1–Q5 has a synthetic miniature where the answer is computable; that is not a coincidence, it is the selection filter. If you cannot construct a small instance, you usually do not yet understand the question well enough to ask it.

2. It sits exactly at a literature contradiction. The most fertile questions live on the seam where two credible papers disagree, because the disagreement guarantees that *someone* is wrong about *something specific*, and resolving it cannot be a no-op. Q1 sits on the seam between "consistency orders robustness" and Ge's own "margin predicts better than dimension." Q5 sits on the seam between "invariance hurts" (Frei/Li) and "invariance helps" (Saha–Gokhale, Chen–Zhu). A question that no published paper would dispute is rarely worth asking; a question that forces two published papers to reconcile almost always is.

3. The answer changes a decision. Ask what someone would *do* differently once they know the answer. Q1's answer changes which scalar you compute to screen architectures. Q4's answer changes whether you trust a PGD number at all. Q3's answer changes whether you can advertise a *certificate* or only a *predictor*. If both possible answers lead to the same action, the question is decorative; drop it.

Be honest: some good-looking questions did not pan out. Taste includes being wrong gracefully. Several attractive questions in this project bought less than they promised. Polishing the consistency metric (Q1's first direction) was tempting and continuous with prior work, but if the axis is the wrong *kind* of quantity, a better thermometer does not help — a lesson you only learn by trying it or by reasoning hard about why not. Chasing an *unconditional* converse for the certificate (Q3) is a dead end the degenerate-case move kills in an afternoon; the salvageable question is the *conditional* one, which is strictly humbler. And the cleanest part of Q5 — the optimization story on symmetric data — yields a real theorem, while the part everyone wants — the cluster-geometry case in the realistic feature-learning regime — stays a conjecture supported by experiments, not a proof. A primer that pretended otherwise would be teaching you the wrong lesson. Knowing which of your questions is provable, which is merely true, and which is only plausible is not a footnote to the research. It *is* the research.

You now have the five questions, the moves that generate them, and the criteria that select them. Before you read Part III, go back to your five committed answers from the *commit before Part III*

prompts. Write, for each, the single observation that would most change your mind. Then, and only then, turn the page.

Part III: Results and proofs

Here are the answers to the Part II problems, each motivated, labelled honestly (classical, clean rephrasing, or new contribution), and proved in full. Compare them against the directions you committed to.

15 Results: what survives an invariance, and what it costs

Part II asked five questions. Each came from a real friction point: the old project's monotone "more consistency, less robustness" story broke and could not be explained; the base papers openly contradict each other on whether invariance helps or hurts; and no prior result ties a *translation* invariance to an ℓ_2 robust radius. This part answers those questions with theorems and full proofs. Two honesty rules run through the whole part, and we state them once, plainly, so no proof reads as a bigger claim than it is.

Remark 15.1 (Calibration: what is new here and what is borrowed). Most of the *machinery* below is classical. Group averaging, fixed subspaces, the Lipschitz–margin certificate, power-spectrum and bispectrum invariants, the implicit-bias characterization of gradient flow, and the lazy/NTK picture are all established tools, taught in Part I and cited there. What this part contributes is narrow and specific: (1) an *exact projection-margin accounting* of what discriminative signal a given invariance keeps (Theorem 15.5); (2) a *two-sided bracket* on the robust radius (a pair of bounds, lower and upper, that trap the true radius from both sides) that completes the missing upper arm of the known certificate chain (Theorem 15.8); (3) a *negative* optimization result, that on *orbit-closed* data (data where every shift of a training point is itself a training point of the same label) imposing invariance is margin-free (Theorem 15.15); (4) a *positive* optimization result in the *lazy* (wide-network, fixed-kernel) regime, that invariance lowers the Lipschitz constant at equal margin (Theorem 15.17). We never call η/L a "new robustness quantity": it is a standard certificate, and our angle is only to compute it *after* identifying the invariant feature space and then to track which of its two parts (the margin η or the Lipschitz term L) a given architecture moves.

Remark 15.2 (Two plain-language terms we will need). A *converse* to an inequality $A \le B$ is a bound in the other direction, $B \le C\,A$ for some constant C: it says A is not just a lower estimate of B but pins B down to within a factor. A certificate $r_2 \ge \eta/L$ becomes *useful as a predictor* only if it has a converse, otherwise the true radius could be arbitrarily larger than the certificate. A function g is *co-Lipschitz* (or lower-Lipschitz) with constant α along a path if it *cannot* change too slowly: $|g(z) - g(x)| \ge \alpha\,\|z - x\|_2$. Ordinary Lipschitz continuity ($|g(z) - g(x)| \le L\,\|z - x\|_2$) caps how fast g changes; the co-Lipschitz bound puts a floor under it. The certificate's missing converse is exactly a co-Lipschitz statement, as we will see.

Remark 15.3 (Notation: one margin, several names). The same conceptual quantity — the *margin*, "how decisively is a point classified" — wears different symbols depending on the setting, and we flag this once so it never confuses. In Part I's robustness section the *logit* margin (top logit minus best competitor) was $M_{F,x}$; in Part I's learning section the *geometric* (SVM) margin, a distance to the boundary, was γ. In this part we write η for the *feature margin* of an invariant score (the gap the surviving signal supports) and $\mu = y\,g(x)$ for the signed margin used inside the radius bracket;

for a max-margin separator these coincide, $\mu = \eta$. Likewise r_2 here is the ℓ_2 robust radius written $r(x)$ in Part I. Finally, Part III's Fourier proofs use the *unitary* DFT $\hat{x}_k = d^{-1/2} \sum_j x_j e^{-2\pi i jk/d}$ (so Parseval is an exact isometry), whereas Part I fixed the *unnormalised* forward DFT (so Parseval carried a factor of N); both use the same minus-sign root of unity $\omega = e^{-2\pi i/d}$.

15.1 Q2 answered: the invariant linear margin is a projected convex-hull distance

The question this answers. A shift-invariant linear classifier must give every circular shift of a signal the same score. So it cannot use any feature that a shift can change. *Which* discriminative signal is left? Part I showed (for cyclic shifts) that the fixed subspace is the one-dimensional line spanned by the all-ones vector, so an invariant linear score sees only the mean (DC) component. The clean general statement is not special to cyclic shifts or to the DC line: for *any* finite group of orthogonal symmetries, the surviving signal is the data *projected onto the fixed subspace*, and the best achievable invariant margin is exactly half the distance between the two projected convex hulls. This is the structural anchor for everything that follows.

Status. This is a clean generalization, not a new phenomenon. Ge et al. (2021) prove the cyclic/DC special case (an invariant linear classifier is governed by the DC gap), and group averaging onto a fixed subspace is standard representation theory. The modest contribution is the general finite-group statement with the convex-hull normalization of the margin; the cyclic case recovers Ge, including the $2/\sqrt{d}$ collapse, as a corollary. We do not claim the cyclic effect as new.

We use one fact from Part I about two finite point sets A, B in $\mathbb{R}^m$: the largest margin of a hyperplane that separates them, where the margin is the smallest distance from any point to the hyperplane, equals $\frac{1}{2}\operatorname{dist}(\operatorname{conv} A, \operatorname{conv} B)$, the half-distance between their convex hulls. This is the geometric (convex-hull) form of the hard-margin support vector machine (Bennett and Bredensteiner, 2000; Burges, 1998). We will reuse it verbatim, so we restate it as the lemma we lean on.

Lemma 15.4 (Hard-margin = half convex-hull distance)**.** *For finite sets $A, B \subset \mathbb{R}^m$ with $\operatorname{conv} A \cap \operatorname{conv} B = \varnothing$, the maximum over unit vectors n and biases b of the separation margin*

$$\min\Big(\min_{a\in A}(n^\top a + b),\ \min_{b'\in B}(-n^\top b' - b)\Big)$$

equals $\frac{1}{2}\operatorname{dist}(\operatorname{conv} A, \operatorname{conv} B)$, and is attained by the unit normal pointing along the segment joining the two closest hull points, with the hyperplane placed at its midpoint.

Theorem 15.5 (Invariant linear margin for a finite orthogonal group)**.** *Let a finite group G act on $\mathbb{R}^d$ by orthogonal matrices $\{R_g\}_{g\in G}$. Define the fixed subspace and the group-average (Reynolds) projector*

$$V^G := \{v : R_g v = v \text{ for all } g \in G\}, \qquad P_G := \frac{1}{|G|}\sum_{g\in G} R_g.$$

A linear score $h(x) = w^\top x + b$ is exactly invariant, $h(R_g x) = h(x)$ for all x and g, if and only if $w \in V^G$. For finite labelled sets $X_+, X_- \subset \mathbb{R}^d$, the largest ℓ_2 margin achievable by an exactly invariant linear classifier is

$$\gamma_{\text{inv}} = \tfrac{1}{2}\operatorname{dist}\big(\operatorname{conv}(P_G X_+),\ \operatorname{conv}(P_G X_-)\big).$$

Proof. Step 1 (which normals are invariant). Fix g. The identity $h(R_g x) = h(x)$ for *all* x means $w^\top R_g x + b = w^\top x + b$ for all x, i.e. $w^\top R_g x = w^\top x$ for all x. Subtracting, $(R_g^\top w - w)^\top x = 0$ for all

$x \in \mathbb{R}^d$. A vector orthogonal to all of $\mathbb{R}^d$ is zero, so $R_g^\top w = w$. *(orthogonality of the action)* Because R_g is orthogonal, $R_g^\top = R_g^{-1} = R_{g^{-1}}$, and as g ranges over the group so does g^{-1}; hence $R_g^\top w = w$ for all g is the same condition as $R_g w = w$ for all g, that is, $w \in V^G$. The converse is the same computation read backwards: if $w \in V^G$ then $R_g^\top w = w$ gives $w^\top R_g x = w^\top x$.

Step 2 (P_G is the orthogonal projector onto V^G). First P_G is idempotent: *(group closure)*

$$P_G^2 = \frac{1}{|G|^2} \sum_{g,h \in G} R_g R_h = \frac{1}{|G|^2} \sum_{g,h} R_{gh} = \frac{1}{|G|^2} \sum_h \Big(\sum_g R_{gh} \Big) = \frac{1}{|G|^2} \sum_h |G|\, P_G = P_G,$$

where for each fixed h the inner sum over g runs over all group elements gh, hence equals $|G|\, P_G$. Next P_G is symmetric: *(orthogonality again)* $P_G^\top = \frac{1}{|G|} \sum_g R_g^\top = \frac{1}{|G|} \sum_g R_{g^{-1}} = P_G$. A symmetric idempotent is an orthogonal projector. Its range is V^G: if $v \in V^G$ then $P_G v = \frac{1}{|G|} \sum_g R_g v = \frac{1}{|G|} \sum_g v = v$, so $V^G \subseteq \mathrm{range}(P_G)$; and for any x, $R_h(P_G x) = \frac{1}{|G|} \sum_g R_{hg} x = P_G x$ by the same reindexing, so $\mathrm{range}(P_G) \subseteq V^G$.

Step 3 (invariant classification = classification of the projection). Take any invariant $w \in V^G$. By Step 2, $w = P_G w$, and since P_G is symmetric,

$$w^\top x = (P_G w)^\top x = w^\top P_G x.$$

So evaluating the invariant score at x is the same as evaluating the *same* normal w at the projected point $P_G x$, with the same $\|w\|_2$. Therefore an invariant linear classifier on $X_+ \cup X_-$ is exactly an ordinary linear classifier on the projected sets $P_G X_+$ and $P_G X_-$, and the signed margins match point by point. Conversely any linear classifier in the projected space lifts (take its normal, which already lies in V^G) to an invariant one.

Step 4 (apply the hard-margin lemma). By Step 3, maximizing the invariant margin over original-space data is the same problem as maximizing the ordinary linear margin over the projected data $P_G X_+$, $P_G X_-$. Lemma 15.4 with $A = P_G X_+$ and $B = P_G X_-$ gives that this maximum equals $\frac{1}{2} \operatorname{dist}(\operatorname{conv}(P_G X_+), \operatorname{conv}(P_G X_-))$, attained by the closest-points normal and the midpoint bias. This is the claim. □

Remark 15.6 (Why the right object is a distance, not a one-sided gap). A natural but wrong guess writes the invariant margin as a labelled gap such as $\frac{1}{2}(\min_{X_+} f_{\mathrm{dc}} - \max_{X_-} f_{\mathrm{dc}})_+$, which is only correct when the positive class happens to sit "above" the negative one along the fixed direction. The margin is symmetric in the labels: it is the *distance* between the two projected convex hulls, which is orientation-free and is zero exactly when they overlap. This is the symmetric form proved above.

Recovering Ge's $2/\sqrt{d}$ collapse. For cyclic shifts on $\mathbb{R}^d$ the fixed subspace is the single line $\mathrm{span}\{\mathbf{1}\}$ and $P_G = \Pi_{\mathrm{inv}} = ee^\top$ with $e = d^{-1/2}\mathbf{1}$ (Part I). Then $P_G x = (e^\top x)e = f_{\mathrm{dc}}(x)\, e$, so the projected data lie on one line and are identified isometrically with the scalar DC values $f_{\mathrm{dc}}(x)$. The two projected hulls become two intervals I_+, I_- on that line, and Theorem 15.5 reduces to

$$\gamma_{\mathrm{inv}} = \tfrac{1}{2} \operatorname{dist}(I_+, I_-).$$

Take Ge's black/white dot example: $x_+ = e_j$ and $x_- = -e_j$ (a single bright pixel versus a single dark pixel). Their DC values are $f_{\mathrm{dc}}(\pm e_j) = \pm d^{-1/2}$, so the interval distance is $2/\sqrt{d}$ and the invariant margin is $\gamma_{\mathrm{inv}} = 1/\sqrt{d}$. Yet *without* the invariance constraint a linear classifier may use the coordinate x_j directly, separating $\pm e_j$ with margin 1. The invariant margin therefore decays like $1/\sqrt{d}$ in the image size d while the unconstrained margin stays constant: this is exactly the $2/\sqrt{d}$ collapse of Ge et al. (2021), recovered as the cyclic specialization of the projection identity. The general theorem says *why* it happens: the invariance projects away every coordinate except the mean, and a localized pattern almost entirely lives in the coordinates that get projected away.

15.2 Q3 answered: the η/L certificate, its missing converse, and the two-sided bracket

The question this answers. Part I gave the Lipschitz–margin certificate: if an invariant score g has feature margin η and Lipschitz constant L, then the ℓ_2 robust radius satisfies $r_2(g, x) \geq \eta/L$. This is the link from an invariance to an ℓ_2 radius that no prior paper supplied. But a one-sided certificate is a weak predictor unless we also know how much it can *under*estimate the true radius. So Q3 splits into three sub-questions. First, is the certificate ever loose by an unbounded amount (no universal converse)? Second, under what condition can we recover an upper bound, turning the certificate into a two-sided bracket that pins r_2 down? Third, when is the bracket exact?

Status. The lower bound is the textbook Lipschitz–margin certificate (Hein and Andriushchenko, 2017; Tsuzuku et al., 2018); we do not claim it. The genuine contribution here is the *upper arm*. Tsuzuku et al. (2018) state a certificate chain whose top inequality (certificate $\leq$ true radius) they say cannot be guaranteed, and measure the resulting gap empirically as 1.8–2.4×; prior upper bounds on the true radius were only empirical, read off by running a minimal-distortion attack (Croce and Hein, 2020). Theorem 15.8 supplies that missing arm under an explicit, checkable reachability condition (a co-Lipschitz bound along one path to the boundary). The linear case (Corollary 15.10) is the classical point-to-hyperplane distance (Hein and Andriushchenko, 2017; Cortes and Vapnik, 1995), included as a tightness sanity check.

We first show the certificate has *no* universal converse, so we know an upper arm must require an extra hypothesis.

Proposition 15.7 (No universal converse for η/L)**.** *There is no universal constant C such that every invariant classifier obeying the Lipschitz–margin certificate also obeys $r_2(g, x) \leq C\,\eta/L$. The failure occurs even when the invariant feature depends only on the DC coordinate.*

Proof. We exhibit a one-parameter family whose ratio $r_2/(\eta/L)$ grows without bound. *Step 1 (a DC-only invariant feature).* Write $t = e^\top x = f_{\mathrm{dc}}(x)$, the DC coordinate, which is shift-invariant. For an integer $n \geq 1$ set the invariant scalar feature and classifier

$$\Psi_n(x) = t^{2n+1}, \qquad g_n(x) = \Psi_n(x).$$

Take the two data points $x_+ = e$ (label $+1$) and $x_- = -e$ (label -1), where $e = d^{-1/2}\mathbf{1}$ so that $t(x_\pm) = \pm 1$.

Step 2 (the feature margin). Then $g_n(e) = 1^{2n+1} = 1$ and $g_n(-e) = (-1)^{2n+1} = -1$. In the one-dimensional feature space the two classes sit at $+1$ and -1, so the signed feature margin is $\eta = 1$.

Step 3 (the Lipschitz constant on the relevant segment). On the DC segment $\{te : t \in [-1, 1]\}$, the derivative of Ψ_n in t is $(2n+1)t^{2n}$, of magnitude at most $2n+1$, attained at $t = \pm 1$. *(maximum over t of $|(2n+1)t^{2n}|$ on $[-1, 1]$)* Hence the Lipschitz constant is $L_n = 2n + 1$, and the certificate gives $\eta/L_n = 1/(2n+1)$.

Step 4 (the true radius). The decision boundary is $g_n = 0$, i.e. $t = 0$. The closest point to $x_+ = e$ with $t = 0$ is the origin, at Euclidean distance $\|e - 0\|_2 = 1$; perturbing inside the anti-invariant subspace V_{ac} (where $\mathbf{1}^\top \delta = 0$) does not change t at all, so it cannot move the score. Therefore the true robust radius is $r_2(g_n, e) = 1$.

Step 5 (the ratio is unbounded). Combining Steps 3 and 4,

$$\frac{r_2(g_n, e)}{\eta/L_n} = \frac{1}{1/(2n+1)} = 2n + 1 \xrightarrow[n\to\infty]{} \infty.$$

No single constant C can dominate $2n+1$ for all n, so no universal converse exists. Every feature here depends only on t, proving the last sentence. □

The reason the certificate is loose in Proposition 15.7 is now visible: the feature t^{2n+1} is steep near the data (L_n large) but *flat* near the boundary $t = 0$, so the upper Lipschitz constant overstates how fast the score actually falls toward the boundary. Putting a *floor* under that fall, a co-Lipschitz bound (Remark 15.2), is exactly what closes the gap.

Theorem 15.8 (Two-sided robust-radius bracket: a conditional converse)**.** *Let g be continuous and L-Lipschitz in ℓ_2 on a convex set S containing x, its nearest boundary point, and the path γ below; let g be correct at x with margin $\mu = y\,g(x) > 0$, and write $\mathcal{B} = \{z : g(z) = 0\}$ for the decision boundary.*

(i) Lower arm: $r_2(g, x) \geq \mu/L$.

(ii) Upper arm: *if there is a continuous path $\gamma : [0, T] \to S$ with $\gamma(0) = x$, $g(\gamma(T)) = 0$, and a co-Lipschitz bound $|g(\gamma(t)) - g(x)| \geq \alpha \, \|\gamma(t) - x\|_2$ up to the first boundary hit, then $r_2(g, x) \leq \mu/\alpha$, and necessarily $\alpha \leq L$.*

Hence, with the local bi-Lipschitz condition number $\kappa := L/\alpha \geq 1$,

$$\frac{\mu}{L} \;\leq\; r_2(g, x) \;\leq\; \frac{\mu}{\alpha} = \kappa \cdot \frac{\mu}{L},$$

so the certificate determines the robust radius up to the factor κ, and is exact ($r_2 = \mu/L$) if and only if $\kappa = 1$.

Proof. Lower arm. For any admissible δ, the Lipschitz bound gives $|g(x+\delta) - g(x)| \leq L\,\|\delta\|_2$. If $\|\delta\|_2 < \mu/L$ then $|g(x+\delta) - g(x)| < \mu = y\,g(x)$, so $y\,g(x+\delta) > 0$ and the sign cannot flip. Hence no adversarial example exists inside radius μ/L, i.e. $r_2(g, x) \geq \mu/L$. *(this is the Part I certificate; restated for completeness)*

Upper arm. Evaluate the co-Lipschitz hypothesis at the first boundary hit $t = T$, where $g(\gamma(T)) = 0$:

$$\mu = |g(x)| = |g(x) - g(\gamma(T))| \;\geq\; \alpha\,\|\gamma(T) - x\|_2,$$

using $g(\gamma(T)) = 0$ in the middle and the co-Lipschitz floor on the right. Rearranging *(divide by $\alpha > 0$)*,

$$\|\gamma(T) - x\|_2 \;\leq\; \frac{\mu}{\alpha}.$$

Now $\gamma(T) \in \mathcal{B}$ is a point where the score is zero, so an arbitrarily small further push across $\mathcal{B}$ flips the prediction; the robust radius, being the *distance to the boundary set*, is at most the distance to this one boundary point: *(infimum over boundary points is at most one particular boundary point)*

$$r_2(g, x) = \operatorname{dist}(x, \mathcal{B}) \;\leq\; \|\gamma(T) - x\|_2 \;\leq\; \frac{\mu}{\alpha}.$$

The constants compare correctly. Along γ the same secant $|g(\gamma(t)) - g(x)|$ is floored by $\alpha\|\gamma(t) - x\|_2$ and capped by $L\|\gamma(t) - x\|_2$, so $\alpha\,\|\gamma(t) - x\|_2 \leq L\,\|\gamma(t) - x\|_2$, hence $\alpha \leq L$ and $\kappa = L/\alpha \geq 1$. Chaining the two arms gives the bracket, and equality $r_2 = \mu/L$ forces $\mu/\alpha = \mu/L$, i.e. $\kappa = 1$. □

Remark 15.9 (When is the upper arm usable?). The lower arm $r_2 \geq \mu/L$ is free (it needs only the Lipschitz constant). The upper arm needs an *existence* witness: one path to the boundary with a positive co-Lipschitz slope α. This is not automatic on an arbitrary trained network, so the bracket

is a *conditional* converse, and making it usable means producing such a path. Two cases where it is concrete: (i) for an explicit invariant feature, such as the power-spectrum quadratic surrogate below, α is computable in closed form on a ball, giving a numerical bracket; (ii) for a general differentiable score one can descend along $-\nabla g$ from x to the boundary and take α to be the smallest gradient norm met along that path, which is a valid (if loose) co-Lipschitz floor. The honest gap between "a conditional converse exists" and "a tight bracket on a trained CNN" is exactly the difficulty of certifying a good α, just as the lower arm's looseness is the difficulty of certifying a tight L.

Corollary 15.10 (Linear invariant features: the bracket is exact)**.** *If $g(x) = w^\top x$ with $w \in V^G$, then along the straight path $\gamma(t) = x - t\, w/\|w\|_2$ one has $L = \alpha = \|w\|_2$, so $\kappa = 1$ and*

$$r_2(g, x) = \frac{\mu}{\|w\|_2} = \frac{\eta}{\|w\|_2},$$

the exact point-to-hyperplane distance (Hein and Andriushchenko, 2017; Cortes and Vapnik, 1995).

Proof. For linear g, $|g(x) - g(z)| = |w^\top(x - z)| \le \|w\|_2 \, \|x - z\|_2$ by Cauchy–Schwarz, with equality exactly when $x - z$ is parallel to w. So the upper Lipschitz constant is $L = \|w\|_2$, and along the straight path in direction $-w$ the inequality is an equality, giving co-Lipschitz constant $\alpha = \|w\|_2$ as well. Thus $\kappa = 1$, and the path reaches the boundary at distance $\mu/\|w\|_2$; Theorem 15.8 pins this as the exact radius. The margin μ equals the feature margin η for the max-margin separator, giving the stated form. □

Remark 15.11 (Reconciling the no-converse counterexample with the bracket). The bracket is not contradicted by Proposition 15.7; it explains it. For $\Psi_n(x) = t^{2n+1}$ at $x = e$, the straight DC path to the origin has the feature drop from 1 to 0 over Euclidean distance 1, so its secant co-Lipschitz constant is $\alpha = 1$, while the upper Lipschitz on the segment is $L = 2n + 1$. The bracket reads $[\mu/L,\, \mu/\alpha] = [1/(2n + 1),\, 1]$ and correctly contains $r_2 = 1$, sitting at the μ/α end. The failure of a *universal* converse is precisely that $\kappa = 2n + 1$ is unbounded *across the model family*, not that any single model lacks a bracket. Note also the difference between the *pointwise* lower-Lipschitz $\inf_t |\Psi_n'|$, which vanishes at the boundary (a metric-subregularity failure (Dontchev and Rockafellar, 2009)) and so kills any derivative-based converse, and the *secant* co-Lipschitz above, which stays positive and yields the finite bracket.

Remark 15.12 (A computable instance). For the power-spectrum surrogate of Section 15.3, both arms are explicit: the upper Lipschitz satisfies $L \le 2B$ on the ball $\|x\|_2 \le B$, and along the gradient-descent path the co-Lipschitz constant is the minimum gradient norm $\alpha = \inf_{z \in \gamma} \|\nabla g(z)\|_2$, strictly positive as long as the path avoids the single critical point of the quadratic. This gives the first two-sided radius statement for an invariant-feature model, with measured condition number $\kappa \approx 1.6$–2.4 and the attack-found radius lying strictly inside the bracket for every seed.

15.3 Q2, deeper: a nonlinear invariant the linear one cannot match

The question this answers. Theorem 15.5 shows linear invariance is weak: for cyclic shifts it keeps only the mean. The natural next step is a *nonlinear* invariant feature that keeps more, while still being computable by a standard layer. The classical choice is the power spectrum $\{|\hat{x}_k|^2\}_k$, which is shift-invariant because a shift only rotates Fourier phases. The point of this subsection is twofold: first, a single convolution–square–global-pool layer literally *realizes* power-spectrum coordinates, so this is not an exotic feature but what such a layer already computes; second, the power spectrum is still blind to *phase*, and a degree-three invariant, the bispectrum, separates phase-coded classes that the power spectrum cannot, including DC-matched pairs that no shift-invariant linear feature separates either.

Status. Both facts are known-and-reproved. Power spectra, autocorrelation, and scattering invariants are classical (Bruna and Mallat, 2012); the realization lemma is a low-level architectural identity (with a corrected DFT normalization and real-filter handling). The bispectrum and its completeness-up-to-shift are classical (Kakarala, 2012). The only fresh angle is the *application*: using a higher-order invariant to defeat exactly the localized, phase-coded corner where Ge's forced trade-off bites, and quantifying its cost as a larger Lipschitz constant, hence a smaller η/L at equal margin.

Lemma 15.13 (A conv–square–pool layer realizes the power spectrum)**.** *Use the unitary DFT $\hat{x}_k = d^{-1/2}\sum_j x_j e^{-2\pi ijk/d}$. Define the unnormalised circular cross-correlation $(w \star x)_s = \sum_j w_j x_{j+s \bmod d}$ and the feature*

$$\Phi_w(x) := \frac{1}{d}\sum_{s=0}^{d-1}(w \star x)_s^2,$$

i.e. correlate x with a filter w, square the response, and global-average-pool. Then for real w, x,

$$\Phi_w(x) = \sum_{k=0}^{d-1} |\hat{w}_k|^2\,|\hat{x}_k|^2,$$

a nonnegative-weighted combination of the power-spectrum coordinates $|\hat{x}_k|^2$. Real filters can isolate every nonredundant real power coordinate ($|\hat{x}_0|^2$, $|\hat{x}_{d/2}|^2$ when d is even, and each conjugate-pair sum $|\hat{x}_k|^2 + |\hat{x}_{d-k}|^2$).

Proof. Step 1 (DFT of a correlation). By the convolution/correlation theorem for the unitary DFT, the transform of $w \star x$ is $\widehat{w \star x}_k = \sqrt{d}\,\overline{\hat{w}_k}\,\hat{x}_k$ *(the $\sqrt{d}$ is the unitary normalization; the conjugate is because correlation flips one argument).*

Step 2 (Parseval turns the pooled square into a frequency sum). The pooling sum is a squared ℓ_2 norm in space, and the unitary DFT preserves ℓ_2 norms (Parseval):

$$\Phi_w(x) = \frac{1}{d}\sum_s |(w \star x)_s|^2 = \frac{1}{d}\sum_k |\widehat{w \star x}_k|^2.$$

Step 3 (substitute Step 1). Insert $\widehat{w \star x}_k = \sqrt{d}\,\overline{\hat{w}_k}\hat{x}_k$ and take squared modulus:

$$\Phi_w(x) = \frac{1}{d}\sum_k \left|\sqrt{d}\,\overline{\hat{w}_k}\hat{x}_k\right|^2 = \frac{1}{d}\sum_k d\,|\hat{w}_k|^2|\hat{x}_k|^2 = \sum_k |\hat{w}_k|^2\,|\hat{x}_k|^2,$$

the $\sqrt{d}$ squaring to d and cancelling the $1/d$. This is the claimed identity.

Step 4 (which coordinates real filters reach). For real w, conjugate symmetry $\hat{w}_{d-k} = \overline{\hat{w}_k}$ forces $|\hat{w}_{d-k}|^2 = |\hat{w}_k|^2$, so a real filter weights $|\hat{x}_k|^2$ and $|\hat{x}_{d-k}|^2$ *equally*; it cannot separate a conjugate pair. But it *can* isolate a pair: choose $\hat{w}$ supported only on $\{k, d-k\}$ with conjugate values (which makes w real), giving exactly the pair sum $|\hat{x}_k|^2 + |\hat{x}_{d-k}|^2$; the self-conjugate coordinates $k = 0$ and (for even d) $k = d/2$ are isolated singly. Linear combinations of these give the stated real span. □

So one standard layer escapes the one-dimensional invariant linear family of Theorem 15.5 and sees the whole power spectrum. But the power spectrum still discards phase, and that is exactly what Ge's localized example exploits.

Proposition 15.14 (The bispectrum separates phase-coded classes the power spectrum cannot)**.** *The bispectrum $B_{k_1,k_2}(x) = \hat{x}_{k_1}\hat{x}_{k_2}\overline{\hat{x}_{k_1+k_2}}$ is shift-invariant and is realizable as a degree-three convolution–product–pool feature. There exist classes with* identical *power spectra that a single bispectrum coordinate separates: for Ge's dot, $x_+ = e_j$ and $x_- = -e_j$ have $|\hat{x}_k|^2 = 1/d$ for all k, so every power-spectrum feature is blind to the difference, yet*

$$B_{0,0}(e_j) = +d^{-3/2}, \qquad B_{0,0}(-e_j) = -d^{-3/2},$$

separating them with margin $d^{-3/2}$. (The linear DC invariant also separates this particular pair, through $f_{\mathrm{dc}}(\pm e_j) = \pm d^{-1/2}$; the sharper claim, proved in Step 3, is that the bispectrum separates any *two non-shift-equivalent orbits, including pairs with identical DC and identical power spectrum, which no shift-invariant linear feature can do.) This separability comes at the cost of a larger Lipschitz constant, $O(B^2)$ versus $O(B)$ for the power spectrum on the ball $\|x\|_2 \le B$, i.e. a smaller certified η/L at equal margin.*

Proof. Step 1 (shift-invariance of the bispectrum). Under $x \mapsto S_s x$ each Fourier coordinate picks up a unit phase, $\hat{x}_k \mapsto \omega^{sk}\hat{x}_k$ with $\omega = e^{-2\pi i/d}$ (the forward DFT convention of Part I; the shift theorem $\mathrm{DFT}(R_\tau x)_k = \omega^{k\tau}\hat{x}_k$). The three factors of B_{k_1,k_2} acquire phases ω^{sk_1}, ω^{sk_2}, and (from the conjugate) $\omega^{-s(k_1+k_2)}$, whose product is $\omega^{s(k_1+k_2-k_1-k_2)} = \omega^0 = 1$. The phases cancel, so B_{k_1,k_2} is invariant. It is a cubic form in x, hence realizable by circular correlations, a pointwise product, and global average pooling (a degree-three analogue of Lemma 15.13).

Step 2 (the dot has a flat power spectrum). For $x = e_j$ with the unitary DFT, $\hat{x}_k = d^{-1/2}\omega^{jk}$ where $\omega = e^{-2\pi i/d}$, so $|\hat{x}_k|^2 = 1/d$ for every k (the phase has modulus one). The same holds for $-e_j$. The two power spectra are identical, so by Lemma 15.13 no power-spectrum feature distinguishes them. A shift-invariant *linear* feature sees only the DC value (Theorem 15.5); for this particular pair the DC value $f_{\mathrm{dc}}(\pm e_j) = \pm d^{-1/2}$ does differ, so the linear invariant separates $\pm e_j$, but only with the weak $1/\sqrt{d}$ margin of the Ge collapse, and it would be *completely* blind to any DC-matched pair such as two shifts of a phase-coded signal.

Step 3 (a bispectrum coordinate separates them). Take $k_1 = k_2 = 0$. Then $B_{0,0}(x) = \hat{x}_0\hat{x}_0\overline{\hat{x}_0} = \hat{x}_0\,|\hat{x}_0|^2$. For e_j, $\hat{x}_0 = d^{-1/2}$ and $|\hat{x}_0|^2 = 1/d$, so $B_{0,0}(e_j) = d^{-1/2}\cdot d^{-1} = d^{-3/2}$. For $-e_j$ the sign of $\hat{x}_0$ flips, giving $-d^{-3/2}$. The single coordinate $B_{0,0}$ takes opposite signs on the two classes, separating them with margin $d^{-3/2}$. More generally, when all Fourier coefficients are nonzero the bispectrum determines the signal up to a global shift (Kakarala, 2012), so it separates any two shift-orbits that are not shift-equivalent.

Step 4 (the Lipschitz cost). A cubic form has gradient of order $\|x\|_2^2$, so on the ball $\|x\|_2 \le B$ the bispectrum is locally Lipschitz with constant $O(B^2)$, versus $O(B)$ for the quadratic power spectrum. At equal margin a larger L means a smaller certified η/L, which is the stated trade: the higher-order invariant buys separability of the phase-coded corner at the price of a smaller certificate. □

15.4 Q5 answered, part one: on orbit-closed data invariance is margin-free

The question this answers. The reproduction's quadrant diagnostic (Part II, Act II) plotted models on a consistency-versus-robustness plane and could not say *why* a model lands where it lands, nor whether building in invariance helps. Q5 asks for the optimization story. The first half of the answer is a surprising *negative* result: when the data is orbit-closed (every shift of a training point is also a training point with the same label), imposing exact shift-invariance does not change the max-margin value at all. Gradient descent on the unconstrained network already self-symmetrizes for free, so there is no margin to be gained by tying weights. This explains the experiment in which unconstrained and shift-tied networks reach essentially the same robustness on generic orbit data.

Status. A medium-strength novel candidate. It is the nonlinear-ReLU, orbit-closed analogue of the linear steerable/augmentation equivalence of Chen and Zhu (2023); the linear case is theirs, the ReLU min-norm accounting below is the new content. We use one architectural identity from Part I: a convolution-plus-global-pool unit is the same function as a two-layer net whose hidden weights are tied across the shift orbit of one filter (the "orbit bundle"). We also use the standard fact that a positively homogeneous ReLU unit $a\,\sigma(w^\top x)$ can be rescaled $(a, w) \mapsto (a/c,\, cw)$ without changing its function, so its minimum-norm representation costs $|a|\,\|w\|$ (taking $c^2 = |a|/\|w\|$ balances a^2 and $\|w\|^2$).

Theorem 15.15 (Invariance is margin-free on orbit-closed data)**.** *Let the data be orbit-closed with labels constant on orbits. For two-layer ReLU networks $f(x) = \sum_r a_r \sigma(w_r^\top x)$ (positively homogeneous, no bias), the minimum parameter norm $\frac{1}{2}\sum_r (a_r^2 + \|w_r\|^2)$ achieving unit margin is the* same *for the unconstrained class and for the shift-invariant (tied orbit-bundle) class. Equivalently, imposing exact shift-invariance changes neither the max-margin value nor the achievable margin.*

Proof. Write $C^\star_{\text{unc}}$ and $C^\star_{\text{inv}}$ for the two minimum costs. We show $C^\star_{\text{inv}} \geq C^\star_{\text{unc}}$ and $C^\star_{\text{inv}} \leq C^\star_{\text{unc}}$.

Step 1 ($\geq$, the easy direction). The tied (invariant) class is a *subset* of the unconstrained class: every invariant network is in particular some network. Minimizing the same cost over a smaller feasible set cannot give a smaller value, so $C^\star_{\text{inv}} \geq C^\star_{\text{unc}}$.

Step 2 (the symmetrization map). Let $f^\star = \sum_r a_r \sigma(w_r^\top x)$ be any unconstrained network with margin ≥ 1. Define its orbit average

$$f_{\text{sym}}(x) = \frac{1}{d}\sum_{s=0}^{d-1} f^\star(S_s x).$$

Reindexing the pooling sum shows f_{sym} is shift-invariant, and by the orbit-bundle identity it lies in the tied class.

Step 3 (symmetrization preserves the margin). Fix a training point (x_i, y_i). Because the data is orbit-closed with orbit-constant labels, each shifted point $S_s x_i$ is itself a training point with the same label y_i, so $y_i f^\star(S_s x_i) \geq 1$ for every s. Averaging over s *(average of numbers each ≥ 1 is ≥ 1)*,

$$y_i f_{\text{sym}}(x_i) = \frac{1}{d}\sum_s y_i f^\star(S_s x_i) \geq \frac{1}{d}\sum_s 1 = 1.$$

So f_{sym} still has unit margin.

Step 4 (the min-norm cost of one unit). For a single ReLU unit $a\,\sigma(w^\top x)$, positive homogeneity of σ means $a\,\sigma(w^\top x) = (a/c)\,\sigma((cw)^\top x)$ for any $c > 0$, so we may rescale freely. The cost $\frac{1}{2}((a/c)^2 + \|cw\|^2)$ is minimized over c by the arithmetic–geometric-mean balance at $c^2 = |a|/\|w\|$, giving minimal cost $|a|\,\|w\|$. *(this is the standard min-norm value of one homogeneous unit)*

Step 5 (the bundle realizing f_{sym} costs no more than $f^\star$). Expanding the average, $f_{\text{sym}}(x) = \frac{1}{d}\sum_r \sum_s a_r\,\sigma(w_r^\top S_s x) = \frac{1}{d}\sum_r \sum_s a_r\,\sigma((S_{-s} w_r)^\top x)$, using that S_s is orthogonal so $w_r^\top S_s x = (S_s^\top w_r)^\top x = (S_{-s} w_r)^\top x$. This is a network whose units are $\{(a_r/d,\ S_{-s} w_r)\}_{r,s}$. By Step 4 its min-norm cost is

$$\sum_r \sum_s \frac{|a_r|}{d}\,\|S_{-s} w_r\| = \sum_r \sum_s \frac{|a_r|}{d}\,\|w_r\| = \sum_r |a_r|\,\|w_r\|,$$

where the shift S_{-s} preserves the norm (orthogonality) and the inner sum over the d shifts of the factor $1/d$ collapses. But $\sum_r |a_r|\,\|w_r\|$ is exactly the min-norm cost of the original $f^\star$ by Step 4. Hence f_{sym} is realized in the tied class at cost no larger than the cost of $f^\star$.

Step 6 (conclude). Taking $f^\star$ to be a unconstrained *minimizer*, Step 5 produces a feasible invariant network of the same cost, so $C^\star_{\text{inv}} \le C^\star_{\text{unc}}$. With Step 1, the two are equal. There is therefore an invariant max-margin solution of the same norm: invariance is margin-free here. □

Remark 15.16 (What it does and does not give). Theorem 15.15 equalizes the *margin*; it does not say which max-margin solution gradient descent selects, and the certified radius η/L depends on the realized Lipschitz term L, not the margin alone. So invariance can still change η/L by changing L even when it cannot change η. That residual selection question, on data where the orbit-closed premise fails, is the content of the next subsection and its open conjecture.

15.5 Q5 answered, part two: in the lazy regime invariance lowers the Lipschitz constant

The question this answers. Theorem 15.15 says invariance cannot help *the margin* on orbit-closed data. But on data with near-orthogonal cluster geometry, experiment shows it *does* help η/L, and the gap is largest in the lazy (NTK) training regime. The remaining question is whether that advantage is provable. The answer is yes in the lazy, smooth-activation regime: there the trained network is the minimum-RKHS-norm interpolant of a fixed kernel, orbit-averaging is an orthogonal projection in that RKHS, and a projection cannot increase the norm, hence cannot increase the Lipschitz constant, at equal margin. So invariance buys robustness through a smaller L, exactly the lever Remark 15.16 left open.

Status. A medium-strength novel candidate, with an honestly limited scope. It proves the cluster conjecture *only* in the lazy, smooth-activation regime, by combining the lazy/NTK fixed-kernel picture (Jacot et al., 2018; Chizat et al., 2019) with the RKHS orbit-averaging projection of Elesedy (2021) and the invariant-kernel symmetrization of Haasdonk and Burkhardt (2007). The smooth-activation caveat is real and we state it: the bare two-layer ReLU NTK feature map is *not* Lipschitz (Bietti and Mairal, 2019), so the Lipschitz step needs a smooth activation; the rich (small-init) and bare-ReLU trajectories stay open. From Part I we use the lazy regime (a wide network behaves like a fixed kernel), the RKHS norm and its Lipschitz bound $|f(x) - f(y)| \le \|f\|_K \, L_\Phi \, \|x - y\|_2$, and orthogonal projection (a projection is non-expansive and gives a Pythagorean norm split).

Theorem 15.17 (Invariance lowers the Lipschitz constant in the lazy, smooth-activation regime)**.** *Work in the lazy/NTK regime, where each trained network equals the minimum-RKHS-norm interpolant of its fixed neural tangent kernel K (Jacot et al., 2018; Chizat et al., 2019), with a smooth activation so that the tangent feature map Φ is L_Φ-Lipschitz on the data ball (Bietti and Mairal, 2019). Let the data be orbit-closed with orbit-constant labels and let K be G-invariant, so the Reynolds average $\Pi_G f = \frac{1}{|G|}\sum_{g\in G} f(g\cdot)$ is defined on the RKHS $\mathcal{H}_K$ and the shift-tied bundle realizes exactly the invariant subspace $\mathcal{H}^G = \Pi_G \mathcal{H}_K$ with the restricted norm (Haasdonk and Burkhardt, 2007). Let f_{unc} be the unconstrained min-norm interpolant and f_{inv} the min-norm interpolant in $\mathcal{H}^G$. Then*

(i) *Π_G is the orthogonal projection of $\mathcal{H}_K$ onto $\mathcal{H}^G$, non-expansive, with $\|f\|_K^2 = \|\Pi_G f\|_K^2 + \|(I - \Pi_G)f\|_K^2$ (Elesedy, 2021);*

(ii) *$\Pi_G f_{\text{unc}}$ still interpolates with the same unit margin, so $\|f_{\text{inv}}\|_K \le \|\Pi_G f_{\text{unc}}\|_K \le \|f_{\text{unc}}\|_K$;*

(iii) *hence $L_{\text{inv}} \le L_\Phi \, \|f_{\text{inv}}\|_K \le L_\Phi \, \|f_{\text{unc}}\|_K$, and at the common unit margin the invariant model has the weakly larger certified ratio η/L. The gap is the discarded non-invariant energy $\|(I - \Pi_G)f_{\text{unc}}\|_K^2$ (the orbit-variance), strict unless f_{unc} is already invariant.*

Proof. (i) Π_G is an orthogonal projection. Because K is G-invariant, the averaging operator Π_G maps $\mathcal{H}_K$ into itself, and it is idempotent ($\Pi_G^2 = \Pi_G$, the same group-reindexing as in Theorem 15.5, Step 2) and self-adjoint on $\mathcal{H}_K$. An idempotent self-adjoint operator has spectrum in $\{0, 1\}$ and is the orthogonal projection onto its range, which is the invariant subspace $\mathcal{H}^G$. For an orthogonal projection, every f splits as $f = \Pi_G f + (I - \Pi_G)f$ into orthogonal pieces, so by the Pythagorean theorem $\|f\|_K^2 = \|\Pi_G f\|_K^2 + \|(I - \Pi_G)f\|_K^2$ (Elesedy, 2021); dropping the second nonnegative term gives non-expansiveness $\|\Pi_G f\|_K \le \|f\|_K$.

(ii) the projected interpolant is still feasible, with no larger norm. Orbit-constant labels mean $y_i f_{\text{unc}}(g \cdot x_i) \ge 1$ for every g (each $g \cdot x_i$ is a training point of the same label, interpolated by f_{unc}). Averaging over g,

$$y_i \, (\Pi_G f_{\text{unc}})(x_i) = \frac{1}{|G|} \sum_g y_i f_{\text{unc}}(g \cdot x_i) \ge 1,$$

so $\Pi_G f_{\text{unc}} \in \mathcal{H}^G$ is feasible for the invariant min-norm problem. Since f_{inv} is the *minimizer* of that problem, $\|f_{\text{inv}}\|_K \le \|\Pi_G f_{\text{unc}}\|_K$, and by (i) this is $\le \|f_{\text{unc}}\|_K$.

(iii) smaller norm gives smaller Lipschitz, hence larger η/L. For any RKHS function, the reproducing property and Cauchy–Schwarz give

$$|f(x) - f(y)| = |\langle f, \ \Phi(x) - \Phi(y)\rangle_K| \le \|f\|_K \, \|\Phi(x) - \Phi(y)\|_K \le \|f\|_K \, L_\Phi \, \|x - y\|_2,$$

the last step using that Φ is L_Φ-Lipschitz on the data ball (smooth activation) (Bietti and Mairal, 2019). So a valid Lipschitz constant is $L \le L_\Phi \, \|f\|_K$. Applying this to each interpolant and inserting (ii), $L_{\text{inv}} \le L_\Phi \, \|f_{\text{inv}}\|_K \le L_\Phi \, \|f_{\text{unc}}\|_K$. The two models share the same unit margin η, so the invariant model's certified η/L_{inv} is at least the unconstrained model's. The size of the gap is controlled by the discarded energy $\|(I - \Pi_G)f_{\text{unc}}\|_K^2$ from the split in (i): it is zero exactly when f_{unc} is already invariant, and strictly positive otherwise. □

Remark 15.18 (The ReLU caveat, stated honestly). Steps (i)–(ii) are activation-independent: they are pure RKHS geometry. Only step (iii) uses smoothness. The bare two-layer ReLU NTK feature map is *not* Lipschitz; its RKHS contains unit-norm functions of unbounded Lipschitz constant (Bietti and Mairal, 2019), so for ReLU one must either take a smooth activation or carry $\sup \|\nabla \Phi\|$ on the data ball explicitly. Within that scope the theorem proves the cluster conjecture, exactly in the regime where the project's experiments report the largest gap (orbit-variance ≈ 0.3, tied-over-unconstrained η/L up to $4.6\times$). The rich (small-init) regime and the bare-ReLU trajectory remain open.

15.6 The open frontier: the rich-regime cluster conjecture and how to attack it

Theorem 15.17 closes the lazy half. The rich (feature-learning, small-init) ReLU half is genuinely open. We state it as a conjecture, then, in the spirit of a primer, list the candidate proof paths and the Part I tools each one needs, so a collaborator can pick one up.

Conjecture 15.19 (Invariance helps on cluster geometry)**.** *On near-orthogonal cluster-geometry orbit data, the tied-invariant two-layer ReLU network's gradient flow reaches a strictly larger η/L than the unconstrained network's, with the gap growing as initialization moves toward the lazy regime. Equivalently, weight sharing plus pooling re-points the implicit bias toward the robust invariant solution.*

Why it is hard, in one line each: the orbit data is rank-structured, *not* the near-orthogonal $\Omega(d)$-cluster model whose non-robust selection is known, so that conclusion does not transfer; the tied program has its own constrained KKT conditions that must be written, not inherited; homogeneity gives convergence to *some* KKT point but not *which* one; and η/L depends on the realized Lipschitz term, so one must track the input-gradient norm along the trajectory, not just the functional margin.

Candidate proof paths and the tools each needs.

1. **Frequency-domain reduction (one orbit per class).** Replace the ReLU head by the conv–square–pool surrogate so the features are power-spectrum coordinates (Lemma 15.13); the invariant margin program then becomes a linear/quadratic max-margin in Fourier-magnitude space, possibly solvable in closed form, and one compares its η/L to the unconstrained KKT point on the same data, then swaps the surrogate for ReLU. *Tools:* the *Fourier/power-spectrum invariants* and the *convex-hull max-margin* geometry, both introduced in the invariance and learning clusters of Part I.

2. **Explicit KKT in Fourier coordinates.** Use the circulant/DFT diagonalization of the shift action so that frequencies decouple, write the unconstrained and constrained KKT systems per frequency, and solve frequency by frequency. *Tools:* the *implicit-bias/KKT machinery* for homogeneous networks (Lyu and Li, 2020; Ji and Telgarsky, 2019; Soudry et al., 2018) from the learning cluster of Part I, plus the *per-irreducible (DFT) decomposition* of the shift group from the invariance cluster.

3. **Rich versus lazy contrast.** Run the same construction at small init (rich, feature-learning) and at NTK scale (lazy), and show the selected η/L differs, with the invariant constraint mattering most in the rich regime. *Tools:* the *NTK/lazy-versus-rich* distinction from the learning cluster of Part I; the lazy end is already pinned by Theorem 15.17.

4. **Bound the two ends separately.** Lower-bound the invariant network's achieved margin by the projected convex-hull margin of Theorem 15.5 in feature space, upper-bound the unconstrained network's η/L via a feature-averaging direction, then compare. *Tools:* the *projection-margin accounting* of Section 15.1 and the *feature-averaging implicit-bias* results (Li et al., 2024) from the learning cluster.

Each route also needs the supporting setup lemmas, the bare η/L lower bound, the orbit-rank and orbit-bundle identities, and the KKT scaffolding; we have stated here only the load-bearing results and leave the routine scaffolding to the reader who picks up a route.

15.7 Tying theory to the benchmark: the capacity-matched η/L decomposition

The question this answers. Q1 and Q4: if shift-consistency does not order robustness, what single measurable quantity does, and does the answer survive a strong attack at matched capacity? The empirical headline is that η/L, computed in the invariant feature space, is that quantity, and the theory above predicts *which* of its two parts a given architecture moves.

Status. This is the best empirical candidate, and its framing is the honest one of Remark 15.1: not "does invariance help?" but "when capacity is controlled, which term, margin η or Lipschitz L, does a given invariance move?" Robustness is measured by a standardized strong attack (AutoAttack) at matched parameter counts (Croce and Hein, 2020; Croce et al., 2020), with capacity controlled as in architecture-robustness studies (Peng et al., 2023).

The findings, briefly. Across capacity-matched arms (standard, anti-aliased, exact-cyclic, shift-augmented, identical parameter counts), η/L versus the ℓ_2 robust radius tracks at Pearson 0.998, while shift-consistency *anti*-predicts robustness. The decomposition attributes mechanism exactly as the theory says it should: anti-aliasing raises η/L by lowering the Lipschitz term L (Theorem 15.17's lever), and exact cyclic invariance lowers η/L by shrinking the margin η (the real-data form of the

Ge collapse of Section 15.1). Under adversarial training, where AutoAttack becomes informative with no gradient masking, shift-consistency still anti-predicts robustness (Pearson -0.88), and the *threat-matched* ratio $\eta/\|\nabla M\|_1$, using the ℓ_∞ dual norm, predicts AutoAttack robust accuracy at Pearson 0.90, whereas the mismatched ℓ_2 ratio does not. The two laws replicate on MNIST and Fashion-MNIST and under an ℓ_2 threat. This is the validated margin/Lipschitz decomposition that turns the contradictory "helps or hurts" literature into a single sharp question with a measurable answer.

References

Bassam Bamieh. Discovering transforms: a tutorial on circulant matrices, circular convolution, and the discrete fourier transform, 2018. arXiv:1805.05533.

Kristin P. Bennett and Erin J. Bredensteiner. Duality and geometry in SVM classifiers. In *International Conference on Machine Learning (ICML)*, 2000.

Alberto Bietti and Julien Mairal. On the inductive bias of neural tangent kernels. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. arXiv:1905.12173.

Bernhard E. Boser, Isabelle M. Guyon, and Vladimir N. Vapnik. A training algorithm for optimal margin classifiers. In *Workshop on Computational Learning Theory (COLT)*, 1992.

Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: grids, groups, graphs, geodesics, and gauges, 2021. arXiv:2104.13478.

Joan Bruna and Stéphane Mallat. Invariant scattering convolution networks, 2012. arXiv:1203.1513.

Christopher J. C. Burges. A tutorial on support vector machines for pattern recognition. *Data Mining and Knowledge Discovery*, 2(2):121–167, 1998.

Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In *IEEE Symposium on Security and Privacy (S&P)*, 2017. arXiv:1608.04644.

Hua Chen, Mona Zehni, and Zhizhen Zhao. Multi-target detection and bispectrum inversion, 2018.

Ziyu Chen and Wei Zhu. On the implicit bias of linear equivariant steerable networks, 2023. arXiv:2303.04198.

Lénaïc Chizat, Edouard Oyallon, and Francis Bach. On lazy training in differentiable programming. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. arXiv:1812.07956.

Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.

Francesco Croce and Matthias Hein. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. In *International Conference on Machine Learning (ICML)*, 2020. arXiv:2003.01690.

Francesco Croce, Maksym Andriushchenko, Vikash Sehwag, Edoardo Debenedetti, Nicolas Flammarion, Mung Chiang, Prateek Mittal, and Matthias Hein. RobustBench: a standardized adversarial robustness benchmark, 2020. arXiv:2010.09670.

Pierre de Buyl. Derivation of the wiener–khinchin theorem. online note, n.d. accessed 2026.

Asen L. Dontchev and R. Tyrrell Rockafellar. *Implicit Functions and Solution Mappings: A View from Variational Analysis*. Springer, 2009.

Bryn Elesedy. Provably strict generalisation benefit for invariance in kernel methods. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2106.02346.

Spencer Frei, Gal Vardi, Peter L. Bartlett, and Nathan Srebro. The double-edged sword of implicit bias: generalization vs. robustness in ReLU networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.01456.

Angus Galloway, Anna Golubeva, Thomas Tanay, Medhat Moussa, and Graham W. Taylor. Batch normalization is a cause of adversarial vulnerability. In *ICML Workshop on Identifying and Understanding Deep Learning Phenomena*, 2019. arXiv:1905.02161.

Paul Garrett. Projectors, projection maps, orthogonal projections. University of Minnesota lecture notes, n.d. accessed 2026.

Songwei Ge, Vasu Singla, Ronen Basri, and David Jacobs. Shift invariance can reduce adversarial robustness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2103.02695.

Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6572.

Roe Goodman. The discrete fourier transform and wavelets (Math 357). Rutgers University lecture notes, n.d. accessed 2026.

Bernard Haasdonk and Hans Burkhardt. Invariant kernel functions for pattern analysis and machine learning. *Machine Learning*, 68(1):35–61, 2007.

Matthias Hein and Maksym Andriushchenko. Formal guarantees on the robustness of a classifier against adversarial manipulation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv:1705.08475.

Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: convergence and generalization in neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. arXiv:1806.07572.

Ziwei Ji and Matus Telgarsky. The implicit bias of gradient descent on nonseparable data. In *Conference on Learning Theory (COLT)*, 2019.

Steven G. Johnson. Notes on circulant matrices (MIT 18.06). MIT 18.06 lecture notes, n.d. accessed 2026.

Ramakrishna Kakarala. The bispectrum as a source of phase-sensitive invariants for Fourier descriptors: a group-theoretic approach. *Journal of Mathematical Imaging and Vision*, 2012. arXiv:0902.0196.

Sandesh Kamath, Amit Deshpande, and K. V. Subrahmanyam. Invariance vs. robustness of neural networks. In *ICLR (workshop/OpenReview)*, 2020.

Sandesh Kamath, Amit Deshpande, K. V. Subrahmanyam, and Vineeth N. Balasubramanian. Can we have it all? on the trade-off between spatial and adversarial robustness of neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2002.11318.

J. Zico Kolter and Aleksander Madry. Adversarial robustness: theory and practice. NeurIPS tutorial, https://adversarial-ml-tutorial.org, 2018.

Hannah Lawrence, Kristian Georgiev, Andrew Dienes, and Bobak T. Kiani. Implicit bias of linear equivariant networks, 2021. arXiv:2110.06084.

Binghui Li, Zhixuan Pan, Kaifeng Lyu, and Jian Li. Feature averaging: an implicit bias of gradient descent leading to non-robustness in neural networks. *arXiv:2410.10322*, 2024.

Kaifeng Lyu and Jian Li. Gradient descent maximizes the margin of homogeneous neural networks. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1906.05890.

Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1706.06083.

Hancheng Min and René Vidal. Can implicit bias imply adversarial robustness? In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2405.15942.

MIT OpenCourseWare. Power spectral density (MIT 6.011). 6.011 lecture notes, n.d. accessed 2026.

ShengYun Peng, Weilin Xu, Cory Cornelius, Kevin Li, Rahul Duggal, Duen Horng Chau, and Jason Martin. RobArch: designing robust architectures against adversarial attacks, 2023. arXiv:2301.03110.

Jérôme Rony, Luiz G. Hafemann, Luiz S. Oliveira, Ismail Ben Ayed, Robert Sabourin, and Eric Granger. Decoupling direction and norm for efficient gradient-based ℓ_2 adversarial attacks and defenses. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. arXiv:1811.09600.

Sourajit Saha and Tejas Gokhale. Improving shift invariance in convolutional neural networks with translation invariant polyphase sampling (TIPS). In *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2024. arXiv:2404.07410.

Kevin Scaman and Aladin Virmaux. Lipschitz regularity of deep neural networks: analysis and efficient estimation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. arXiv:1805.10965.

Daniel Soudry, Elad Hoffer, Mor Shpigel Nacson, Suriya Gunasekar, and Nathan Srebro. The implicit bias of gradient descent on separable data. *Journal of Machine Learning Research (JMLR)*, 2018. arXiv:1710.10345.

Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. In *International Conference on Learning Representations (ICLR)*, 2014. arXiv:1312.6199.

Texas A&M University. Orthogonal projections (math 423, lecture 36). MATH 423 lecture notes, n.d. accessed 2026.

Florian Tramèr, Jens Behrmann, Nicholas Carlini, Nicolas Papernot, and Jörn-Henrik Jacobsen. Fundamental tradeoffs between invariance and sensitivity to adversarial perturbations. In *International Conference on Machine Learning (ICML)*, 2020. arXiv:2002.04599.

Yusuke Tsuzuku, Issei Sato, and Masashi Sugiyama. Lipschitz-margin training: scalable certification of perturbation invariance for deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. arXiv:1802.04034.

Vladimir N. Vapnik. *Statistical Learning Theory*. Wiley, 1998.

Longwei Wang, Ifrat Ikhtear Uddin, K. C. Santosh, Chaowei Zhang, Xiao Qin, and Yang Zhou. Bridging symmetry and robustness: on the role of equivariance in enhancing adversarial robustness, 2025. arXiv:2510.16171.

Wikipedia. Bispectrum. https://en.wikipedia.org/wiki/Bispectrum, n.d.a. accessed 2026.

Wikipedia. Convolution theorem. https://en.wikipedia.org/wiki/Convolution_theorem, n.d.b. accessed 2026.

Wikipedia. Equivariant map. https://en.wikipedia.org/wiki/Equivariant_map, n.d.c. accessed 2026.

Wikipedia. Group action. https://en.wikipedia.org/wiki/Group_action, n.d.d. accessed 2026.

Wikipedia. Lipschitz continuity. https://en.wikipedia.org/wiki/Lipschitz_continuity, n.d.e. accessed 2026.

Wikipedia. Projection (linear algebra). https://en.wikipedia.org/wiki/Projection_(linear_algebra), n.d.f. accessed 2026.

Richard Zhang. Making convolutional networks shift-invariant again. In *International Conference on Machine Learning (ICML)*, 2019. arXiv:1904.11486.